

Dist Diff

from: 07785a8508d67e0146478490498061b185ce9264 (07785a8508d6)

to: d5782cac35fee0ba4345335b70bb80d553b90e37 (d5782cac35fe)

changed rendered sections: 4

skipped binary/non-UTF-8 files: 0

Changed Files

M index.html (6954 B / f1156eae69fa → 77433 B / 8ab941e91919)

M main.js (63353 B / 9bee5a3328fc → 33823 B / 04a8e344941e)

D main.js :: inline worker jsContent #1 (52396 B / 865fe4169163 → -)

M style.css (11844 B / 017d5c46ea47 → 14083 B / 3bb92984b4f4)

==== M index.html (6954 B / f1156eae69fa → 77433 B / 8ab941e91919) ====

```
1 1 | <!doctype html>
2 2 | <html lang="ko">
3 3 | <head>
4 4 | <meta charset="UTF-8" />
5 5 | <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 6 | <title>TS Chess</title>
7 - | <script type="module" crossorigin src="/./main.js"></script>
8 - | <link rel="stylesheet" crossorigin href="/./style.css" />
9 + | <script id="ts-chess-shared-core-source" type="text/plain">
10 + |   var TSCSChessShared = (function (exports) {
11 + |     'use strict';
12 + |     const MASK64 = 0xffffffffffffffff;
13 + |     function rotl(x, k) {
14 + |       return ((x << k) | (x >> (64n - k))) & 0xffffffffffffffff;
15 + |     }
16 + |     function wrappingMul(x, y) {
17 + |       return (x * y) & MASK64;
18 + |     }
19 + |     function xoroshiro128(state) {
20 + |       return function () {
21 + |         let s0 = BigInt(state & MASK64);
22 + |         let s1 = BigInt((state >> 64n) & MASK64);
23 + |         const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
24 + |         s1 ^= s0;
25 + |         s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64;
26 + |         s1 = rotl(s1, 37n);
27 + |         state = (s1 << 64n) | s0;
28 + |         return result;
29 + |       };
30 + |     }
31 + |     const rand = xoroshiro128(0xa187eb39cdcaed8f31c4b365b102e01en);
32 + |     const PIECE_KEYS = Array.from({ length: 2 }, () =>
33 + |       Array.from({ length: 6 }, () => Array.from({ length: 128 }, () => rand())));
34 + |     const EP_KEYS = Array.from({ length: 8 }, () => rand());
35 + |     const CASTLING_KEYS = Array.from({ length: 16 }, () => rand());
36 + |     const SIDE_KEY = rand();
37 + |     const WHITE = 'w';
38 + |     const BLACK = 'b';
39 + |     const PAWN = 'p';
40 + |     const KNIGHT = 'n';
41 + |     const BISHOP = 'b';
42 + |     const ROOK = 'r';
43 + |     const QUEEN = 'q';
44 + |     const KING = 'k';
45 + |     const DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1';
46 + |     class Move {
47 + |       color;
48 + |       from;
49 + |       to;
50 + |       piece;
51 + |       captured;
52 + |       promotion;
53 + |       /**
54 + |        * @deprecated This field is deprecated and will be removed in version 2.0.0.
55 + |        * Please use move descriptor functions instead: `isCapture`, `isPromotion`,
```

```

55 + | | | | *`isEnPassant`,`isKingsideCastle`,`isQueensideCastle`,`isCastle`,and
56 + | | | | *`isBigPawn`
57 + | | | | */
58 + | | | | flags;
59 + | | | | san;
60 + | | | | lan;
61 + | | | | before;
62 + | | | | after;
63 + | | | | constructor(chess, internal){
64 + | | | |   const{ color, piece, from, to, flags, captured, promotion }= internal;
65 + | | | |   const fromAlgebraic = algebraic(from);
66 + | | | |   const toAlgebraic = algebraic(to);
67 + | | | |   this.color = color;
68 + | | | |   this.piece = piece;
69 + | | | |   this.from = fromAlgebraic;
70 + | | | |   this.to = toAlgebraic;
71 + | | | |   this.san = chess['_moveToSan'](internal, chess['_moves']({ legal: true }));
72 + | | | |   this.lan = fromAlgebraic + toAlgebraic;
73 + | | | |   this.before = chess.fen();
74 + | | | |   chess['_makeMove'](internal);
75 + | | | |   this.after = chess.fen();
76 + | | | |   chess['_undoMove']();
77 + | | | |   this.flags = '';
78 + | | | |   for (const flag in BITS){
79 + | | | |     if (BITS[flag] & flags){
80 + | | | |       this.flags += FLAGS[flag];
81 + | | | |     }
82 + | | | |   }
83 + | | | |   if (captured){
84 + | | | |     this.captured = captured;
85 + | | | |   }
86 + | | | |   if (promotion){
87 + | | | |     this.promotion = promotion;
88 + | | | |     this.lan += promotion;
89 + | | | |   }
90 + | | | | }
91 + | | | | isCapture(){
92 + | | | |   return this.flags.indexOf(FLAGS['CAPTURE']) > -1;
93 + | | | | }
94 + | | | | isPromotion(){
95 + | | | |   return this.flags.indexOf(FLAGS['PROMOTION']) > -1;
96 + | | | | }
97 + | | | | isEnPassant(){
98 + | | | |   return this.flags.indexOf(FLAGS['EP_CAPTURE']) > -1;
99 + | | | | }
100 + | | | | isKingsideCastle(){
101 + | | | |   return this.flags.indexOf(FLAGS['KSIDE_CASTLE']) > -1;
102 + | | | | }
103 + | | | | isQueensideCastle(){
104 + | | | |   return this.flags.indexOf(FLAGS['QSIDE_CASTLE']) > -1;
105 + | | | | }
106 + | | | | isBigPawn(){
107 + | | | |   return this.flags.indexOf(FLAGS['BIG_PAWN']) > -1;
108 + | | | | }
109 + | | | | }
110 + | | | | const EMPTY = -1;
111 + | | | | const FLAGS = {
112 + | | | |   NORMAL: 'n',
113 + | | | |   CAPTURE: 'c',
114 + | | | |   BIG_PAWN: 'b',
115 + | | | |   EP_CAPTURE: 'e',
116 + | | | |   PROMOTION: 'p',
117 + | | | |   KSIDE_CASTLE: 'k',
118 + | | | |   QSIDE_CASTLE: 'q',
119 + | | | |   NULL_MOVE: '-',
120 + | | | | };
121 + | | | | const SQUARES = [
122 + | | | |   'a8',
123 + | | | |   'b8',
124 + | | | |   'c8',
125 + | | | |   'd8',

```

```

126 + | | | | | 'e8',
127 + | | | | | 'f8',
128 + | | | | | 'g8',
129 + | | | | | 'h8',
130 + | | | | | 'a7',
131 + | | | | | 'b7',
132 + | | | | | 'c7',
133 + | | | | | 'd7',
134 + | | | | | 'e7',
135 + | | | | | 'f7',
136 + | | | | | 'g7',
137 + | | | | | 'h7',
138 + | | | | | 'a6',
139 + | | | | | 'b6',
140 + | | | | | 'c6',
141 + | | | | | 'd6',
142 + | | | | | 'e6',
143 + | | | | | 'f6',
144 + | | | | | 'g6',
145 + | | | | | 'h6',
146 + | | | | | 'a5',
147 + | | | | | 'b5',
148 + | | | | | 'c5',
149 + | | | | | 'd5',
150 + | | | | | 'e5',
151 + | | | | | 'f5',
152 + | | | | | 'g5',
153 + | | | | | 'h5',
154 + | | | | | 'a4',
155 + | | | | | 'b4',
156 + | | | | | 'c4',
157 + | | | | | 'd4',
158 + | | | | | 'e4',
159 + | | | | | 'f4',
160 + | | | | | 'g4',
161 + | | | | | 'h4',
162 + | | | | | 'a3',
163 + | | | | | 'b3',
164 + | | | | | 'c3',
165 + | | | | | 'd3',
166 + | | | | | 'e3',
167 + | | | | | 'f3',
168 + | | | | | 'g3',
169 + | | | | | 'h3',
170 + | | | | | 'a2',
171 + | | | | | 'b2',
172 + | | | | | 'c2',
173 + | | | | | 'd2',
174 + | | | | | 'e2',
175 + | | | | | 'f2',
176 + | | | | | 'g2',
177 + | | | | | 'h2',
178 + | | | | | 'a1',
179 + | | | | | 'b1',
180 + | | | | | 'c1',
181 + | | | | | 'd1',
182 + | | | | | 'e1',
183 + | | | | | 'f1',
184 + | | | | | 'g1',
185 + | | | | | 'h1',
186 + | | | | | ];
187 + | | | | | const BITS = {
188 + | | | | |     NORMAL: 1,
189 + | | | | |     CAPTURE: 2,
190 + | | | | |     BIG_PAWN: 4,
191 + | | | | |     EP_CAPTURE: 8,
192 + | | | | |     PROMOTION: 16,
193 + | | | | |     KSIDE_CASTLE: 32,
194 + | | | | |     QSIDE_CASTLE: 64,
195 + | | | | |     NULL_MOVE: 128,
196 + | | | | | };

```

```

197 + | | | | | const 0x88 = {
198 + | | | | | a8: 0,
199 + | | | | | b8: 1,
200 + | | | | | c8: 2,
201 + | | | | | d8: 3,
202 + | | | | | e8: 4,
203 + | | | | | f8: 5,
204 + | | | | | g8: 6,
205 + | | | | | h8: 7,
206 + | | | | | a7: 16,
207 + | | | | | b7: 17,
208 + | | | | | c7: 18,
209 + | | | | | d7: 19,
210 + | | | | | e7: 20,
211 + | | | | | f7: 21,
212 + | | | | | g7: 22,
213 + | | | | | h7: 23,
214 + | | | | | a6: 32,
215 + | | | | | b6: 33,
216 + | | | | | c6: 34,
217 + | | | | | d6: 35,
218 + | | | | | e6: 36,
219 + | | | | | f6: 37,
220 + | | | | | g6: 38,
221 + | | | | | h6: 39,
222 + | | | | | a5: 48,
223 + | | | | | b5: 49,
224 + | | | | | c5: 50,
225 + | | | | | d5: 51,
226 + | | | | | e5: 52,
227 + | | | | | f5: 53,
228 + | | | | | g5: 54,
229 + | | | | | h5: 55,
230 + | | | | | a4: 64,
231 + | | | | | b4: 65,
232 + | | | | | c4: 66,
233 + | | | | | d4: 67,
234 + | | | | | e4: 68,
235 + | | | | | f4: 69,
236 + | | | | | g4: 70,
237 + | | | | | h4: 71,
238 + | | | | | a3: 80,
239 + | | | | | b3: 81,
240 + | | | | | c3: 82,
241 + | | | | | d3: 83,
242 + | | | | | e3: 84,
243 + | | | | | f3: 85,
244 + | | | | | g3: 86,
245 + | | | | | h3: 87,
246 + | | | | | a2: 96,
247 + | | | | | b2: 97,
248 + | | | | | c2: 98,
249 + | | | | | d2: 99,
250 + | | | | | e2: 100,
251 + | | | | | f2: 101,
252 + | | | | | g2: 102,
253 + | | | | | h2: 103,
254 + | | | | | a1: 112,
255 + | | | | | b1: 113,
256 + | | | | | c1: 114,
257 + | | | | | d1: 115,
258 + | | | | | e1: 116,
259 + | | | | | f1: 117,
260 + | | | | | g1: 118,
261 + | | | | | h1: 119,
262 + | | | | | };
263 + | | | | | const PAWN_OFFSETS = {
264 + | | | | | b: [16, 32, 17, 15],
265 + | | | | | w: [-16, -32, -17, -15],
266 + | | | | | };
267 + | | | | | const PIECE_OFFSETS = {

```

```

268 + | | | | | n: [-18, -33, -31, -14, 18, 33, 31, 14],
269 + | | | | | b: [-17, -15, 17, 15],
270 + | | | | | r: [-16, 1, 16, -1],
271 + | | | | | q: [-17, -16, -15, 1, 17, 16, 15, -1],
272 + | | | | | k: [-17, -16, -15, 1, 17, 16, 15, -1],
273 + | | | | | };
274 + | | | | | const ATTACKS = [
275 + | | | | | 20, 0, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20,
276 + | | | | | 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 20,
277 + | | | | | 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20,
278 + | | | | | 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 24, 24, 24, 24, 24, 24, 56,
279 + | | | | | 0, 56, 24, 24, 24, 24, 24, 24, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
280 + | | | | | 0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0,
281 + | | | | | 0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0,
282 + | | | | | 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0,
283 + | | | | | 20,
284 + | | | | | ];
285 + | | | | | const RAYS = [
286 + | | | | | 17, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15,
287 + | | | | | 0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15, 0, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0, 0, 15,
288 + | | | | | 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 16, 0, 15,
289 + | | | | | 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, -1,
290 + | | | | | -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
291 + | | | | | -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0, 0, -17, 0, 0, 0, 0, 0, 0,
292 + | | | | | 0, 0, -15, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, 0, -17,
293 + | | | | | 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, 0, -17, 0, 0, -15, 0, 0, 0, 0, 0, 0, -16, 0, 0,
294 + | | | | | 0, 0, 0, 0, -17,
295 + | | | | | ];
296 + | | | | | const PIECE_MASKS = { p: 1, n: 2, b: 4, r: 8, q: 16, k: 32 };
297 + | | | | | const SYMBOLS = 'pnbrqkPNBRQK';
298 + | | | | | const PROMOTIONS = [KNIGHT, BISHOP, ROOK, QUEEN];
299 + | | | | | const RANK_1 = 7;
300 + | | | | | const RANK_2 = 6;
301 + | | | | | const RANK_7 = 1;
302 + | | | | | const RANK_8 = 0;
303 + | | | | | const ROOKS = {
304 + | | | | | w: [
305 + | | | | | { square: 0x88.a1, flag: BITS.QSIDE_CASTLE },
306 + | | | | | { square: 0x88.h1, flag: BITS.KSIDE_CASTLE },
307 + | | | | | ],
308 + | | | | | b: [
309 + | | | | | { square: 0x88.a8, flag: BITS.QSIDE_CASTLE },
310 + | | | | | { square: 0x88.h8, flag: BITS.KSIDE_CASTLE },
311 + | | | | | ],
312 + | | | | | };
313 + | | | | | const SECOND_RANK = { b: RANK_7, w: RANK_2 };
314 + | | | | | function rank(square) {
315 + | | | | | return square >> 4;
316 + | | | | | }
317 + | | | | | function file(square) {
318 + | | | | | return square & 15;
319 + | | | | | }
320 + | | | | | function isDigit(c) {
321 + | | | | | return '0123456789'.indexOf(c) !== -1;
322 + | | | | | }
323 + | | | | | function algebraic(square) {
324 + | | | | | const f = file(square);
325 + | | | | | const r = rank(square);
326 + | | | | | return 'abcdefgh'.substring(f, f + 1) + '87654321'.substring(r, r + 1);
327 + | | | | | }
328 + | | | | | function swapColor(color) {
329 + | | | | | return color === WHITE ? BLACK : WHITE;
330 + | | | | | }
331 + | | | | | function validateFen(fen) {
332 + | | | | | const tokens = fen.split(/\s+/);
333 + | | | | | if (tokens.length !== 6) {
334 + | | | | | return {
335 + | | | | | ok: false,
336 + | | | | | error: 'Invalid FEN: must contain six space-delimited fields',
337 + | | | | | };
338 + | | | | | }

```

```

339 + | | | | | const moveNumber = parseInt(tokens[5], 10);
340 + | | | | | if (isNaN(moveNumber) || moveNumber ≤ 0) {
341 + | | | | |   return {
342 + | | | | |     ok: false,
343 + | | | | |     error: 'Invalid FEN: move number must be a positive integer',
344 + | | | | |   };
345 + | | | | | }
346 + | | | | | const halfMoves = parseInt(tokens[4], 10);
347 + | | | | | if (isNaN(halfMoves) || halfMoves < 0) {
348 + | | | | |   return {
349 + | | | | |     ok: false,
350 + | | | | |     error: 'Invalid FEN: half move counter number must be a non-negative integer',
351 + | | | | |   };
352 + | | | | | }
353 + | | | | | if (!/^(?![abcdefgh][36])$/ .test(tokens[3])) {
354 + | | | | |   return { ok: false, error: 'Invalid FEN: en-passant square is invalid' };
355 + | | | | | }
356 + | | | | | if (/^[^kKqQ-]/ .test(tokens[2])) {
357 + | | | | |   return { ok: false, error: 'Invalid FEN: castling availability is invalid' };
358 + | | | | | }
359 + | | | | | if (!/^(w|b)$/ .test(tokens[1])) {
360 + | | | | |   return { ok: false, error: 'Invalid FEN: side-to-move is invalid' };
361 + | | | | | }
362 + | | | | | const rows = tokens[0].split('/');
363 + | | | | | if (rows.length !== 8) {
364 + | | | | |   return {
365 + | | | | |     ok: false,
366 + | | | | |     error: 'Invalid FEN: piece data does not contain 8 '/'-delimited rows',
367 + | | | | |   };
368 + | | | | | }
369 + | | | | | for (let i = 0; i < rows.length; i++) {
370 + | | | | |   let sumFields = 0;
371 + | | | | |   let previousWasNumber = false;
372 + | | | | |   for (let k = 0; k < rows[i].length; k++) {
373 + | | | | |     if (isDigit(rows[i][k])) {
374 + | | | | |       if (previousWasNumber) {
375 + | | | | |         return {
376 + | | | | |           ok: false,
377 + | | | | |           error: 'Invalid FEN: piece data is invalid (consecutive number)',
378 + | | | | |         };
379 + | | | | |       }
380 + | | | | |       sumFields += parseInt(rows[i][k], 10);
381 + | | | | |       previousWasNumber = true;
382 + | | | | |     } else {
383 + | | | | |       if (!/^[prnbqkPRNBQK]$/ .test(rows[i][k])) {
384 + | | | | |         return {
385 + | | | | |           ok: false,
386 + | | | | |           error: 'Invalid FEN: piece data is invalid (invalid piece)',
387 + | | | | |         };
388 + | | | | |       }
389 + | | | | |       sumFields += 1;
390 + | | | | |       previousWasNumber = false;
391 + | | | | |     }
392 + | | | | |   }
393 + | | | | |   if (sumFields !== 8) {
394 + | | | | |     return {
395 + | | | | |       ok: false,
396 + | | | | |       error: 'Invalid FEN: piece data is invalid (too many squares in rank)',
397 + | | | | |     };
398 + | | | | |   }
399 + | | | | | }
400 + | | | | | if ((tokens[3][1] === '3' & tokens[1] === 'w') || (tokens[3][1] === '6' & tokens[1] === 'b')) {
401 + | | | | |   return { ok: false, error: 'Invalid FEN: illegal en-passant square' };
402 + | | | | | }
403 + | | | | | const kings = [
404 + | | | | |   { color: 'white', regex: /K/g },
405 + | | | | |   { color: 'black', regex: /k/g },
406 + | | | | | ];
407 + | | | | | for (const { color, regex } of kings) {
408 + | | | | |   if (!regex.test(tokens[0])) {
409 + | | | | |     return { ok: false, error: `Invalid FEN: missing ${color} king` };

```

```

410 + | | | | | }
411 + | | | | | if ((tokens[0].match(regex) || []).length > 1) {
412 + | | | | |   return { ok: false, error: `Invalid FEN: too many ${color} kings` };
413 + | | | | | }
414 + | | | | | }
415 + | | | | | if (Array.from(rows[0] + rows[7]).some((char) => char.toUpperCase() === 'P')) {
416 + | | | | |   return {
417 + | | | | |     ok: false,
418 + | | | | |     error: 'Invalid FEN: some pawns are on the edge rows',
419 + | | | | |   };
420 + | | | | | }
421 + | | | | | return { ok: true };
422 + | | | | | }
423 + | | | | | function getDisambiguator(move, moves) {
424 + | | | | |   const from = move.from;
425 + | | | | |   const to = move.to;
426 + | | | | |   const piece = move.piece;
427 + | | | | |   let ambiguities = 0;
428 + | | | | |   let sameRank = 0;
429 + | | | | |   let sameFile = 0;
430 + | | | | |   for (let i = 0, len = moves.length; i < len; i++) {
431 + | | | | |     const ambigFrom = moves[i].from;
432 + | | | | |     const ambigTo = moves[i].to;
433 + | | | | |     const ambigPiece = moves[i].piece;
434 + | | | | |     if (piece === ambigPiece && from !== ambigFrom && to === ambigTo) {
435 + | | | | |       ambiguities++;
436 + | | | | |       if (rank(from) === rank(ambigFrom)) {
437 + | | | | |         sameRank++;
438 + | | | | |       }
439 + | | | | |       if (file(from) === file(ambigFrom)) {
440 + | | | | |         sameFile++;
441 + | | | | |       }
442 + | | | | |     }
443 + | | | | |   }
444 + | | | | |   if (ambiguities > 0) {
445 + | | | | |     if (sameRank > 0 && sameFile > 0) {
446 + | | | | |       return algebraic(from);
447 + | | | | |     } else if (sameFile > 0) {
448 + | | | | |       return algebraic(from).charAt(1);
449 + | | | | |     } else {
450 + | | | | |       return algebraic(from).charAt(0);
451 + | | | | |     }
452 + | | | | |   }
453 + | | | | |   return '';
454 + | | | | | }
455 + | | | | | function addMove(moves, color, from, to, piece, captured = void 0, flags = BITS.NORMAL) {
456 + | | | | |   const r = rank(to);
457 + | | | | |   if (piece === PAWN && (r === RANK_1 || r === RANK_8)) {
458 + | | | | |     for (let i = 0; i < PROMOTIONS.length; i++) {
459 + | | | | |       const promotion = PROMOTIONS[i];
460 + | | | | |       moves.push({
461 + | | | | |         color,
462 + | | | | |         from,
463 + | | | | |         to,
464 + | | | | |         piece,
465 + | | | | |         captured,
466 + | | | | |         promotion,
467 + | | | | |         flags: flags | BITS.PROMOTION,
468 + | | | | |       });
469 + | | | | |     }
470 + | | | | |   } else {
471 + | | | | |     moves.push({
472 + | | | | |       color,
473 + | | | | |       from,
474 + | | | | |       to,
475 + | | | | |       piece,
476 + | | | | |       captured,
477 + | | | | |       flags,
478 + | | | | |     });
479 + | | | | |   }
480 + | | | | | }

```

```

481 + | | | | | class Chess {
482 + | | | | |   _board = new Array(128);
483 + | | | | |   _turn = WHITE;
484 + | | | | |   _kings = { w: EMPTY, b: EMPTY };
485 + | | | | |   _epSquare = -1;
486 + | | | | |   _halfMoves = 0;
487 + | | | | |   _moveNumber = 0;
488 + | | | | |   _history = [];
489 + | | | | |   _castling = { w: 0, b: 0 };
490 + | | | | |   _hash = 0n;
491 + | | | | |   // tracks number of times a position has been seen for repetition checking
492 + | | | | |   _positionCount = /* @_PURE */ new Map();
493 + | | | | |   constructor(fen = DEFAULT_POSITION, { skipValidation = false } = {}) {
494 + | | | | |     this.load(fen, { skipValidation });
495 + | | | | |   }
496 + | | | | |   clear() {
497 + | | | | |     this._board = new Array(128);
498 + | | | | |     this._kings = { w: EMPTY, b: EMPTY };
499 + | | | | |     this._turn = WHITE;
500 + | | | | |     this._castling = { w: 0, b: 0 };
501 + | | | | |     this._epSquare = EMPTY;
502 + | | | | |     this._halfMoves = 0;
503 + | | | | |     this._moveNumber = 1;
504 + | | | | |     this._history = [];
505 + | | | | |     this._hash = this._computeHash();
506 + | | | | |     this._positionCount = /* @_PURE */ new Map();
507 + | | | | |   }
508 + | | | | |   load(fen, { skipValidation = false } = {}) {
509 + | | | | |     let tokens = fen.split(/\s+/);
510 + | | | | |     if (tokens.length ≥ 2 && tokens.length < 6) {
511 + | | | | |       const adjustments = ['- ', '- ', '0', '1'];
512 + | | | | |       fen = tokens.concat(adjustments.slice(-(6 - tokens.length))).join(' ');
513 + | | | | |     }
514 + | | | | |     tokens = fen.split(/\s+/);
515 + | | | | |     if (!skipValidation) {
516 + | | | | |       const { ok, error } = validateFen(fen);
517 + | | | | |       if (!ok) {
518 + | | | | |         throw new Error(error);
519 + | | | | |       }
520 + | | | | |     }
521 + | | | | |     const position = tokens[0];
522 + | | | | |     let square = 0;
523 + | | | | |     this.clear();
524 + | | | | |     for (let i = 0; i < position.length; i++) {
525 + | | | | |       const piece = position.charAt(i);
526 + | | | | |       if (piece === '/') {
527 + | | | | |         square += 8;
528 + | | | | |       } else if (isDigit(piece)) {
529 + | | | | |         square += parseInt(piece, 10);
530 + | | | | |       } else {
531 + | | | | |         const color = piece < 'a' ? WHITE : BLACK;
532 + | | | | |         this._put({ type: piece.toLowerCase(), color }, algebraic(square));
533 + | | | | |         square++;
534 + | | | | |       }
535 + | | | | |     }
536 + | | | | |     this._turn = tokens[1];
537 + | | | | |     if (tokens[2].indexOf('K') > -1) {
538 + | | | | |       this._castling.w |= BITS.KSIDE_CASTLE;
539 + | | | | |     }
540 + | | | | |     if (tokens[2].indexOf('Q') > -1) {
541 + | | | | |       this._castling.w |= BITS.QSIDE_CASTLE;
542 + | | | | |     }
543 + | | | | |     if (tokens[2].indexOf('k') > -1) {
544 + | | | | |       this._castling.b |= BITS.KSIDE_CASTLE;
545 + | | | | |     }
546 + | | | | |     if (tokens[2].indexOf('q') > -1) {
547 + | | | | |       this._castling.b |= BITS.QSIDE_CASTLE;
548 + | | | | |     }
549 + | | | | |     this._epSquare = tokens[3] === '-' ? EMPTY : 0x88[tokens[3]];
550 + | | | | |     this._halfMoves = parseInt(tokens[4], 10);
551 + | | | | |     this._moveNumber = parseInt(tokens[5], 10);

```



```

552 + |         this._hash = this._computeHash();
553 + |         this._incPositionCount();
554 + |     }
555 + |     fen({ forceEnpassantSquare = false } = {}) = {
556 + |         let empty = 0;
557 + |         let fen = '';
558 + |         for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
559 + |             if (this._board[i]) {
560 + |                 if (empty > 0) {
561 + |                     fen += empty;
562 + |                     empty = 0;
563 + |                 }
564 + |                 const { color, type: piece } = this._board[i];
565 + |                 fen += color ≡ WHITE ? piece.toUpperCase() : piece.toLowerCase();
566 + |             } else {
567 + |                 empty++;
568 + |             }
569 + |             if ((i + 1) & 136) {
570 + |                 if (empty > 0) {
571 + |                     fen += empty;
572 + |                 }
573 + |                 if (i ≡ 0x88.h1) {
574 + |                     fen += '/';
575 + |                 }
576 + |                 empty = 0;
577 + |                 i += 8;
578 + |             }
579 + |         }
580 + |         let castling = '';
581 + |         if (this._castling[WHITE] & BITS.KSIDE_CASTLE) {
582 + |             castling += 'K';
583 + |         }
584 + |         if (this._castling[WHITE] & BITS.QSIDE_CASTLE) {
585 + |             castling += 'Q';
586 + |         }
587 + |         if (this._castling[BLACK] & BITS.KSIDE_CASTLE) {
588 + |             castling += 'k';
589 + |         }
590 + |         if (this._castling[BLACK] & BITS.QSIDE_CASTLE) {
591 + |             castling += 'q';
592 + |         }
593 + |         castling = castling || '-';
594 + |         let epSquare = '-';
595 + |         if (this._epSquare ≡ EMPTY) {
596 + |             if (forceEnpassantSquare) {
597 + |                 epSquare = algebraic(this._epSquare);
598 + |             } else {
599 + |                 const bigPawnSquare = this._epSquare + (this._turn ≡ WHITE ? 16 : -16);
600 + |                 const squares = [bigPawnSquare + 1, bigPawnSquare - 1];
601 + |                 for (const square of squares) {
602 + |                     if (square & 136) {
603 + |                         continue;
604 + |                     }
605 + |                     const color = this._turn;
606 + |                     if (this._board[square]?.color ≡ color & this._board[square]?.type ≡ PAWN) {
607 + |                         this._makeMove({
608 + |                             color,
609 + |                             from: square,
610 + |                             to: this._epSquare,
611 + |                             piece: PAWN,
612 + |                             captured: PAWN,
613 + |                             flags: BITS.EP_CAPTURE,
614 + |                         });
615 + |                         const isLegal = !this._isKingAttacked(color);
616 + |                         this._undoMove();
617 + |                         if (isLegal) {
618 + |                             epSquare = algebraic(this._epSquare);
619 + |                             break;
620 + |                         }
621 + |                     }
622 + |                 }

```

```

623 + |         }
624 + |     }
625 + |     return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
626 + | }
627 + | _pieceKey(i) {
628 + |     if (!this._board[i]) {
629 + |         return 0n;
630 + |     }
631 + |     const { color, type } = this._board[i];
632 + |     const colorIndex = {
633 + |         w: 0,
634 + |         b: 1,
635 + |     }[color];
636 + |     const typeIndex = {
637 + |         p: 0,
638 + |         n: 1,
639 + |         b: 2,
640 + |         r: 3,
641 + |         q: 4,
642 + |         k: 5,
643 + |     }[type];
644 + |     return PIECE_KEYS[colorIndex][typeIndex][i];
645 + | }
646 + | _epKey() {
647 + |     return this._epSquare === EMPTY ? 0n : EP_KEYS[this._epSquare & 7];
648 + | }
649 + | _castlingKey() {
650 + |     const index = (this._castling.w >> 5) | (this._castling.b >> 3);
651 + |     return CASTLING_KEYS[index];
652 + | }
653 + | _computeHash() {
654 + |     let hash = 0n;
655 + |     for (let i = 0x88.a8; i <= 0x88.h1; i++) {
656 + |         if (i & 136) {
657 + |             i += 7;
658 + |             continue;
659 + |         }
660 + |         if (this._board[i]) {
661 + |             hash ^= this._pieceKey(i);
662 + |         }
663 + |     }
664 + |     hash ^= this._epKey();
665 + |     hash ^= this._castlingKey();
666 + |     if (this._turn === 'b') {
667 + |         hash ^= SIDE_KEY;
668 + |     }
669 + |     return hash;
670 + | }
671 + | reset() {
672 + |     this.load(DEFAULT_POSITION);
673 + | }
674 + | get(square) {
675 + |     return this._board[0x88[square]];
676 + | }
677 + | _set(sq, piece) {
678 + |     this._hash ^= this._pieceKey(sq);
679 + |     this._board[sq] = piece;
680 + |     this._hash ^= this._pieceKey(sq);
681 + | }
682 + | _put({ type, color }, square) {
683 + |     if (SYMBOLS.indexOf(type.toLowerCase()) === -1) {
684 + |         return false;
685 + |     }
686 + |     if (!(square in 0x88)) {
687 + |         return false;
688 + |     }
689 + |     const sq = 0x88[square];
690 + |     if (type === KING && !(this._kings[color] === EMPTY || this._kings[color] === sq)) {
691 + |         return false;
692 + |     }
693 + |     const currentPieceOnSquare = this._board[sq];

```

```

694 + | | | | | if (currentPieceOnSquare && currentPieceOnSquare.type === KING) {
695 + | | | | | | this._kings[currentPieceOnSquare.color] = EMPTY;
696 + | | | | | }
697 + | | | | | this._set(sq, { type, color });
698 + | | | | | if (type === KING) {
699 + | | | | | | this._kings[color] = sq;
700 + | | | | | }
701 + | | | | | return true;
702 + | | | | | }
703 + | | | | | _clear(sq) {
704 + | | | | | | this._hash ^= this._pieceKey(sq);
705 + | | | | | | delete this._board[sq];
706 + | | | | | }
707 + | | | | | _attacked(color, square, verbose) {
708 + | | | | | | const attackers = [];
709 + | | | | | | for (let i = 0x88.a8; i <= 0x88.h1; i++) {
710 + | | | | | | | if (i & 136) {
711 + | | | | | | | | i += 7;
712 + | | | | | | | | continue;
713 + | | | | | | | }
714 + | | | | | | | if (this._board[i] === void 0 || this._board[i].color !== color) {
715 + | | | | | | | | continue;
716 + | | | | | | | }
717 + | | | | | | | const piece = this._board[i];
718 + | | | | | | | const difference = i - square;
719 + | | | | | | | if (difference === 0) {
720 + | | | | | | | | continue;
721 + | | | | | | | }
722 + | | | | | | | const index = difference + 119;
723 + | | | | | | | if (ATTACKS[index] & PIECE_MASKS[piece.type]) {
724 + | | | | | | | | if (piece.type === PAWN) {
725 + | | | | | | | | | if (
726 + | | | | | | | | | | (difference > 0 && piece.color === WHITE) ||
727 + | | | | | | | | | | (difference <= 0 && piece.color === BLACK)
728 + | | | | | | | | | ) {
729 + | | | | | | | | | | if (!verbose) {
730 + | | | | | | | | | | | return true;
731 + | | | | | | | | | | } else {
732 + | | | | | | | | | | | attackers.push(algebraic(i));
733 + | | | | | | | | | | }
734 + | | | | | | | | | }
735 + | | | | | | | | continue;
736 + | | | | | | | }
737 + | | | | | | | if (piece.type === 'n' || piece.type === 'k') {
738 + | | | | | | | | if (!verbose) {
739 + | | | | | | | | | return true;
740 + | | | | | | | | } else {
741 + | | | | | | | | | attackers.push(algebraic(i));
742 + | | | | | | | | | continue;
743 + | | | | | | | | }
744 + | | | | | | | }
745 + | | | | | | | const offset = RAYS[index];
746 + | | | | | | | let j = i + offset;
747 + | | | | | | | let blocked = false;
748 + | | | | | | | while (j !== square) {
749 + | | | | | | | | if (this._board[j] !== null) {
750 + | | | | | | | | | blocked = true;
751 + | | | | | | | | | break;
752 + | | | | | | | | }
753 + | | | | | | | | j += offset;
754 + | | | | | | | }
755 + | | | | | | | if (!blocked) {
756 + | | | | | | | | if (!verbose) {
757 + | | | | | | | | | return true;
758 + | | | | | | | | } else {
759 + | | | | | | | | | attackers.push(algebraic(i));
760 + | | | | | | | | | continue;
761 + | | | | | | | | }
762 + | | | | | | | }
763 + | | | | | | }
764 + | | | | | }

```

```

765 + | | | | | if(verbose) {
766 + | | | | | | return attackers;
767 + | | | | | } else {
768 + | | | | | | return false;
769 + | | | | | }
770 + | | | | | }
771 + | | | | | attackers(square, attackedBy) {
772 + | | | | | | if(!attackedBy) {
773 + | | | | | | | return this._attacked(this._turn, 0x88[square], true);
774 + | | | | | | } else {
775 + | | | | | | | return this._attacked(attackedBy, 0x88[square], true);
776 + | | | | | | }
777 + | | | | | }
778 + | | | | | _isKingAttacked(color) {
779 + | | | | | | const square = this._kings[color];
780 + | | | | | | return square === -1 ? false : this._attacked(swapColor(color), square);
781 + | | | | | }
782 + | | | | | hash() {
783 + | | | | | | return this._hash.toString(16);
784 + | | | | | }
785 + | | | | | isAttacked(square, attackedBy) {
786 + | | | | | | return this._attacked(attackedBy, 0x88[square]);
787 + | | | | | }
788 + | | | | | isCheck() {
789 + | | | | | | return this._isKingAttacked(this._turn);
790 + | | | | | }
791 + | | | | | inCheck() {
792 + | | | | | | return this.isCheck();
793 + | | | | | }
794 + | | | | | isCheckmate() {
795 + | | | | | | return this.isCheck() && this._moves().length === 0;
796 + | | | | | }
797 + | | | | | isStalemate() {
798 + | | | | | | return !this.isCheck() && this._moves().length === 0;
799 + | | | | | }
800 + | | | | | isInsufficientMaterial() {
801 + | | | | | | const pieces = {
802 + | | | | | | | b: 0,
803 + | | | | | | | n: 0,
804 + | | | | | | | r: 0,
805 + | | | | | | | q: 0,
806 + | | | | | | | k: 0,
807 + | | | | | | | p: 0,
808 + | | | | | | };
809 + | | | | | | const bishops = [];
810 + | | | | | | let numPieces = 0;
811 + | | | | | | let squareColor = 0;
812 + | | | | | | for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
813 + | | | | | | | squareColor = (squareColor + 1) % 2;
814 + | | | | | | | if (i & 136) {
815 + | | | | | | | | i += 7;
816 + | | | | | | | | continue;
817 + | | | | | | | }
818 + | | | | | | | const piece = this._board[i];
819 + | | | | | | | if (piece) {
820 + | | | | | | | | pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1;
821 + | | | | | | | | if (piece.type === BISHOP) {
822 + | | | | | | | | | bishops.push(squareColor);
823 + | | | | | | | | }
824 + | | | | | | | numPieces++;
825 + | | | | | | }
826 + | | | | | | }
827 + | | | | | | if (numPieces === 2) {
828 + | | | | | | | return true;
829 + | | | | | | } else if (
830 + | | | | | | | // k vs. kn .... or .... k vs. kb
831 + | | | | | | | numPieces === 3 &&
832 + | | | | | | | (pieces[BISHOP] === 1 || pieces[KNIGHT] === 1)
833 + | | | | | | ) {
834 + | | | | | | | return true;
835 + | | | | | | } else if (numPieces === pieces[BISHOP] + 2) {

```

```

836 + |         let sum = 0;
837 + |         const len = bishops.length;
838 + |         for (let i = 0; i < len; i++) {
839 + |             sum += bishops[i];
840 + |         }
841 + |         if (sum === 0 || sum === len) {
842 + |             return true;
843 + |         }
844 + |     }
845 + |     return false;
846 + | }
847 + | isThreefoldRepetition() {
848 + |     return this._getPositionCount(this._hash) ≥ 3;
849 + | }
850 + | isDrawByFiftyMoves() {
851 + |     return this._halfMoves ≥ 100;
852 + | }
853 + | isDraw() {
854 + |     return (
855 + |         this.isDrawByFiftyMoves() ||
856 + |         this.isStalemate() ||
857 + |         this.isInsufficientMaterial() ||
858 + |         this.isThreefoldRepetition()
859 + |     );
860 + | }
861 + | isGameOver() {
862 + |     return this.isCheckmate() || this.isDraw();
863 + | }
864 + | moves({ verbose = false, square = void 0, piece = void 0 } = {}) {
865 + |     const moves = this._moves({ square, piece });
866 + |     if (verbose) {
867 + |         return moves.map((move) => new Move(this, move));
868 + |     } else {
869 + |         return moves.map((move) => this._moveToSan(move, moves));
870 + |     }
871 + | }
872 + | _moves({ legal = true, piece = void 0, square = void 0 } = {}) {
873 + |     const forSquare = square ? square.toLowerCase() : void 0;
874 + |     const forPiece = piece ? piece.toLowerCase();
875 + |     const moves = [];
876 + |     const us = this._turn;
877 + |     const them = swapColor(us);
878 + |     let firstSquare = 0x88.a8;
879 + |     let lastSquare = 0x88.h1;
880 + |     let singleSquare = false;
881 + |     if (forSquare) {
882 + |         if (!(forSquare in 0x88)) {
883 + |             return [];
884 + |         } else {
885 + |             firstSquare = lastSquare = 0x88[forSquare];
886 + |             singleSquare = true;
887 + |         }
888 + |     }
889 + |     for (let from = firstSquare; from ≤ lastSquare; from++) {
890 + |         if (from & 136) {
891 + |             from += 7;
892 + |             continue;
893 + |         }
894 + |         if (!this._board[from] || this._board[from].color === them) {
895 + |             continue;
896 + |         }
897 + |         const { type } = this._board[from];
898 + |         let to;
899 + |         if (type === PAWN) {
900 + |             if (forPiece & forPiece !== type) continue;
901 + |             to = from + PAWN_OFFSETS[us][0];
902 + |             if (!this._board[to]) {
903 + |                 addMove(moves, us, from, to, PAWN);
904 + |                 to = from + PAWN_OFFSETS[us][1];
905 + |                 if (SECOND_RANK[us] === rank(from) & !this._board[to]) {
906 + |                     addMove(moves, us, from, to, PAWN, void 0, BITS.BIG_PAWN);

```

```

907 +         }
908 +     }
909 +     for (let j = 2; j < 4; j++) {
910 +         to = from + PAWN_OFFSETS[us][j];
911 +         if (to & 136) continue;
912 +         if (this._board[to]?.color === them) {
913 +             addMove(moves, us, from, to, PAWN, this._board[to].type, BITS.CAPTURE);
914 +         } else if (to === this._epSquare) {
915 +             addMove(moves, us, from, to, PAWN, PAWN, BITS.EP_CAPTURE);
916 +         }
917 +     }
918 + } else {
919 +     if (forPiece && forPiece !== type) continue;
920 +     for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++) {
921 +         const offset = PIECE_OFFSETS[type][j];
922 +         to = from;
923 +         while (true) {
924 +             to += offset;
925 +             if (to & 136) break;
926 +             if (!this._board[to]) {
927 +                 addMove(moves, us, from, to, type);
928 +             } else {
929 +                 if (this._board[to].color === us) break;
930 +                 addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
931 +                 break;
932 +             }
933 +             if (type === KNIGHT || type === KING) break;
934 +         }
935 +     }
936 + }
937 + }
938 + if (forPiece === void 0 || forPiece === KING) {
939 +     if (!singleSquare || lastSquare === this._kings[us]) {
940 +         if (this._castling[us] & BITS.KSIDE_CASTLE) {
941 +             const castlingFrom = this._kings[us];
942 +             const castlingTo = castlingFrom + 2;
943 +             if (
944 +                 !this._board[castlingFrom + 1] &&
945 +                 !this._board[castlingTo] &&
946 +                 !this._attacked(them, this._kings[us]) &&
947 +                 !this._attacked(them, castlingFrom + 1) &&
948 +                 !this._attacked(them, castlingTo)
949 +             ) {
950 +                 addMove(moves, us, this._kings[us], castlingTo, KING, void 0, BITS.KSIDE_CASTLE);
951 +             }
952 +         }
953 +         if (this._castling[us] & BITS.QSIDE_CASTLE) {
954 +             const castlingFrom = this._kings[us];
955 +             const castlingTo = castlingFrom - 2;
956 +             if (
957 +                 !this._board[castlingFrom - 1] &&
958 +                 !this._board[castlingFrom - 2] &&
959 +                 !this._board[castlingFrom - 3] &&
960 +                 !this._attacked(them, this._kings[us]) &&
961 +                 !this._attacked(them, castlingFrom - 1) &&
962 +                 !this._attacked(them, castlingTo)
963 +             ) {
964 +                 addMove(moves, us, this._kings[us], castlingTo, KING, void 0, BITS.QSIDE_CASTLE);
965 +             }
966 +         }
967 +     }
968 + }
969 + if (!legal || this._kings[us] === -1) {
970 +     return moves;
971 + }
972 + const legalMoves = [];
973 + for (let i = 0, len = moves.length; i < len; i++) {
974 +     this._makeMove(moves[i]);
975 +     if (!this._isKingAttacked(us)) {
976 +         legalMoves.push(moves[i]);
977 +     }

```

```
978 + |         .this._undoMove();
979 + |     }
980 + |     return legalMoves;
981 + | }
982 + | move(move) {
983 + |     let moveObj = null;
984 + |     const moves = this._moves();
985 + |     for (let i = 0, len = moves.length; i < len; i++) {
986 + |         if (
987 + |             move.from === algebraic(moves[i].from) &&
988 + |             move.to === algebraic(moves[i].to) &&
989 + |             (!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
990 + |         ) {
991 + |             moveObj = moves[i];
992 + |             break;
993 + |         }
994 + |     }
995 + |     if (!moveObj) {
996 + |         throw new Error(`Invalid move: ${JSON.stringify(move)}`);
997 + |     }
998 + |     const prettyMove = new Move(this, moveObj);
999 + |     this._makeMove(moveObj);
1000 + |     this._incPositionCount();
1001 + |     return prettyMove;
1002 + | }
1003 + | _push(move) {
1004 + |     this._history.push({
1005 + |         move,
1006 + |         kings: { b: this._kings.b, w: this._kings.w },
1007 + |         turn: this._turn,
1008 + |         castling: { b: this._castling.b, w: this._castling.w },
1009 + |         epSquare: this._epSquare,
1010 + |         halfMoves: this._halfMoves,
1011 + |         moveNumber: this._moveNumber,
1012 + |     });
1013 + | }
1014 + | _movePiece(from, to) {
1015 + |     this._hash ^= this._pieceKey(from);
1016 + |     this._board[to] = this._board[from];
1017 + |     delete this._board[from];
1018 + |     this._hash ^= this._pieceKey(to);
1019 + | }
1020 + | _makeMove(move) {
1021 + |     const us = this._turn;
1022 + |     const them = swapColor(us);
1023 + |     this._push(move);
1024 + |     if (move.flags & BITS.NULL_MOVE) {
1025 + |         if (us === BLACK) {
1026 + |             this._moveNumber++;
1027 + |         }
1028 + |         this._halfMoves++;
1029 + |         this._turn = them;
1030 + |         this._epSquare = EMPTY;
1031 + |         return;
1032 + |     }
1033 + |     this._hash ^= this._epKey();
1034 + |     this._hash ^= this._castlingKey();
1035 + |     if (move.captured) {
1036 + |         this._hash ^= this._pieceKey(move.to);
1037 + |     }
1038 + |     this._movePiece(move.from, move.to);
1039 + |     if (move.flags & BITS.EP_CAPTURE) {
1040 + |         if (this._turn === BLACK) {
1041 + |             this._clear(move.to - 16);
1042 + |         } else {
1043 + |             this._clear(move.to + 16);
1044 + |         }
1045 + |     }
1046 + |     if (move.promotion) {
1047 + |         this._clear(move.to);
1048 + |         this._set(move.to, { type: move.promotion, color: us });
```

```

1049 + | | | | | }
1050 + | | | | | if (this._board[move.to].type === KING) {
1051 + | | | | | | this._kings[us] = move.to;
1052 + | | | | | | if (move.flags & BITS.KSIDE_CASTLE) {
1053 + | | | | | | | const castlingTo = move.to - 1;
1054 + | | | | | | | const castlingFrom = move.to + 1;
1055 + | | | | | | | this._movePiece(castlingFrom, castlingTo);
1056 + | | | | | | } else if (move.flags & BITS.QSIDE_CASTLE) {
1057 + | | | | | | | const castlingTo = move.to + 1;
1058 + | | | | | | | const castlingFrom = move.to - 2;
1059 + | | | | | | | this._movePiece(castlingFrom, castlingTo);
1060 + | | | | | | }
1061 + | | | | | | this._castling[us] = 0;
1062 + | | | | | | }
1063 + | | | | | | if (this._castling[us]) {
1064 + | | | | | | | for (let i = 0, len = ROOKS[us].length; i < len; i++) {
1065 + | | | | | | | | if (move.from === ROOKS[us][i].square & this._castling[us] & ROOKS[us][i].flag) {
1066 + | | | | | | | | | this._castling[us] ^= ROOKS[us][i].flag;
1067 + | | | | | | | | | break;
1068 + | | | | | | | | }
1069 + | | | | | | | }
1070 + | | | | | | }
1071 + | | | | | | if (this._castling[them]) {
1072 + | | | | | | | for (let i = 0, len = ROOKS[them].length; i < len; i++) {
1073 + | | | | | | | | if (move.to === ROOKS[them][i].square & this._castling[them] & ROOKS[them][i].flag) {
1074 + | | | | | | | | | this._castling[them] ^= ROOKS[them][i].flag;
1075 + | | | | | | | | | break;
1076 + | | | | | | | | }
1077 + | | | | | | | }
1078 + | | | | | | }
1079 + | | | | | | this._hash ^= this._castlingKey();
1080 + | | | | | | if (move.flags & BITS.BIG_PAWN) {
1081 + | | | | | | | let epSquare;
1082 + | | | | | | | if (us === BLACK) {
1083 + | | | | | | | | epSquare = move.to - 16;
1084 + | | | | | | | } else {
1085 + | | | | | | | | epSquare = move.to + 16;
1086 + | | | | | | | }
1087 + | | | | | | | if (
1088 + | | | | | | | | ((move.to - 1) & 136) &
1089 + | | | | | | | | | this._board[move.to - 1].type === PAWN &
1090 + | | | | | | | | | this._board[move.to - 1].color === them) ||
1091 + | | | | | | | | ((move.to + 1) & 136) &
1092 + | | | | | | | | | this._board[move.to + 1].type === PAWN &
1093 + | | | | | | | | | this._board[move.to + 1].color === them)
1094 + | | | | | | | | ) {
1095 + | | | | | | | | | this._epSquare = epSquare;
1096 + | | | | | | | | | this._hash ^= this._epKey();
1097 + | | | | | | | | } else {
1098 + | | | | | | | | | this._epSquare = EMPTY;
1099 + | | | | | | | | }
1100 + | | | | | | | } else {
1101 + | | | | | | | | this._epSquare = EMPTY;
1102 + | | | | | | | }
1103 + | | | | | | | if (move.piece === PAWN) {
1104 + | | | | | | | | this._halfMoves = 0;
1105 + | | | | | | | } else if (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)) {
1106 + | | | | | | | | this._halfMoves = 0;
1107 + | | | | | | | } else {
1108 + | | | | | | | | this._halfMoves++;
1109 + | | | | | | | }
1110 + | | | | | | | if (us === BLACK) {
1111 + | | | | | | | | this._moveNumber++;
1112 + | | | | | | | }
1113 + | | | | | | | this._turn = them;
1114 + | | | | | | | this._hash ^= SIDE_KEY;
1115 + | | | | | | }
1116 + | | | | | | undo() {
1117 + | | | | | | | const hash = this._hash;
1118 + | | | | | | | const move = this._undoMove();
1119 + | | | | | | | if (move) {

```



```

1120 + |         const prettyMove = new Move(this, move);
1121 + |         this._decPositionCount(hash);
1122 + |         return prettyMove;
1123 + |     }
1124 + |     return null;
1125 + | }
1126 + | _undoMove() {
1127 + |     const old = this._history.pop();
1128 + |     if (old === void 0) {
1129 + |         return null;
1130 + |     }
1131 + |     this._hash ^= this._epKey();
1132 + |     this._hash ^= this._castlingKey();
1133 + |     const move = old.move;
1134 + |     this._kings = old.kings;
1135 + |     this._turn = old.turn;
1136 + |     this._castling = old.castling;
1137 + |     this._epSquare = old.epSquare;
1138 + |     this._halfMoves = old.halfMoves;
1139 + |     this._moveNumber = old.moveNumber;
1140 + |     this._hash ^= this._epKey();
1141 + |     this._hash ^= this._castlingKey();
1142 + |     this._hash ^= SIDE_KEY;
1143 + |     const us = this._turn;
1144 + |     const them = swapColor(us);
1145 + |     if (move.flags & BITS.NULL_MOVE) {
1146 + |         return move;
1147 + |     }
1148 + |     this._movePiece(move.to, move.from);
1149 + |     if (move.piece) {
1150 + |         this._clear(move.from);
1151 + |         this._set(move.from, { type: move.piece, color: us });
1152 + |     }
1153 + |     if (move.captured) {
1154 + |         if (move.flags & BITS.EP_CAPTURE) {
1155 + |             let index;
1156 + |             if (us === BLACK) {
1157 + |                 index = move.to - 16;
1158 + |             } else {
1159 + |                 index = move.to + 16;
1160 + |             }
1161 + |             this._set(index, { type: PAWN, color: them });
1162 + |         } else {
1163 + |             this._set(move.to, { type: move.captured, color: them });
1164 + |         }
1165 + |     }
1166 + |     if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
1167 + |         let castlingTo, castlingFrom;
1168 + |         if (move.flags & BITS.KSIDE_CASTLE) {
1169 + |             castlingTo = move.to + 1;
1170 + |             castlingFrom = move.to - 1;
1171 + |         } else {
1172 + |             castlingTo = move.to - 2;
1173 + |             castlingFrom = move.to + 1;
1174 + |         }
1175 + |         this._movePiece(castlingFrom, castlingTo);
1176 + |     }
1177 + |     return move;
1178 + | }
1179 + | // Convert a move from 0x88 coordinates to Standard Algebraic Notation.
1180 + | _moveToSan(move, moves) {
1181 + |     let output = '';
1182 + |     if (move.flags & BITS.KSIDE_CASTLE) {
1183 + |         output = 'O-O';
1184 + |     } else if (move.flags & BITS.QSIDE_CASTLE) {
1185 + |         output = 'O-O-O';
1186 + |     } else {
1187 + |         if (move.piece !== PAWN) {
1188 + |             const disambiguator = getDisambiguator(move, moves);
1189 + |             output += move.piece.toUpperCase() + disambiguator;
1190 + |         }

```

```

1191 + |         if (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)) {
1192 + |             if (move.piece === PAWN) {
1193 + |                 output += algebraic(move.from)[0];
1194 + |             }
1195 + |             output += 'x';
1196 + |         }
1197 + |         output += algebraic(move.to);
1198 + |         if (move.promotion) {
1199 + |             output += '=' + move.promotion.toUpperCase();
1200 + |         }
1201 + |     }
1202 + |     this._makeMove(move);
1203 + |     if (this.isCheck()) {
1204 + |         if (this.isCheckmate()) {
1205 + |             output += '#';
1206 + |         } else {
1207 + |             output += '+';
1208 + |         }
1209 + |     }
1210 + |     this._undoMove();
1211 + |     return output;
1212 + | }
1213 + | turn() {
1214 + |     return this._turn;
1215 + | }
1216 + | board() {
1217 + |     const output = [];
1218 + |     let row = [];
1219 + |     for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
1220 + |         if (this._board[i] == null) {
1221 + |             row.push(null);
1222 + |         } else {
1223 + |             row.push({
1224 + |                 square: algebraic(i),
1225 + |                 type: this._board[i].type,
1226 + |                 color: this._board[i].color,
1227 + |             });
1228 + |         }
1229 + |         if ((i + 1) & 136) {
1230 + |             output.push(row);
1231 + |             row = [];
1232 + |             i += 8;
1233 + |         }
1234 + |     }
1235 + |     return output;
1236 + | }
1237 + | squareColor(square) {
1238 + |     if (square in 0x88) {
1239 + |         const sq = 0x88[square];
1240 + |         return (rank(sq) + file(sq)) % 2 === 0 ? 'light' : 'dark';
1241 + |     }
1242 + |     return null;
1243 + | }
1244 + | history({ verbose = false } = {}) {
1245 + |     const reversedHistory = [];
1246 + |     const moveHistory = [];
1247 + |     while (this._history.length > 0) {
1248 + |         reversedHistory.push(this._undoMove());
1249 + |     }
1250 + |     while (true) {
1251 + |         const move = reversedHistory.pop();
1252 + |         if (!move) {
1253 + |             break;
1254 + |         }
1255 + |         if (verbose) {
1256 + |             moveHistory.push(new Move(this, move));
1257 + |         } else {
1258 + |             moveHistory.push(this._moveToSan(move, this._moves()));
1259 + |         }
1260 + |         this._makeMove(move);
1261 + |     }

```

```

1262 + | | | | |return moveHistory;
1263 + | | | | |}
1264 + | | | | |/*
1265 + | | | | | * Keeps track of position occurrence counts for the purpose of repetition
1266 + | | | | | * checking. Old positions are removed from the map if their counts are reduced to 0.
1267 + | | | | | */
1268 + | | | | |_getPositionCount(hash) {
1269 + | | | | |return this._positionCount.get(hash) ?? 0;
1270 + | | | | |}
1271 + | | | | |_incPositionCount() {
1272 + | | | | |this._positionCount.set(this._hash, (this._positionCount.get(this._hash) ?? 0) + 1);
1273 + | | | | |}
1274 + | | | | |_decPositionCount(hash) {
1275 + | | | | |const currentCount = this._positionCount.get(hash) ?? 0;
1276 + | | | | |if (currentCount === 1) {
1277 + | | | | |this._positionCount.delete(hash);
1278 + | | | | |} else {
1279 + | | | | |this._positionCount.set(hash, currentCount - 1);
1280 + | | | | |}
1281 + | | | | |}
1282 + | | | | |moveNumber() {
1283 + | | | | |return this._moveNumber;
1284 + | | | | |}
1285 + | | | | |}
1286 + | | | | |exports.BISHOP = BISHOP;
1287 + | | | | |exports.BLACK = BLACK;
1288 + | | | | |exports.Chess = Chess;
1289 + | | | | |exports.DEFAULT_POSITION = DEFAULT_POSITION;
1290 + | | | | |exports.KING = KING;
1291 + | | | | |exports.KNIGHT = KNIGHT;
1292 + | | | | |exports.Move = Move;
1293 + | | | | |exports.PAWN = PAWN;
1294 + | | | | |exports.QUEEN = QUEEN;
1295 + | | | | |exports.ROOK = ROOK;
1296 + | | | | |exports.SQUARES = SQUARES;
1297 + | | | | |exports.WHITE = WHITE;
1298 + | | | | |exports.validateFen = validateFen;
1299 + | | | | |exports.xoroshiro128 = xoroshiro128;
1300 + | | | | |Object.defineProperty(exports, Symbol.toStringTag, {value: 'Module'});
1301 + | | | | |return exports;
1302 + | | | | |})({});
1303 + | | | | |
1304 + | | | | |globalThis.TSChessShared = TSChessShared;
1305 + | | | | |</script>
1306 + | | | | |
1307 + | | | | |<script id="ai-worker-source-1" type="text/plain">
1308 + | | | | |!(function () {
1309 + | | | | |'use strict';
1310 + | | | | |const sharedCore = globalThis.TSChessShared;
1311 + | | | | |if (!sharedCore) throw new Error('Missing shared chess runtime: TSChessShared');
1312 + | | | | |const Chess = sharedCore.Chess;
1313 + | | | | |sharedCore.SQUARES;
1314 + | | | | |const HARD_MAX_DEPTH = Number.POSITIVE_INFINITY,
1315 + | | | | |PIECE_VALUES = {p: 100, n: 320, b: 330, r: 500, q: 900, k: 0 },
1316 + | | | | |STARTING_NON_PAWN_MATERIAL =
1317 + | | | | |2 * (PIECE_VALUES.q + 2 * PIECE_VALUES.r + 2 * PIECE_VALUES.b + 2 * PIECE_VALUES.n),
1318 + | | | | |OPENING_PHASE_END_MATERIAL = STARTING_NON_PAWN_MATERIAL / 2;
1319 + | | | | |function searchRoot(chess, depth, ctx, previousBestMove) {
1320 + | | | | |const candidates = [],
1321 + | | | | |legalMoves = orderMoves(chess.moves({ verbose: !0 }), previousBestMove);
1322 + | | | | |for (const move of legalMoves) {
1323 + | | | | |checkTime(ctx, chess.move(toMoveCommand(move)));
1324 + | | | | |const score = -negamax(chess, depth - 1, -1e7, 1e7, 1, ctx);
1325 + | | | | |chess.undo(), candidates.push({ move: toMoveCommand(move), san: move.san, score: score });
1326 + | | | | |}
1327 + | | | | |if (0 === candidates.length) throw new SearchTimeout();
1328 + | | | | |const principal = (function selectBestRootMove(candidates) {
1329 + | | | | |return candidates.reduce((currentBest, candidate) =>
1330 + | | | | |candidate.score > currentBest.score ? candidate : currentBest,
1331 + | | | | |);
1332 + | | | | |})(candidates),

```

```

1333 + | | | | | selection = (function selectRootMove(candidates, settings, best) {
1334 + | | | | |   const level = settings.randomness / 30,
1335 + | | | | |   windowCp = 20 + 150 * level,
1336 + | | | | |   floorScore = best.score - windowCp,
1337 + | | | | |   ticketPower = 4 - 1.5 * level;
1338 + | | | | |   if (
1339 + | | | | |     (function shouldForceBestMove(settings, best) {
1340 + | | | | |       return (
1341 + | | | | |         0 === settings.randomness ||
1342 + | | | | |         (function isMateProtectedScore(score) {
1343 + | | | | |           return Math.abs(score) ≥ 99e4;
1344 + | | | | |         })(best.score) ||
1345 + | | | | |         ('hard' === settings.preset && best.score < 0)
1346 + | | | | |       );
1347 + | | | | |     })(settings, best)
1348 + | | | | |   )
1349 + | | | | |   return {
1350 + | | | | |     selected: best,
1351 + | | | | |     debug: createDeterministicDebug(
1352 + | | | | |       candidates,
1353 + | | | | |       best,
1354 + | | | | |       best,
1355 + | | | | |       windowCp,
1356 + | | | | |       floorScore,
1357 + | | | | |       ticketPower,
1358 + | | | | |     ),
1359 + | | | | |   };
1360 + | | | | |   const ticketCandidates = (function selectTicketCandidates(candidates, settings, best) {
1361 + | | | | |     return 'hard' === settings.preset && best.score > 0
1362 + | | | | |     ? candidates.filter((candidate) ⇒ candidate.score > 0)
1363 + | | | | |     : candidates;
1364 + | | | | |   })(candidates, settings, best),
1365 + | | | | |   weightedCandidates = ticketCandidates
1366 + | | | | |   .map((candidate) ⇒ ({
1367 + | | | | |     candidate: candidate,
1368 + | | | | |     tickets: (Math.max(0, candidate.score - floorScore) / windowCp) ** ticketPower,
1369 + | | | | |   })))
1370 + | | | | |   .filter(({ tickets: tickets }) ⇒ tickets > 0),
1371 + | | | | |   totalTickets = weightedCandidates.reduce(
1372 + | | | | |     (total, { tickets: tickets }) ⇒ total + tickets,
1373 + | | | | |     0,
1374 + | | | | |   );
1375 + | | | | |   if (totalTickets ≤ 0)
1376 + | | | | |     return {
1377 + | | | | |       selected: best,
1378 + | | | | |       debug: createDeterministicDebug(
1379 + | | | | |         candidates,
1380 + | | | | |         best,
1381 + | | | | |         best,
1382 + | | | | |         windowCp,
1383 + | | | | |         floorScore,
1384 + | | | | |         ticketPower,
1385 + | | | | |       ),
1386 + | | | | |     };
1387 + | | | | |   let draw = Math.random() * totalTickets,
1388 + | | | | |   selected = best;
1389 + | | | | |   for (const { candidate: candidate, tickets: tickets } of weightedCandidates)
1390 + | | | | |     if (((draw -= tickets), draw ≤ 0)) {
1391 + | | | | |       selected = candidate;
1392 + | | | | |       break;
1393 + | | | | |     }
1394 + | | | | |   return {
1395 + | | | | |     selected: selected,
1396 + | | | | |     debug: createWeightedDebug(
1397 + | | | | |       candidates,
1398 + | | | | |       best,
1399 + | | | | |       selected,
1400 + | | | | |       weightedCandidates,
1401 + | | | | |       windowCp,
1402 + | | | | |       floorScore,
1403 + | | | | |       ticketPower,

```

```

1404 + | | | | | totalTickets,
1405 + | | | | | ),
1406 + | | | | | };
1407 + | | | | | })(candidates, ctx.settings, principal);
1408 + | | | | | return { selected: selection.selected, principal: principal, debug: selection.debug };
1409 + | | | | | }
1410 + | | | | | function shouldStopAfterStableDepth(result, previousPrincipal, settings) {
1411 + | | | | | return (
1412 + | | | | | 'hard' === settings.preset &&
1413 + | | | | | null !== previousPrincipal &&
1414 + | | | | | !isSameMoveCommand(previousPrincipal.move, result.principal.move) &&
1415 + | | | | | !(Math.abs(previousPrincipal.score - result.principal.score) > 90) &&
1416 + | | | | | (function rootScoreGap(debug) {
1417 + | | | | | const [bestCandidate, secondCandidate] = debug.candidates;
1418 + | | | | | return bestCandidate && secondCandidate ? bestCandidate.score - secondCandidate.score : 1e7;
1419 + | | | | | })(result.debug) ≥ 120
1420 + | | | | | );
1421 + | | | | | }
1422 + | | | | | function negamax(chess, depth, alpha, beta, ply, ctx) {
1423 + | | | | | if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
1424 + | | | | | if (depth ≤ 0)
1425 + | | | | | return ctx.settings.quiescence
1426 + | | | | | ? quiescence(chess, alpha, beta, ply, ctx)
1427 + | | | | | : evaluatePosition(chess);
1428 + | | | | | let bestScore = -1e7,
1429 + | | | | | currentAlpha = alpha;
1430 + | | | | | const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null);
1431 + | | | | | for (const move of legalMoves) {
1432 + | | | | | chess.move(toMoveCommand(move));
1433 + | | | | | const score = -negamax(chess, depth - 1, -beta, -currentAlpha, ply + 1, ctx);
1434 + | | | | | if (
1435 + | | | | | (chess.undo(),
1436 + | | | | | (bestScore = Math.max(bestScore, score)),
1437 + | | | | | (currentAlpha = Math.max(currentAlpha, score)),
1438 + | | | | | currentAlpha ≥ beta)
1439 + | | | | | )
1440 + | | | | | break;
1441 + | | | | | }
1442 + | | | | | return bestScore;
1443 + | | | | | }
1444 + | | | | | function quiescence(chess, alpha, beta, ply, ctx) {
1445 + | | | | | if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
1446 + | | | | | let currentAlpha = alpha;
1447 + | | | | | const standPat = evaluatePosition(chess);
1448 + | | | | | if (standPat ≥ beta) return beta;
1449 + | | | | | if (((currentAlpha = Math.max(currentAlpha, standPat)), ply ≥ 6)) return currentAlpha;
1450 + | | | | | const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null),
1451 + | | | | | tacticalMoves = chess.isCheck() ? legalMoves : legalMoves.filter(isTacticalMove);
1452 + | | | | | for (const move of tacticalMoves) {
1453 + | | | | | chess.move(toMoveCommand(move));
1454 + | | | | | const score = -quiescence(chess, -beta, -currentAlpha, ply + 1, ctx);
1455 + | | | | | if ((chess.undo(), score ≥ beta)) return beta;
1456 + | | | | | currentAlpha = Math.max(currentAlpha, score);
1457 + | | | | | }
1458 + | | | | | return currentAlpha;
1459 + | | | | | }
1460 + | | | | | function evaluatePosition(chess) {
1461 + | | | | | const board = chess.board(),
1462 + | | | | | pawns = [],
1463 + | | | | | bishops = { w: 0, b: 0 },
1464 + | | | | | kings = {};
1465 + | | | | | let whiteScore = 0,
1466 + | | | | | blackScore = 0,
1467 + | | | | | nonPawnMaterial = 0;
1468 + | | | | | for (const row of board)
1469 + | | | | | for (const pieceOnSquare of row) {
1470 + | | | | | if (null === pieceOnSquare) continue;
1471 + | | | | | const { color: color, type: type, square: square } = pieceOnSquare,
1472 + | | | | | pieceScore = PIECE_VALUES[type] + pieceSquareScore(type, color, square);
1473 + | | | | | ('w' === color ? (whiteScore += pieceScore) : (blackScore += pieceScore),
1474 + | | | | | 'p' === type

```

```

1475 + |         ? pawns.push({ color: color, file: fileIndex(square), rank: rankNumber(square) })
1476 + |         : 'k' !== type && (nonPawnMaterial += PIECE_VALUES[type]),
1477 + |         'b' === type && (bishops[color] += 1),
1478 + |         'k' === type && (kings[color] = square));
1479 + |     }
1480 + |     ((whiteScore += bishops.w ≥ 2 ? 35 : 0),
1481 + |     (blackScore += bishops.b ≥ 2 ? 35 : 0),
1482 + |     (whiteScore += pawnStructureScore(pawns, 'w')),
1483 + |     (blackScore += pawnStructureScore(pawns, 'b')),
1484 + |     (whiteScore += openingPrinciplesScore(board, pawns, 'w', nonPawnMaterial)),
1485 + |     (blackScore += openingPrinciplesScore(board, pawns, 'b', nonPawnMaterial)),
1486 + |     (whiteScore += developmentScore(board, 'w')),
1487 + |     (blackScore += developmentScore(board, 'b')),
1488 + |     (whiteScore += kingSafetyScore(kings.w, pawns, 'w', nonPawnMaterial)),
1489 + |     (blackScore += kingSafetyScore(kings.b, pawns, 'b', nonPawnMaterial)));
1490 + |     let score = (whiteScore - blackScore) * ('w' === chess.turn() ? 1 : -1);
1491 + |     return ((score += 2 * chess.moves().length), chess.isCheck() && (score -= 45), score);
1492 + | }
1493 + |
1494 + | function pieceSquareScore(type, color, square) {
1495 + |     const file = fileIndex(square),
1496 + |           relativeRank = 'w' === color ? rankNumber(square) : 9 - rankNumber(square),
1497 + |           centerDistance = Math.abs(file - 3.5) + Math.abs(rankNumber(square) - 4.5);
1498 + |     switch (type) {
1499 + |     case 'p':
1500 + |         return 7 * relativeRank + (file ≥ 2 && file ≤ 5 ? 8 : 0);
1501 + |     case 'n':
1502 + |         return 42 - 12 * centerDistance - (1 === relativeRank ? 12 : 0);
1503 + |     case 'b':
1504 + |         return 30 - 7 * centerDistance;
1505 + |     case 'r':
1506 + |         return (relativeRank ≥ 7 ? 18 : 0) + (0 === file || 7 === file ? 4 : 0);
1507 + |     case 'q':
1508 + |         return 14 - 3 * centerDistance;
1509 + |     case 'k':
1510 + |         return relativeRank ≥ 7 ? -18 : 0;
1511 + |     }
1512 + | }
1513 + |
1514 + | function pawnStructureScore(pawns, color) {
1515 + |     const ownPawns = pawns.filter((pawn) => pawn.color === color),
1516 + |           fileCounts = Array.from({ length: 8 }, () => 0);
1517 + |     let score = 0;
1518 + |     for (const pawn of ownPawns) fileCounts[pawn.file] += 1;
1519 + |     for (const pawn of ownPawns) {
1520 + |         const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
1521 + |         (fileCounts[pawn.file] > 1 && (score -= 14),
1522 + |         0 === (fileCounts[pawn.file - 1] ?? 0) &&
1523 + |         0 === (fileCounts[pawn.file + 1] ?? 0) &&
1524 + |         (score -= 10),
1525 + |         isPassedPawn(pawn, pawns) && (score += (relativeRank - 1) * (relativeRank - 1) * 9));
1526 + |     }
1527 + |     return score;
1528 + | }
1529 + |
1530 + | function isPassedPawn(pawn, pawns) {
1531 + |     const enemyColor = (function oppositeColor(color) {
1532 + |         return 'w' === color ? 'b' : 'w';
1533 + |     })(pawn.color);
1534 + |     return !pawns.some(
1535 + |         (candidate) =>
1536 + |         !(candidate.color !== enemyColor || Math.abs(candidate.file - pawn.file) > 1) &&
1537 + |         ('w' === pawn.color ? candidate.rank > pawn.rank : candidate.rank < pawn.rank),
1538 + |         );
1539 + | }
1540 + |
1541 + | function openingPhaseWeight(nonPawnMaterial) {
1542 + |     return clamp(
1543 + |         (nonPawnMaterial - OPENING_PHASE_END_MATERIAL) /
1544 + |         (STARTING_NON_PAWN_MATERIAL - OPENING_PHASE_END_MATERIAL),
1545 + |         0,
1546 + |         1,
1547 + |         );
1548 + | }
1549 + |
1550 + | function openingPrinciplesScore(board, pawns, color, nonPawnMaterial) {

```

```

1546 + | | | | | const phase = openingPhaseWeight(nonPawnMaterial);
1547 + | | | | | if (phase ≤ 0) return 0;
1548 + | | | | | let score = (function centralLineOpeningScore(board, color) {
1549 + | | | | | | let score = 0;
1550 + | | | | | | return (
1551 + | | | | | | | hasPawnOnSquare(board, color, 'w' === color ? 'd2' : 'd7') || (score += 10),
1552 + | | | | | | | hasPawnOnSquare(board, color, 'w' === color ? 'e2' : 'e7') || (score += 12),
1553 + | | | | | | | score
1554 + | | | | | | );
1555 + | | | | | | })(board, color),
1556 + | | | | | | developedCenterPawns = 0;
1557 + | | | | | | for (const pawn of pawns) {
1558 + | | | | | | | if (pawn.color !== color) continue;
1559 + | | | | | | | const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
1560 + | | | | | | | 3 === pawn.file || 4 === pawn.file
1561 + | | | | | | | ? (4 === relativeRank || 5 === relativeRank
1562 + | | | | | | | | ? (score += 76)
1563 + | | | | | | | | : 3 === relativeRank && (score += 30),
1564 + | | | | | | | | relativeRank ≥ 3 && (developedCenterPawns += 1))
1565 + | | | | | | | : (2 !== pawn.file && 5 !== pawn.file) ||
1566 + | | | | | | | (4 === relativeRank || 5 === relativeRank
1567 + | | | | | | | | ? (score += 18)
1568 + | | | | | | | | : 3 === relativeRank && (score += 6));
1569 + | | | | | | }
1570 + | | | | | | return (developedCenterPawns ≥ 2 && (score += 16), Math.round(score * phase));
1571 + | | | | | | }
1572 + | | | | | function hasPawnOnSquare(board, color, square) {
1573 + | | | | | | for (const row of board)
1574 + | | | | | | | for (const piece of row)
1575 + | | | | | | | | if (piece?.color === color && 'p' === piece.type && piece.square === square) return !0;
1576 + | | | | | | | return !1;
1577 + | | | | | | }
1578 + | | | | | function developmentScore(board, color) {
1579 + | | | | | | const homeRank = 'w' === color ? '1' : '8',
1580 + | | | | | | | knightHomeSquares = 'w' === color ? new Set(['b1', 'g1']) : new Set(['b8', 'g8']),
1581 + | | | | | | | bishopHomeSquares = 'w' === color ? new Set(['c1', 'f1']) : new Set(['c8', 'f8']);
1582 + | | | | | | let score = 0;
1583 + | | | | | | for (const row of board)
1584 + | | | | | | | for (const piece of row)
1585 + | | | | | | | | null !== piece &&
1586 + | | | | | | | | piece.color === color &&
1587 + | | | | | | | | ('n' !== piece.type || knightHomeSquares.has(piece.square)) || (score += 18),
1588 + | | | | | | | | 'b' !== piece.type || bishopHomeSquares.has(piece.square) || (score += 16),
1589 + | | | | | | | | 'q' === piece.type &&
1590 + | | | | | | | | piece.square.endsWith(homeRank) &&
1591 + | | | | | | | | hasMovedMinorPieces(board, color) &&
1592 + | | | | | | | | (score -= 12));
1593 + | | | | | | return score;
1594 + | | | | | | }
1595 + | | | | | function hasMovedMinorPieces(board, color) {
1596 + | | | | | | const homeSquares =
1597 + | | | | | | | 'w' === color ? new Set(['b1', 'c1', 'f1', 'g1']) : new Set(['b8', 'c8', 'f8', 'g8']);
1598 + | | | | | | for (const row of board)
1599 + | | | | | | | for (const piece of row)
1600 + | | | | | | | | if (
1601 + | | | | | | | | | null !== piece &&
1602 + | | | | | | | | | piece.color === color &&
1603 + | | | | | | | | | ('n' === piece.type || 'b' === piece.type) &&
1604 + | | | | | | | | | homeSquares.has(piece.square)
1605 + | | | | | | | | )
1606 + | | | | | | | | return !1;
1607 + | | | | | | | return !0;
1608 + | | | | | | }
1609 + | | | | | function kingSafetyScore(kingSquare, pawns, color, nonPawnMaterial) {
1610 + | | | | | | if (!kingSquare || nonPawnMaterial < 1800) return 0;
1611 + | | | | | | const kingFile = fileIndex(kingSquare),
1612 + | | | | | | | kingRank = rankNumber(kingSquare),
1613 + | | | | | | | shieldRank = 'w' === color ? kingRank + 1 : kingRank - 1;
1614 + | | | | | | let score = 2 === kingFile || 6 === kingFile ? 28 : -18;
1615 + | | | | | | for (let file = kingFile - 1; file ≤ kingFile + 1; file += 1)
1616 + | | | | | | | file < 0 ||

```



```

1617 + |         file >= 8 ||
1618 + |         .(pawns.some(
1619 + |             (pawn) => pawn.color === color && pawn.file === file && pawn.rank === shieldRank,
1620 + |         ))
1621 + |         ? (score += 14)
1622 + |         : (score +=
1623 + |             openingCentralShieldScore(kingSquare, pawns, color, file, nonPawnMaterial) ?? -10));
1624 + |     return score;
1625 + | }
1626 + | function openingCentralShieldScore(kingSquare, pawns, color, file, nonPawnMaterial) {
1627 + |     const homeKingSquare = 'w' === color ? 'e1' : 'e8',
1628 + |           phase = openingPhaseWeight(nonPawnMaterial);
1629 + |     return kingSquare !== homeKingSquare || phase <= 0 || (3 !== file && 4 !== file)
1630 + |        ? null
1631 + |        : pawns.some((pawn) => {
1632 + |            if (pawn.color !== color || pawn.file !== file) return !1;
1633 + |            const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
1634 + |            return 3 === relativeRank || 4 === relativeRank;
1635 + |        })
1636 + |        ? Math.round(24 * phase - 10)
1637 + |        : null;
1638 + | }
1639 + | function terminalScore(chess, ply) {
1640 + |     return chess.isCheckmate() ? -1e6 + ply : chess.isDraw() ? 0 : evaluatePosition(chess);
1641 + | }
1642 + | function orderMoves(moves, preferredMove) {
1643 + |     return [...moves].sort(
1644 + |         (left, right) =>
1645 + |             moveOrderingScore(right, preferredMove) - moveOrderingScore(left, preferredMove),
1646 + |     );
1647 + | }
1648 + | function moveOrderingScore(move, preferredMove) {
1649 + |     let score = 0;
1650 + |     return (
1651 + |         preferredMove &&
1652 + |         (function isSameMove(move, command) {
1653 + |             return (
1654 + |                 move.from === command.from &&
1655 + |                 move.to === command.to &&
1656 + |                 (move.promotion ?? void 0) === (command.promotion ?? void 0)
1657 + |             );
1658 + |         })(move, preferredMove) &&
1659 + |         (score += 2e6),
1660 + |         move.san.includes('#') ? (score += 15e5) : move.san.includes('+') && (score += 16e3),
1661 + |         move.isCapture() &&
1662 + |         (score +=
1663 + |             12 * (move.captured ? PIECE_VALUES[move.captured] : PIECE_VALUES.p) -
1664 + |             PIECE_VALUES[move.piece]),
1665 + |         move.isPromotion() && move.promotion && (score += PIECE_VALUES[move.promotion] + 8e3),
1666 + |         (move.isKingsideCastle() || move.isQueensideCastle()) && (score += 1500),
1667 + |         (score += 2 * pieceSquareScore(move.piece, move.color, move.to)),
1668 + |         score
1669 + |     );
1670 + | }
1671 + | function checkTime(ctx) {
1672 + |     if (((ctx.nodes += 1), ctx.nodes % 2048 !== 0)) return;
1673 + |     const now = Date.now();
1674 + |     if (
1675 + |         (now - ctx.lastProgressAt >= 100 &&
1676 + |         publishSearchProgress(ctx, ctx.progressState, now - ctx.startTime, now),
1677 + |         now >= ctx.deadline)
1678 + |     ) {
1679 + |         throw new SearchTimeout();
1680 + |     }
1681 + |     function publishSearchProgress(
1682 + |         ctx,
1683 + |         progressState,
1684 + |         elapsedMs = Date.now() - ctx.startTime,
1685 + |         progressAt = Date.now(),
1686 + |     ) {
1687 + |         ((ctx.progressState = progressState),

```



```

1688 + | | | | | (ctx.lastProgressAt = progressAt),
1689 + | | | | | ctx.onProgress?.({
1690 + | | | | |   kind: 'progress',
1691 + | | | | |   id: ctx.requestId,
1692 + | | | | |   move: progressState.move,
1693 + | | | | |   depthReached: progressState.depthReached,
1694 + | | | | |   nodes: ctx.nodes,
1695 + | | | | |   elapsedMs: elapsedMs,
1696 + | | | | |   score: progressState.score,
1697 + | | | | |   debug: progressState.debug,
1698 + | | | | | }));
1699 + | | | | | }
1700 + | | | | | function isTacticalMove(move) {
1701 + | | | | |   return (
1702 + | | | | |     move.isCapture() || move.isPromotion() || move.san.includes('+') || move.san.includes('#')
1703 + | | | | |   );
1704 + | | | | | }
1705 + | | | | | function createDeterministicDebug(
1706 + | | | | |   candidates,
1707 + | | | | |   principal,
1708 + | | | | |   selected,
1709 + | | | | |   windowCp,
1710 + | | | | |   floorScore,
1711 + | | | | |   ticketPower,
1712 + | | | | | ) {
1713 + | | | | |   return {
1714 + | | | | |     candidates: sortCandidatesByScore(candidates).map((candidate) => ({
1715 + | | | | |       move: candidate.move,
1716 + | | | | |       san: candidate.san,
1717 + | | | | |       score: candidate.score,
1718 + | | | | |       tickets: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
1719 + | | | | |       probability: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
1720 + | | | | |       selected: isSameMoveCommand(candidate.move, selected.move),
1721 + | | | | |       principal: isSameMoveCommand(candidate.move, principal.move),
1722 + | | | | |     })),
1723 + | | | | |     bestScore: principal.score,
1724 + | | | | |     windowCp: windowCp,
1725 + | | | | |     floorScore: floorScore,
1726 + | | | | |     ticketPower: ticketPower,
1727 + | | | | |     totalTickets: 1,
1728 + | | | | |   };
1729 + | | | | | }
1730 + | | | | | function createWeightedDebug(
1731 + | | | | |   candidates,
1732 + | | | | |   principal,
1733 + | | | | |   selected,
1734 + | | | | |   weightedCandidates,
1735 + | | | | |   windowCp,
1736 + | | | | |   floorScore,
1737 + | | | | |   ticketPower,
1738 + | | | | |   totalTickets,
1739 + | | | | | ) {
1740 + | | | | |   return {
1741 + | | | | |     candidates: sortCandidatesByScore(candidates).map((candidate) => {
1742 + | | | | |       const weightedCandidate = weightedCandidates.find(({ candidate: weighted }) =>
1743 + | | | | |         isSameMoveCommand(weighted.move, candidate.move),
1744 + | | | | |       ),
1745 + | | | | |       tickets = weightedCandidate ? tickets ?? 0;
1746 + | | | | |     },
1747 + | | | | |     move: candidate.move,
1748 + | | | | |     san: candidate.san,
1749 + | | | | |     score: candidate.score,
1750 + | | | | |     tickets: tickets,
1751 + | | | | |     probability: totalTickets > 0 ? tickets / totalTickets : 0,
1752 + | | | | |     selected: isSameMoveCommand(candidate.move, selected.move),
1753 + | | | | |     principal: isSameMoveCommand(candidate.move, principal.move),
1754 + | | | | |   };
1755 + | | | | | },
1756 + | | | | | bestScore: principal.score,
1757 + | | | | | windowCp: windowCp,
1758 + | | | | | floorScore: floorScore,

```

```

1759 + | | | | | ticketPower: ticketPower,
1760 + | | | | | totalTickets: totalTickets,
1761 + | | | | | };
1762 + | | | | | }
1763 + | | | | | function sortCandidatesByScore(candidates) {
1764 + | | | | |   return [ ... candidates ].sort((left, right) => right.score - left.score);
1765 + | | | | | }
1766 + | | | | | function toMoveCommand(move) {
1767 + | | | | |   return move.promotion
1768 + | | | | |   ? { from: move.from, to: move.to, promotion: move.promotion }
1769 + | | | | |   : { from: move.from, to: move.to };
1770 + | | | | | }
1771 + | | | | | function isSameMoveCommand(left, right) {
1772 + | | | | |   return (
1773 + | | | | |     left.from === right.from &&
1774 + | | | | |     left.to === right.to &&
1775 + | | | | |     (left.promotion ?? void 0) === (right.promotion ?? void 0)
1776 + | | | | |   );
1777 + | | | | | }
1778 + | | | | | function fileIndex(square) {
1779 + | | | | |   return square.charCodeAt(0) - 97;
1780 + | | | | | }
1781 + | | | | | function rankNumber(square) {
1782 + | | | | |   return Number(square[1]);
1783 + | | | | | }
1784 + | | | | | function clamp(value, min, max) {
1785 + | | | | |   return Math.min(max, Math.max(min, value));
1786 + | | | | | }
1787 + | | | | | class SearchTimeout extends Error {}
1788 + | | | | | const workerScope = globalThis;
1789 + | | | | | workerScope.addEventListener('message', (event) => {
1790 + | | | | |   const response = (function searchBestMove(request, onProgress) {
1791 + | | | | |     const chess = new Chess(request.fen),
1792 + | | | | |     settings = (function normalizeAiSettings(settings) {
1793 + | | | | |       const timeLimitMs = Math.max(100, Math.round(settings.timeLimitMs));
1794 + | | | | |       return {
1795 + | | | | |         preset: settings.preset,
1796 + | | | | |         maxDepth:
1797 + | | | | |           ((depth = settings.maxDepth),
1798 + | | | | |             (preset = settings.preset),
1799 + | | | | |             'hard' === preset ? HARD_MAX_DEPTH : clamp(Math.round(depth), 1, 5)),
1800 + | | | | |             timeLimitMs: timeLimitMs,
1801 + | | | | |             randomness: 30,
1802 + | | | | |             quiescence: !0,
1803 + | | | | |           };
1804 + | | | | |       var depth, preset;
1805 + | | | | |     })(request.settings),
1806 + | | | | |     legalMoves = chess.moves({ verbose: !0 }),
1807 + | | | | |     startTime = Date.now(),
1808 + | | | | |     ctx = {
1809 + | | | | |       requestId: request.id,
1810 + | | | | |       settings: settings,
1811 + | | | | |       deadline: startTime + settings.timeLimitMs,
1812 + | | | | |       startTime: startTime,
1813 + | | | | |       nodes: 0,
1814 + | | | | |       onProgress: onProgress,
1815 + | | | | |       lastProgressAt: startTime,
1816 + | | | | |       progressState: { move: null, depthReached: 0, score: 0 },
1817 + | | | | |     };
1818 + | | | | |     if (0 === legalMoves.length)
1819 + | | | | |       return {
1820 + | | | | |         id: request.id,
1821 + | | | | |         move: null,
1822 + | | | | |         depthReached: 0,
1823 + | | | | |         nodes: 0,
1824 + | | | | |         elapsedMs: 0,
1825 + | | | | |         score: terminalScore(chess, 0),
1826 + | | | | |       };
1827 + | | | | |     let debug,
1828 + | | | | |     best = null,
1829 + | | | | |     previousBestMove = null,

```

```

1830 + | | | | | previousPrincipal = null,
1831 + | | | | | depthReached = 0;
1832 + | | | | | for (let depth = 1; depth ≤ settings.maxDepth; depth += 1) {
1833 + | | | | | try {
1834 + | | | | |   const result = searchRoot(chess, depth, ctx, previousBestMove),
1835 + | | | | |   elapsedMs = Date.now() - startTime;
1836 + | | | | |   if (
1837 + | | | | |     ((best = result.selected),
1838 + | | | | |     (debug = result.debug),
1839 + | | | | |     (previousBestMove = result.principal.move),
1840 + | | | | |     (depthReached = depth),
1841 + | | | | |     publishSearchProgress(
1842 + | | | | |       ctx,
1843 + | | | | |       { move: best.move, depthReached: depthReached, score: best.score, debug: debug },
1844 + | | | | |       elapsedMs,
1845 + | | | | |     ),
1846 + | | | | |     shouldStopAfterStableDepth(result, previousPrincipal, settings))
1847 + | | | | |   )
1848 + | | | | |     break;
1849 + | | | | |   previousPrincipal = result.principal;
1850 + | | | | | } catch (error) {
1851 + | | | | |   if (error instanceof SearchTimeout) break;
1852 + | | | | |   throw error;
1853 + | | | | | }
1854 + | | | | | if (Date.now() ≥ ctx.deadline) break;
1855 + | | | | | }
1856 + | | | | | return (
1857 + | | | | |   (best ??= (function chooseFallbackMove(chess) {
1858 + | | | | |     const [move] = orderMoves(chess.moves({ verbose: !0 }), null);
1859 + | | | | |     if (!move) return { move: { from: 'a1', to: 'a1' }, score: terminalScore(chess, 0) };
1860 + | | | | |     chess.move(toMoveCommand(move));
1861 + | | | | |     const score = -evaluatePosition(chess);
1862 + | | | | |     return (chess.undo(), { move: toMoveCommand(move), score: score });
1863 + | | | | |   })(chess)),
1864 + | | | | |   {
1865 + | | | | |     id: request.id,
1866 + | | | | |     move: best.move,
1867 + | | | | |     depthReached: depthReached,
1868 + | | | | |     nodes: ctx.nodes,
1869 + | | | | |     elapsedMs: Date.now() - startTime,
1870 + | | | | |     score: best.score,
1871 + | | | | |     debug: debug,
1872 + | | | | |   }
1873 + | | | | | );
1874 + | | | | | })(event.data, (progress) => {
1875 + | | | | |   workerScope.postMessage(progress);
1876 + | | | | | });
1877 + | | | | | workerScope.postMessage({ ... response, kind: 'result' });
1878 + | | | | | });
1879 + | | | | | })();
1880 + | | | | | </script>
1881 + | | | | |
1882 + | | | | | <script>
1883 + | | | | |   (() => {
1884 + | | | | |     const element = document.getElementById('ts-chess-shared-core-source');
1885 + | | | | |     if (!element) {
1886 + | | | | |       throw new Error('Missing shared chess runtime source: ts-chess-shared-core-source');
1887 + | | | | |     }
1888 + | | | | |     new Function(element.textContent || '')();
1889 + | | | | |   })();
1890 + | | | | | </script>
1891 + | | | | | <script src="/main.js" defer></script>
1892 + | | | | | <link rel="stylesheet" href="/style.css" />
9 1893 | | | | | </head>
10 1894 | | | | | <body>
11 1895 | | | | |   <main id="chess-app" class="chess-app">
12 1896 | | | | |     <aside class="control-panel" aria-label="게임 컨트롤">
13 1897 | | | | |       <section class="control-group" aria-label="모드">
14 1898 | | | | |         <span class="control-label">모드</span>
15 1899 | | | | |         <div class="segmented-control" role="group" aria-label="플레이 모드">
16 1900 | | | | |           <button class="segmented-button" type="button" data-mode="local">로컬 2인</button>

```

```

17 1901 | | | | |<button class="segmented-button" type="button" data-mode="ai">AI · 상대</button>
18 1902 | | | | |</div>
19 1903 | | | | |</section>
20 1904 | | | | |
21 1905 | | | | |<section id="color-controls" class="control-group" aria-label="진영">
22 1906 | | | | |<span class="control-label">진영</span>
23 1907 | | | | |<div class="segmented-control" role="group" aria-label="인간 · 진영">
24 1908 | | | | |<button class="segmented-button" type="button" data-color="w">백</button>
25 1909 | | | | |<button class="segmented-button" type="button" data-color="b">흑</button>
26 1910 | | | | |</div>
27 1911 | | | | |</section>
28 1912 | | | | |
29 1913 | | | | |<section id="ai-controls" class="control-group ai-settings" aria-label="AI · 설정">
30 1914 | | | | |<span class="control-label">AI · 설정</span>
31 1915 | | | | |<div
32 1916 | | | | |<class="segmented-control segmented-control-three"
33 1917 | | | | |<role="group"
34 1918 | | | | |<aria-label="AI · 프리셋"
35 1919 | | | | |>
36 1920 | | | | |<button class="segmented-button" type="button" data-preset="easy">쉬움</button>
37 1921 | | | | |<button class="segmented-button" type="button" data-preset="normal">보통</button>
38 1922 | | | | |<button class="segmented-button" type="button" data-preset="hard">어려움</button>
39 1923 | | | | |</div>
40 1924 | | | | |<label class="range-control">
41 1925 | | | | |<span>최대 깊이<strong id="ai-depth-value"></strong></span>
42 1926 | | | | |<input id="ai-depth-input" type="range" min="1" max="5" step="1" />
43 1927 | | | | |</label>
44 1928 | | | | |<label class="range-control">
45 1929 | | | | |<span>시간 제한<strong id="ai-time-value"></strong></span>
46 1930 | | | | |<input id="ai-time-input" type="range" min="500" max="5000" step="250" />
47 1931 | | | | |</label>
48 1932 | | | | |</section>
49 1933 | | | | |
50 1934 | | | | |<section class="control-group clock-settings" aria-label="시간 · 설정">
51 1935 | | | | |<span class="control-label">시간</span>
52 1936 | | | | |<div class="clock-preset-control" role="group" aria-label="시간 · 프리셋">
53 1937 | | | | |<button class="segmented-button" type="button" data-clock-preset="1/1">1/1</button>
54 1938 | | | | |<button class="segmented-button" type="button" data-clock-preset="3/2">3/2</button>
55 1939 | | | | |<button class="segmented-button" type="button" data-clock-preset="5/3">5/3</button>
56 1940 | | | | |<button class="segmented-button" type="button" data-clock-preset="10/5">10/5</button>
57 1941 | | | | |<button class="segmented-button" type="button" data-clock-preset="15/10">15/10</button>
58 1942 | | | | |</div>
59 1943 | | | | |</section>
60 1944 | | | | |
61 1945 | | | | |<section class="action-row" aria-label="게임 · 동작">
62 1946 | | | | |<button id="new-game-button" class="primary-button" type="button">새 · 게임</button>
63 1947 | | | | |<button id="undo-button" class="ghost-button" type="button">되돌리기</button>
64 1948 | | | | |</section>
65 1949 | | | | |</aside>
66 1950 | | | | |
67 1951 | | | | |<section class="board-stage" aria-label="체스판">
68 1952 | | | | |<div id="chess-board" class="chess-board" role="grid" aria-label="체스판"></div>
69 1953 | | | | |</section>
70 1954 | | | | |
71 1955 | | | | |<aside class="clock-panel" aria-label="체스 · 타이머">
72 1956 | | | | |<div id="clock-display" class="clock-display">
73 1957 | | | | |<div id="white-clock" class="clock-card">
74 1958 | | | | |<span class="clock-side">백</span>
75 1959 | | | | |<strong id="white-clock-time" class="clock-time"></strong>
76 1960 | | | | |</div>
77 1961 | | | | |<div id="black-clock" class="clock-card">
78 1962 | | | | |<span class="clock-side">흑</span>
79 1963 | | | | |<strong id="black-clock-time" class="clock-time"></strong>
80 1964 | | | | |</div>
81 1965 | | | | |</div>
82 1966 | | | | |</aside>
83 1967 | | | | |
84 1968 | | | | |<aside class="info-panel" aria-label="게임 · 정보">
85 1969 | | | | |<section class="status-panel" aria-live="polite">
86 1970 | | | | |<span class="control-label">상태</span>
87 1971 | | | | |<strong id="game-status" class="game-status"></strong>

```

```

88 1972 | | | | | <dl class="status-grid">
89 1973 | | | | | | <div>
90 1974 | | | | | | | <dt>차례</dt>
91 1975 | | | | | | | <dd id="turn-meta"></dd>
92 1976 | | | | | | | </div>
93 1977 | | | | | | | <div>
94 1978 | | | | | | | <dt>모드</dt>
95 1979 | | | | | | | <dd id="mode-meta"></dd>
96 1980 | | | | | | | </div>
97 1981 | | | | | | | <div>
98 1982 | | | | | | | <dt>수순</dt>
99 1983 | | | | | | | <dd id="move-meta"></dd>
100 1984 | | | | | | | </div>
101 1985 | | | | | | | </dl>
102 1986 | | | | | | | </section>
103 1987 | | | | | | |
104 1988 | | | | | | | <section id="ai-candidate-panel" class="ai-candidate-panel" aria-label="AI 정보" hidden>
105 1989 | | | | | | | | <div class="ai-candidate-header">
106 1990 | | | | | | | | | <span class="control-label">AI 정보</span>
107 1991 | | | | | | | | | <span id="ai-candidate-meta" class="ai-candidate-meta"></span>
108 1992 | | | | | | | | | </div>
109 1993 | | | | | | | | | <dl class="ai-info-grid">
110 1994 | | | | | | | | | | <div>
111 1995 | | | | | | | | | | | <dt>상태</dt>
112 1996 | | | | | | | | | | | <dd id="ai-info-status"></dd>
113 1997 | | | | | | | | | | | </div>
114 1998 | | | | | | | | | | | <div>
115 1999 | | | | | | | | | | | <dt>난이도</dt>
116 2000 | | | | | | | | | | | <dd id="ai-info-preset"></dd>
117 2001 | | | | | | | | | | | </div>
118 2002 | | | | | | | | | | | <div>
119 2003 | | | | | | | | | | | <dt>인간</dt>
120 2004 | | | | | | | | | | | <dd id="ai-info-human"></dd>
121 2005 | | | | | | | | | | | </div>
122 2006 | | | | | | | | | | | <div>
123 2007 | | | | | | | | | | | <dt>depth</dt>
124 2008 | | | | | | | | | | | <dd id="ai-info-depth"></dd>
125 2009 | | | | | | | | | | | </div>
126 2010 | | | | | | | | | | | <div>
127 2011 | | | | | | | | | | | <dt>nodes</dt>
128 2012 | | | | | | | | | | | <dd id="ai-info-nodes"></dd>
129 2013 | | | | | | | | | | | </div>
130 2014 | | | | | | | | | | | <div>
131 2015 | | | | | | | | | | | <dt>score</dt>
132 2016 | | | | | | | | | | | <dd id="ai-info-score"></dd>
133 2017 | | | | | | | | | | | </div>
134 2018 | | | | | | | | | | | </dl>
135 2019 | | | | | | | | | | | <div class="ai-candidate-table" role="table" aria-label="AI 후보 수 평가">
136 2020 | | | | | | | | | | | | <div class="ai-candidate-row ai-candidate-head" role="row">
137 2021 | | | | | | | | | | | | | <span role="columnheader">SAN</span>
138 2022 | | | | | | | | | | | | | <span role="columnheader">cp</span>
139 2023 | | | | | | | | | | | | | <span role="columnheader">weight</span>
140 2024 | | | | | | | | | | | | | <span role="columnheader">prob</span>
141 2025 | | | | | | | | | | | | | </div>
142 2026 | | | | | | | | | | | | | <div id="ai-candidate-list" class="ai-candidate-list"></div>
143 2027 | | | | | | | | | | | | | </div>
144 2028 | | | | | | | | | | | | | </section>
145 2029 | | | | | | | | | | | | |
146 2030 | | | | | | | | | | | | | <section class="history-panel" aria-label="기보">
147 2031 | | | | | | | | | | | | | | <div class="history-header">
148 2032 | | | | | | | | | | | | | | | <span class="control-label">기보</span>
149 2033 | | | | | | | | | | | | | | | </div>
150 2034 | | | | | | | | | | | | | | | <div id="move-list" class="move-list"></div>
151 2035 | | | | | | | | | | | | | | | </section>
152 2036 | | | | | | | | | | | | | | | </aside>
153 2037 | | | | | | | | | | | | | | |
154 2038 | | | | | | | | | | | | | | | <div id="promotion-overlay" class="promotion-overlay" hidden>
155 2039 | | | | | | | | | | | | | | | | <div
156 2040 | | | | | | | | | | | | | | | | | class="promotion-dialog"
157 2041 | | | | | | | | | | | | | | | | | role="dialog"
158 2042 | | | | | | | | | | | | | | | | | aria-modal="true"

```

```

159 2043 | | | | |aria-labelledby="promotion-title"
160 2044 | | | | |>
161 2045 | | | | |<h2.id="promotion-title">승격</h2>
162 2046 | | | | |<div
163 2047 | | | | |.id="promotion-choices"
164 2048 | | | | |.class="promotion-choices"
165 2049 | | | | |.role="group"
166 2050 | | | | |.aria-label="승격·기물"
167 2051 | | | | |></div>
168 2052 | | | | |</div>
169 2053 | | | | |</div>
170 2054 | | | | |</main>
171 2055 | | | | |</body>
172 2056 | | | | |</html>

```

==== M main.js (63353 B / 9bee5a3328fc → 33823 B / 04a8e344941e) ====

```

1 - const MASK64 = 0xffffffffffffffff;
2 - function rotl(x, k) {
3 -   return 0xffffffffffffffff & ((x << k) | (x >> (64n - k)));
4 - }
5 - function wrappingMul(x, y) {
6 -   return (x * y) & MASK64;
7 - }
8 - const rand = (function xoroshiro128(state) {
9 -   return function () {
10 -     let s0 = BigInt(state & MASK64),
11 -     s1 = BigInt((state >> 64n) & MASK64);
12 -     const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
13 -     return (
14 -       (s1 ^ s0),
15 -       (s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64),
16 -       (s1 = rotl(s1, 37n)),
17 -       (state = (s1 << 64n) | s0),
18 -       result
19 -     );
20 -   };
21 - })(0xa187eb39cdcaed8f31c4b365b102e01en),
22 - PIECE_KEYS = Array.from({ length: 2 }, () =>
23 -   Array.from({ length: 6 }, () => Array.from({ length: 128 }, () => rand()))),
24 - ),
25 - EP_KEYS = Array.from({ length: 8 }, () => rand()),
26 - CASTLING_KEYS = Array.from({ length: 16 }, () => rand()),
27 - SIDE_KEY = rand(),
28 - DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR·w·KQkq·--0·1';
29 - class Move {
30 -   color;
31 -   from;
32 -   to;
33 -   piece;
34 -   captured;
35 -   promotion;
36 -   flags;
37 -   san;
38 -   lan;
39 -   before;
40 -   after;
41 -   constructor(chess, internal) {
42 -     const {
43 -       color: color,
44 -       piece: piece,
45 -       from: from,
46 -       to: to,
47 -       flags: flags,
48 -       captured: captured,
49 -       promotion: promotion,
50 -     } = internal,
51 -     fromAlgebraic = algebraic(from),
52 -     toAlgebraic = algebraic(to);
53 -     ((this.color = color),
54 -     (this.piece = piece),
55 -     (this.from = fromAlgebraic),

```

```

56 - | | | | | (this.to = toAlgebraic),
57 - | | | | | (this.san = chess._moveToSan(internal, chess._moves({ legal: !0 }))),
58 - | | | | | (this.lan = fromAlgebraic + toAlgebraic),
59 - | | | | | (this.before = chess.fen()),
60 - | | | | | chess._makeMove(internal),
61 - | | | | | (this.after = chess.fen()),
62 - | | | | | chess._undoMove(),
63 - | | | | | (this.flags = '');
64 - | | | | | for (const flag in BITS) BITS[flag] & flags && (this.flags += FLAGS[flag]);
65 - | | | | | (captured && (this.captured = captured),
66 - | | | | | promotion && ((this.promotion = promotion), (this.lan += promotion)));
67 - | | | | | }
68 - | | | | | isCapture() {
69 - | | | | | return this.flags.indexOf(FLAGS.CAPTURE) > -1;
70 - | | | | | }
71 - | | | | | isPromotion() {
72 - | | | | | return this.flags.indexOf(FLAGS.PROMOTION) > -1;
73 - | | | | | }
74 - | | | | | isEnPasant() {
75 - | | | | | return this.flags.indexOf(FLAGS.EP_CAPTURE) > -1;
76 - | | | | | }
77 - | | | | | isKingsideCastle() {
78 - | | | | | return this.flags.indexOf(FLAGS.KSIDE_CASTLE) > -1;
79 - | | | | | }
80 - | | | | | isQueensideCastle() {
81 - | | | | | return this.flags.indexOf(FLAGS.QSIDE_CASTLE) > -1;
82 - | | | | | }
83 - | | | | | isBigPawn() {
84 - | | | | | return this.flags.indexOf(FLAGS.BIG_PAWN) > -1;
85 - | | | | | }
86 - | | | | | }
87 - | | | | | const FLAGS = {
88 - | | | | | NORMAL: 'n',
89 - | | | | | CAPTURE: 'c',
90 - | | | | | BIG_PAWN: 'b',
91 - | | | | | EP_CAPTURE: 'e',
92 - | | | | | PROMOTION: 'p',
93 - | | | | | KSIDE_CASTLE: 'k',
94 - | | | | | QSIDE_CASTLE: 'q',
95 - | | | | | NULL_MOVE: '-',
96 - | | | | | },
97 - | | | | | BITS = {
98 - | | | | | NORMAL: 1,
99 - | | | | | CAPTURE: 2,
100 - | | | | | BIG_PAWN: 4,
101 - | | | | | EP_CAPTURE: 8,
102 - | | | | | PROMOTION: 16,
103 - | | | | | KSIDE_CASTLE: 32,
104 - | | | | | QSIDE_CASTLE: 64,
105 - | | | | | NULL_MOVE: 128,
106 - | | | | | },
107 - | | | | | 0x88 = {
108 - | | | | | a8: 0,
109 - | | | | | b8: 1,
110 - | | | | | c8: 2,
111 - | | | | | d8: 3,
112 - | | | | | e8: 4,
113 - | | | | | f8: 5,
114 - | | | | | g8: 6,
115 - | | | | | h8: 7,
116 - | | | | | a7: 16,
117 - | | | | | b7: 17,
118 - | | | | | c7: 18,
119 - | | | | | d7: 19,
120 - | | | | | e7: 20,
121 - | | | | | f7: 21,
122 - | | | | | g7: 22,
123 - | | | | | h7: 23,
124 - | | | | | a6: 32,
125 - | | | | | b6: 33,
126 - | | | | | c6: 34,

```

```

127 - - d6: 35,
128 - - e6: 36,
129 - - f6: 37,
130 - - g6: 38,
131 - - h6: 39,
132 - - a5: 48,
133 - - b5: 49,
134 - - c5: 50,
135 - - d5: 51,
136 - - e5: 52,
137 - - f5: 53,
138 - - g5: 54,
139 - - h5: 55,
140 - - a4: 64,
141 - - b4: 65,
142 - - c4: 66,
143 - - d4: 67,
144 - - e4: 68,
145 - - f4: 69,
146 - - g4: 70,
147 - - h4: 71,
148 - - a3: 80,
149 - - b3: 81,
150 - - c3: 82,
151 - - d3: 83,
152 - - e3: 84,
153 - - f3: 85,
154 - - g3: 86,
155 - - h3: 87,
156 - - a2: 96,
157 - - b2: 97,
158 - - c2: 98,
159 - - d2: 99,
160 - - e2: 100,
161 - - f2: 101,
162 - - g2: 102,
163 - - h2: 103,
164 - - a1: 112,
165 - - b1: 113,
166 - - c1: 114,
167 - - d1: 115,
168 - - e1: 116,
169 - - f1: 117,
170 - - g1: 118,
171 - - h1: 119,
172 - },
173 - PAWN_OFFSETS = { b: [16, 32, 17, 15], w: [-16, -32, -17, -15] },
174 - PIECE_OFFSETS = {
175 - n: [-18, -33, -31, -14, 18, 33, 31, 14],
176 - b: [-17, -15, 17, 15],
177 - r: [-16, 1, 16, -1],
178 - q: [-17, -16, -15, 1, 17, 16, 15, -1],
179 - k: [-17, -16, -15, 1, 17, 16, 15, -1],
180 - },
181 - ATTACKS = [
182 - 20, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20,
183 - 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 20,
184 - 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20,
185 - 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 24, 24, 24, 24, 24, 24, 56,
186 - 0, 56, 24, 24, 24, 24, 24, 24, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
187 - 0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0,
188 - 0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0,
189 - 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 0,
190 - 20,
191 - ],
192 - RAYS = [
193 - 17, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 15,
194 - 0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15, 0, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0, 0, 15,
195 - 0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 0, 17, 0, 16, 0, 15,
196 - 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, -1,
197 - -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,

```



```

198 - | -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0, 0, -17, 0, 0, 0, 0, 0, 0,
199 - | 0, 0, -15, 0, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, 0, -17,
200 - | 0, 0, 0, 0, -15, 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, -17, 0, 0, -15, 0, 0, 0, 0, 0, -16, 0, 0,
201 - | 0, 0, 0, 0, -17,
202 - | ],
203 - | PIECE_MASKS = { p: 1, n: 2, b: 4, r: 8, q: 16, k: 32 },
204 - | PROMOTIONS = ['n', 'b', 'r', 'q'],
205 - | ROOKS = {
206 - |   w: [
207 - |     { square: 0x88.a1, flag: BITS.QSIDE_CASTLE },
208 - |     { square: 0x88.h1, flag: BITS.KSIDE_CASTLE },
209 - |   ],
210 - |   b: [
211 - |     { square: 0x88.a8, flag: BITS.QSIDE_CASTLE },
212 - |     { square: 0x88.h8, flag: BITS.KSIDE_CASTLE },
213 - |   ],
214 - | },
215 - | SECOND_RANK = { b: 1, w: 6 };
216 - | function rank(square) {
217 - |   return square >> 4;
218 - | }
219 - | function file(square) {
220 - |   return 15 & square;
221 - | }
222 - | function isDigit(c) {
223 - |   return -1 !== '0123456789'.indexOf(c);
224 - | }
225 - | function algebraic(square) {
226 - |   const f = file(square),
227 - |         r = rank(square);
228 - |   return 'abcdefgh'.substring(f, f + 1) + '87654321'.substring(r, r + 1);
229 - | }
230 - | function swapColor(color) {
231 - |   return 'w' === color ? 'b' : 'w';
232 - | }
233 - | function addMove(moves, color, from, to, piece, captured = void 0, flags = BITS.NORMAL) {
234 - |   const r = rank(to);
235 - |   if ('p' !== piece || (7 !== r && 0 !== r))
236 - |     moves.push({
237 - |       color: color,
238 - |       from: from,
239 - |       to: to,
240 - |       piece: piece,
241 - |       captured: captured,
242 - |       flags: flags,
243 - |     });
244 - |   else
245 - |     for (let i = 0; i < PROMOTIONS.length; i++) {
246 - |       const promotion = PROMOTIONS[i];
247 - |       moves.push({
248 - |         color: color,
249 - |         from: from,
250 - |         to: to,
251 - |         piece: piece,
252 - |         captured: captured,
253 - |         promotion: promotion,
254 - |         flags: flags | BITS.PROMOTION,
255 - |       });
256 - |     }
257 - | }
258 - | class Chess {
259 - |   _board = new Array(128);
260 - |   _turn = 'w';
261 - |   _kings = { w: -1, b: -1 };
262 - |   _epSquare = -1;
263 - |   _halfMoves = 0;
264 - |   _moveNumber = 0;
265 - |   _history = [];
266 - |   _castling = { w: 0, b: 0 };
267 - |   _hash = 0n;
268 - |   _positionCount = new Map();

```

```

269 - | constructor(fen = DEFAULT_POSITION, { skipValidation: skipValidation = !1 } = {}) {
270 - |   this.load(fen, { skipValidation: skipValidation });
271 - | }
272 - | clear() {
273 - |   ((this._board = new Array(128)),
274 - |   (this._kings = { w: -1, b: -1 }),
275 - |   (this._turn = 'w'),
276 - |   (this._castling = { w: 0, b: 0 }),
277 - |   (this._epSquare = -1),
278 - |   (this._halfMoves = 0),
279 - |   (this._moveNumber = 1),
280 - |   (this._history = []),
281 - |   (this._hash = this._computeHash()),
282 - |   (this._positionCount = new Map()));
283 - | }
284 - | load(fen, { skipValidation: skipValidation = !1 } = {}) {
285 - |   let tokens = fen.split(/\s+/);
286 - |   if (tokens.length ≥ 2 && tokens.length < 6) {
287 - |     const adjustments = ['- ', '- ', '0', '1'];
288 - |     fen = tokens.concat(adjustments.slice(-(6 - tokens.length))).join(' ');
289 - |   }
290 - |   if (((tokens = fen.split(/\s+/)), !skipValidation)) {
291 - |     const { ok: ok, error: error } = (function validateFen(fen) {
292 - |       const tokens = fen.split(/\s+/);
293 - |       if (6 !== tokens.length)
294 - |         return { ok: !1, error: 'Invalid FEN: must contain six space-delimited fields' };
295 - |       const moveNumber = parseInt(tokens[5], 10);
296 - |       if (isNaN(moveNumber) || moveNumber ≤ 0)
297 - |         return { ok: !1, error: 'Invalid FEN: move number must be a positive integer' };
298 - |       const halfMoves = parseInt(tokens[4], 10);
299 - |       if (isNaN(halfMoves) || halfMoves < 0)
300 - |         return {
301 - |           ok: !1,
302 - |           error: 'Invalid FEN: half move counter number must be a non-negative integer',
303 - |         };
304 - |       if (!/^(?![abcdefgh][36])$/ .test(tokens[3]))
305 - |         return { ok: !1, error: 'Invalid FEN: en-passant square is invalid' };
306 - |       if (/^[^kKqQ-]/ .test(tokens[2]))
307 - |         return { ok: !1, error: 'Invalid FEN: castling availability is invalid' };
308 - |       if (!/^(w|b)$/ .test(tokens[1]))
309 - |         return { ok: !1, error: 'Invalid FEN: side-to-move is invalid' };
310 - |       const rows = tokens[0].split('/');
311 - |       if (8 !== rows.length)
312 - |         return { ok: !1, error: 'Invalid FEN: piece data does not contain 8 '/'-delimited rows' };
313 - |       for (let i = 0; i < rows.length; i++) {
314 - |         let sumFields = 0,
315 - |             previousWasNumber = !1;
316 - |         for (let k = 0; k < rows[i].length; k++)
317 - |           if (isDigit(rows[i][k])) {
318 - |             if (previousWasNumber)
319 - |               return { ok: !1, error: 'Invalid FEN: piece data is invalid (consecutive number)' };
320 - |             ((sumFields += parseInt(rows[i][k], 10)), (previousWasNumber = !0));
321 - |           } else {
322 - |             if (!/^[prnbqkPRNBQK]$/.test(rows[i][k]))
323 - |               return { ok: !1, error: 'Invalid FEN: piece data is invalid (invalid piece)' };
324 - |             ((sumFields += 1), (previousWasNumber = !1));
325 - |           }
326 - |       if (8 !== sumFields)
327 - |         return {
328 - |           ok: !1,
329 - |           error: 'Invalid FEN: piece data is invalid (too many squares in rank)',
330 - |         };
331 - |     }
332 - |     if (('3' === tokens[3][1] && 'w' === tokens[1]) || ('6' === tokens[3][1] && 'b' === tokens[1]))
333 - |       return { ok: !1, error: 'Invalid FEN: illegal en-passant square' };
334 - |     const kings = [
335 - |       { color: 'white', regex: /K/g },
336 - |       { color: 'black', regex: /k/g },
337 - |     ];
338 - |     for (const { color: color, regex: regex } of kings) {
339 - |       if (!regex.test(tokens[0]))

```

```

340 - | | | | | return { ok: !1, error: `Invalid FEN: missing ${color} king` };
341 - | | | | | if ((tokens[0].match(regex) || []).length > 1)
342 - | | | | | return { ok: !1, error: `Invalid FEN: too many ${color} kings` };
343 - | | | | | }
344 - | | | | | return Array.from(rows[0] + rows[7]).some((char) => 'P' === char.toUpperCase())
345 - | | | | | ? { ok: !1, error: 'Invalid FEN: some pawns are on the edge rows' }
346 - | | | | | : { ok: !0 };
347 - | | | | | })(fen);
348 - | | | | | if (!ok) throw new Error(error);
349 - | | | | | }
350 - | | | | | const position = tokens[0];
351 - | | | | | let square = 0;
352 - | | | | | this.clear();
353 - | | | | | for (let i = 0; i < position.length; i++) {
354 - | | | | |   const piece = position.charAt(i);
355 - | | | | |   if ('/' === piece) square += 8;
356 - | | | | |   else if (isDigit(piece)) square += parseInt(piece, 10);
357 - | | | | |   else {
358 - | | | | |     const color = piece < 'a' ? 'w' : 'b';
359 - | | | | |     (this._put({ type: piece.toLowerCase(), color: color }, algebraic(square)), square++);
360 - | | | | |   }
361 - | | | | | }
362 - | | | | | ((this._turn = tokens[1]),
363 - | | | | | tokens[2].indexOf('K') > -1 && (this._castling.w |= BITS.KSIDE_CASTLE),
364 - | | | | | tokens[2].indexOf('Q') > -1 && (this._castling.w |= BITS.QSIDE_CASTLE),
365 - | | | | | tokens[2].indexOf('k') > -1 && (this._castling.b |= BITS.KSIDE_CASTLE),
366 - | | | | | tokens[2].indexOf('q') > -1 && (this._castling.b |= BITS.QSIDE_CASTLE),
367 - | | | | | (this._epSquare = '-' === tokens[3] ? -1 : 0x88[tokens[3]]),
368 - | | | | | (this._halfMoves = parseInt(tokens[4], 10)),
369 - | | | | | (this._moveNumber = parseInt(tokens[5], 10)),
370 - | | | | | (this._hash = this._computeHash()),
371 - | | | | | this._incPositionCount());
372 - | | | | | }
373 - | | | | | fen({ forceEnpassantSquare: forceEnpassantSquare = !1 } = {}) {
374 - | | | | |   let empty = 0,
375 - | | | | |   fen = '';
376 - | | | | |   for (let i = 0x88.a8; i <= 0x88.h1; i++) {
377 - | | | | |     if (this._board[i]) {
378 - | | | | |       empty > 0 && ((fen += empty), (empty = 0));
379 - | | | | |       const { color: color, type: piece } = this._board[i];
380 - | | | | |       fen += 'w' === color ? piece.toUpperCase() : piece.toLowerCase();
381 - | | | | |     } else empty++;
382 - | | | | |     (i + 1) & 136 &&
383 - | | | | |     (empty > 0 && (fen += empty), i !== 0x88.h1 && (fen += '/'), (empty = 0), (i += 8));
384 - | | | | |   }
385 - | | | | |   let castling = '';
386 - | | | | |   (this._castling.w & BITS.KSIDE_CASTLE && (castling += 'K'),
387 - | | | | |   this._castling.w & BITS.QSIDE_CASTLE && (castling += 'Q'),
388 - | | | | |   this._castling.b & BITS.KSIDE_CASTLE && (castling += 'k'),
389 - | | | | |   this._castling.b & BITS.QSIDE_CASTLE && (castling += 'q'),
390 - | | | | |   (castling = castling || '-'));
391 - | | | | |   let epSquare = '-';
392 - | | | | |   if (-1 !== this._epSquare)
393 - | | | | |     if (forceEnpassantSquare) epSquare = algebraic(this._epSquare);
394 - | | | | |   else {
395 - | | | | |     const bigPawnSquare = this._epSquare + ('w' === this._turn ? 16 : -16),
396 - | | | | |     squares = [bigPawnSquare + 1, bigPawnSquare - 1];
397 - | | | | |     for (const square of squares) {
398 - | | | | |       if (136 & square) continue;
399 - | | | | |       const color = this._turn;
400 - | | | | |       if (this._board[square]?.color === color && 'p' === this._board[square]?.type) {
401 - | | | | |         this._makeMove({
402 - | | | | |           color: color,
403 - | | | | |           from: square,
404 - | | | | |           to: this._epSquare,
405 - | | | | |           piece: 'p',
406 - | | | | |           captured: 'p',
407 - | | | | |           flags: BITS.EP_CAPTURE,
408 - | | | | |         });
409 - | | | | |         const isLegal = !this._isKingAttacked(color);
410 - | | | | |         if ((this._undoMove(), isLegal)) {

```

```

411 -         .epSquare = algebraic(this._epSquare);
412 -         break;
413 -     }
414 - }
415 - }
416 - }
417 - return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
418 - }
419 - _pieceKey(i) {
420 -     if (!this._board[i]) return 0n;
421 -     const { color: color, type: type } = this._board[i],
422 -           typeIndex = { p: 0, n: 1, b: 2, r: 3, q: 4, k: 5 }[type];
423 -     return PIECE_KEYS[{ w: 0, b: 1 }[color]][typeIndex][i];
424 - }
425 - _epKey() {
426 -     return -1 === this._epSquare ? 0n : EP_KEYS[7 & this._epSquare];
427 - }
428 - _castlingKey() {
429 -     const index = (this._castling.w >> 5) | (this._castling.b >> 3);
430 -     return CASTLING_KEYS[index];
431 - }
432 - _computeHash() {
433 -     let hash = 0n;
434 -     for (let i = 0x88.a8; i < 0x88.h1; i++)
435 -         136 & i ? (i += 7) : this._board[i] && (hash ^= this._pieceKey(i));
436 -     return (
437 -         (hash ^= this._epKey()),
438 -         (hash ^= this._castlingKey()),
439 -         'b' === this._turn && (hash ^= SIDE_KEY),
440 -         hash
441 -     );
442 - }
443 - reset() {
444 -     this.load(DEFAULT_POSITION);
445 - }
446 - get(square) {
447 -     return this._board[0x88[square]];
448 - }
449 - _set(sq, piece) {
450 -     ((this._hash ^= this._pieceKey(sq)),
451 -     (this._board[sq] = piece),
452 -     (this._hash ^= this._pieceKey(sq)));
453 - }
454 - _put({ type: type, color: color }, square) {
455 -     if (-1 === 'pnbrqkPNBRQK'.indexOf(type.toLowerCase())) return !1;
456 -     if (!(square in 0x88)) return !1;
457 -     const sq = 0x88[square];
458 -     if ('k' === type && -1 !== this._kings[color] && this._kings[color] !== sq) return !1;
459 -     const currentPieceOnSquare = this._board[sq];
460 -     return (
461 -         currentPieceOnSquare &&
462 -         ('k' === currentPieceOnSquare.type &&
463 -         (this._kings[currentPieceOnSquare.color] !== -1),
464 -         this._set(sq, { type: type, color: color })),
465 -         'k' === type && (this._kings[color] = sq),
466 -         !0
467 -     );
468 - }
469 - _clear(sq) {
470 -     ((this._hash ^= this._pieceKey(sq)), delete this._board[sq]);
471 - }
472 - _attacked(color, square, verbose) {
473 -     const attackers = [];
474 -     for (let i = 0x88.a8; i < 0x88.h1; i++) {
475 -         if (136 & i) {
476 -             i += 7;
477 -             continue;
478 -         }
479 -         if (void 0 === this._board[i] || this._board[i].color !== color) continue;
480 -         const piece = this._board[i],
481 -               difference = i - square;

```

```

482 - | | | | if (0 === difference) continue;
483 - | | | | const index = difference + 119;
484 - | | | | if (ATTACKS[index] & PIECE_MASKS[piece.type]) {
485 - | | | | | if ('p' === piece.type) {
486 - | | | | | | if ((difference > 0 && 'w' === piece.color) || (difference ≤ 0 && 'b' === piece.color)) {
487 - | | | | | | | if (!verbose) return !0;
488 - | | | | | | | attackers.push(algebraic(i));
489 - | | | | | | }
490 - | | | | | continue;
491 - | | | | | }
492 - | | | | | if ('n' === piece.type || 'k' === piece.type) {
493 - | | | | | | if (verbose) {
494 - | | | | | | | attackers.push(algebraic(i));
495 - | | | | | | | continue;
496 - | | | | | | }
497 - | | | | | | return !0;
498 - | | | | | }
499 - | | | | | const offset = RAYS[index];
500 - | | | | | let j = i + offset,
501 - | | | | | blocked = !1;
502 - | | | | | for (; j ≠ square; ) {
503 - | | | | | | if (null ≠ this._board[j]) {
504 - | | | | | | | blocked = !0;
505 - | | | | | | | break;
506 - | | | | | | }
507 - | | | | | | j += offset;
508 - | | | | | }
509 - | | | | | if (!blocked) {
510 - | | | | | | if (verbose) {
511 - | | | | | | | attackers.push(algebraic(i));
512 - | | | | | | | continue;
513 - | | | | | | }
514 - | | | | | | return !0;
515 - | | | | | }
516 - | | | | }
517 - | | | }
518 - | | return !!verbose && attackers;
519 - | }
520 - attackers(square, attackedBy) {
521 - | return attackedBy
522 - | | ? this._attacked(attackedBy, 0x88[square], !0)
523 - | | : this._attacked(this._turn, 0x88[square], !0);
524 - }
525 - _isKingAttacked(color) {
526 - | const square = this._kings[color];
527 - | return -1 ≠ square && this._attacked(swapColor(color), square);
528 - }
529 - hash() {
530 - | return this._hash.toString(16);
531 - }
532 - isAttacked(square, attackedBy) {
533 - | return this._attacked(attackedBy, 0x88[square]);
534 - }
535 - isCheck() {
536 - | return this._isKingAttacked(this._turn);
537 - }
538 - inCheck() {
539 - | return this.isCheck();
540 - }
541 - isCheckmate() {
542 - | return this.isCheck() && 0 === this._moves().length;
543 - }
544 - isStalemate() {
545 - | return !this.isCheck() && 0 === this._moves().length;
546 - }
547 - isInsufficientMaterial() {
548 - | const pieces = {b: 0, n: 0, r: 0, q: 0, k: 0, p: 0},
549 - | | bishops = [];
550 - | let numPieces = 0,
551 - | | squareColor = 0;
552 - | for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {

```

```

553 - | | | | if (((squareColor = (squareColor + 1) % 2), 136 & i)) {
554 - | | | | | i += 7;
555 - | | | | | continue;
556 - | | | | }
557 - | | | | const piece = this._board[i];
558 - | | | | piece &&
559 - | | | | | ((pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1),
560 - | | | | | 'b' === piece.type && bishops.push(squareColor),
561 - | | | | | numPieces++);
562 - | | | | }
563 - | | | | if (2 === numPieces) return !0;
564 - | | | | if (3 === numPieces && (1 === pieces.b || 1 === pieces.n)) return !0;
565 - | | | | if (numPieces === pieces.b + 2) {
566 - | | | | | let sum = 0;
567 - | | | | | const len = bishops.length;
568 - | | | | | for (let i = 0; i < len; i++) sum += bishops[i];
569 - | | | | | if (0 === sum || sum === len) return !0;
570 - | | | | }
571 - | | | | return !1;
572 - | | | | }
573 - | | | | isThreefoldRepetition() {
574 - | | | | | return this._getPositionCount(this._hash) ≥ 3;
575 - | | | | }
576 - | | | | isDrawByFiftyMoves() {
577 - | | | | | return this._halfMoves ≥ 100;
578 - | | | | }
579 - | | | | isDraw() {
580 - | | | | | return (
581 - | | | | | this.isDrawByFiftyMoves() ||
582 - | | | | | this.isStalemate() ||
583 - | | | | | this.isInsufficientMaterial() ||
584 - | | | | | this.isThreefoldRepetition()
585 - | | | | | );
586 - | | | | }
587 - | | | | isGameOver() {
588 - | | | | | return this.isCheckmate() || this.isDraw();
589 - | | | | }
590 - | | | | moves({ verbose: verbose = !1, square: square, piece: piece } = {}) {
591 - | | | | | const moves = this._moves({ square: square, piece: piece });
592 - | | | | | return verbose
593 - | | | | | ? moves.map((move) => new Move(this, move))
594 - | | | | | : moves.map((move) => this._moveToSan(move, moves));
595 - | | | | }
596 - | | | | _moves({ legal: legal = !0, piece: piece, square: square } = {}) {
597 - | | | | | const forSquare = square ? square.toLowerCase() : void 0,
598 - | | | | | forPiece = piece ? piece.toLowerCase(),
599 - | | | | | moves = [],
600 - | | | | | us = this._turn,
601 - | | | | | them = swapColor(us);
602 - | | | | | let firstSquare = 0x88.a8,
603 - | | | | | lastSquare = 0x88.h1,
604 - | | | | | singleSquare = !1;
605 - | | | | | if (forSquare) {
606 - | | | | | | if (!forSquare in 0x88) return [];
607 - | | | | | | ((firstSquare = lastSquare = 0x88[forSquare]), (singleSquare = !0));
608 - | | | | | }
609 - | | | | | for (let from = firstSquare; from ≤ lastSquare; from++) {
610 - | | | | | | if (136 & from) {
611 - | | | | | | | from += 7;
612 - | | | | | | | continue;
613 - | | | | | | }
614 - | | | | | | if (!this._board[from] || this._board[from].color === them) continue;
615 - | | | | | | const { type: type } = this._board[from];
616 - | | | | | | let to;
617 - | | | | | | if ('p' === type) {
618 - | | | | | | | if (forPiece && forPiece !== type) continue;
619 - | | | | | | | ((to = from + PAWN_OFFSETS[us][0]),
620 - | | | | | | | this._board[to] ||
621 - | | | | | | | (addMove(moves, us, from, to, 'p'),
622 - | | | | | | | (to = from + PAWN_OFFSETS[us][1]),
623 - | | | | | | | SECOND_RANK[us] !== rank(from) ||

```

```

624 - |         this._board[to] ||
625 - |         addMove(moves, us, from, to, 'p', void 0, BITS.BIG_PAWN));
626 - |         for (let j = 2; j < 4; j++)
627 - |             ((to = from + PAWN_OFFSETS[us][j]),
628 - |              136 & to ||
629 - |              (this._board[to]?.color === them
630 - |               ? addMove(moves, us, from, to, 'p', this._board[to].type, BITS.CAPTURE)
631 - |               : to === this._epSquare &
632 - |                 addMove(moves, us, from, to, 'p', 'p', BITS.EP_CAPTURE)));
633 - |     } else {
634 - |         if (forPiece & forPiece !== type) continue;
635 - |         for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++) {
636 - |             const offset = PIECE_OFFSETS[type][j];
637 - |             for (to = from; (to += offset), !(136 & to);) {
638 - |                 if (this._board[to]) {
639 - |                     if (this._board[to].color === us) break;
640 - |                     addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
641 - |                     break;
642 - |                 }
643 - |                 if ((addMove(moves, us, from, to, type), 'n' === type || 'k' === type)) break;
644 - |             }
645 - |         }
646 - |     }
647 - | }
648 - | if (
649 - |     !(
650 - |         (void 0 !== forPiece & 'k' !== forPiece) ||
651 - |         (singleSquare & lastSquare !== this._kings[us])
652 - |     )
653 - | ) {
654 - |     if (this._castling[us] & BITS.KSIDE_CASTLE) {
655 - |         const castlingFrom = this._kings[us],
656 - |             castlingTo = castlingFrom + 2;
657 - |         this._board[castlingFrom + 1] ||
658 - |         this._board[castlingTo] ||
659 - |         this._attacked(them, this._kings[us]) ||
660 - |         this._attacked(them, castlingFrom + 1) ||
661 - |         this._attacked(them, castlingTo) ||
662 - |         addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.KSIDE_CASTLE);
663 - |     }
664 - |     if (this._castling[us] & BITS.QSIDE_CASTLE) {
665 - |         const castlingFrom = this._kings[us],
666 - |             castlingTo = castlingFrom - 2;
667 - |         this._board[castlingFrom - 1] ||
668 - |         this._board[castlingFrom - 2] ||
669 - |         this._board[castlingFrom - 3] ||
670 - |         this._attacked(them, this._kings[us]) ||
671 - |         this._attacked(them, castlingFrom - 1) ||
672 - |         this._attacked(them, castlingTo) ||
673 - |         addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.QSIDE_CASTLE);
674 - |     }
675 - | }
676 - | if (!legal || -1 === this._kings[us]) return moves;
677 - | const legalMoves = [];
678 - | for (let i = 0, len = moves.length; i < len; i++)
679 - |     (this._makeMove(moves[i]),
680 - |      this._isKingAttacked(us) || legalMoves.push(moves[i]),
681 - |      this._undoMove());
682 - | return legalMoves;
683 - | }
684 - | move(move) {
685 - |     let moveObj = null;
686 - |     const moves = this._moves();
687 - |     for (let i = 0, len = moves.length; i < len; i++)
688 - |         if (
689 - |             move.from === algebraic(moves[i].from) &
690 - |             move.to === algebraic(moves[i].to) &
691 - |             (!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
692 - |         ) {
693 - |             moveObj = moves[i];
694 - |             break;

```



```

695 - | | | }
696 - | | | if (!moveObj) throw new Error(`Invalid move: ${JSON.stringify(move)}`);
697 - | | | const prettyMove = new Move(this, moveObj);
698 - | | | return (this._makeMove(moveObj), this._incPositionCount(), prettyMove);
699 - | | | }
700 - | | | _push(move) {
701 - | | |   this._history.push({
702 - | | |     move: move,
703 - | | |     kings: { b: this._kings.b, w: this._kings.w },
704 - | | |     turn: this._turn,
705 - | | |     castling: { b: this._castling.b, w: this._castling.w },
706 - | | |     epSquare: this._epSquare,
707 - | | |     halfMoves: this._halfMoves,
708 - | | |     moveNumber: this._moveNumber,
709 - | | |   });
710 - | | | }
711 - | | | _movePiece(from, to) {
712 - | | |   ((this._hash ^= this._pieceKey(from)),
713 - | | |   (this._board[to] = this._board[from]),
714 - | | |   delete this._board[from],
715 - | | |   (this._hash ^= this._pieceKey(to)));
716 - | | | }
717 - | | | _makeMove(move) {
718 - | | |   const us = this._turn,
719 - | | |   them = swapColor(us);
720 - | | |   if ((this._push(move), move.flags & BITS.NULL_MOVE))
721 - | | |     return (
722 - | | |       'b' === us && this._moveNumber++,
723 - | | |       this._halfMoves++,
724 - | | |       (this._turn = them),
725 - | | |       void (this._epSquare = -1)
726 - | | |     );
727 - | | |   if (
728 - | | |     ((this._hash ^= this._epKey()),
729 - | | |     (this._hash ^= this._castlingKey()),
730 - | | |     move.captured && (this._hash ^= this._pieceKey(move.to)),
731 - | | |     this._movePiece(move.from, move.to),
732 - | | |     move.flags & BITS.EP_CAPTURE &&
733 - | | |     ('b' === this._turn ? this._clear(move.to - 16) : this._clear(move.to + 16)),
734 - | | |     move.promotion &&
735 - | | |     (this._clear(move.to), this._set(move.to, { type: move.promotion, color: us })),
736 - | | |     'k' === this._board[move.to].type)
737 - | | |   ) {
738 - | | |     if (((this._kings[us] = move.to), move.flags & BITS.KSIDE_CASTLE)) {
739 - | | |       const castlingTo = move.to - 1,
740 - | | |       castlingFrom = move.to + 1;
741 - | | |       this._movePiece(castlingFrom, castlingTo);
742 - | | |     } else if (move.flags & BITS.QSIDE_CASTLE) {
743 - | | |       const castlingTo = move.to + 1,
744 - | | |       castlingFrom = move.to - 2;
745 - | | |       this._movePiece(castlingFrom, castlingTo);
746 - | | |     }
747 - | | |     this._castling[us] = 0;
748 - | | |   }
749 - | | |   if (this._castling[us])
750 - | | |     for (let i = 0, len = ROOKS[us].length; i < len; i++)
751 - | | |       if (move.from === ROOKS[us][i].square && this._castling[us] & ROOKS[us][i].flag) {
752 - | | |         this._castling[us] ^= ROOKS[us][i].flag;
753 - | | |         break;
754 - | | |       }
755 - | | |   if (this._castling[them])
756 - | | |     for (let i = 0, len = ROOKS[them].length; i < len; i++)
757 - | | |       if (move.to === ROOKS[them][i].square && this._castling[them] & ROOKS[them][i].flag) {
758 - | | |         this._castling[them] ^= ROOKS[them][i].flag;
759 - | | |         break;
760 - | | |       }
761 - | | |   if (((this._hash ^= this._castlingKey()), move.flags & BITS.BIG_PAWN)) {
762 - | | |     let epSquare;
763 - | | |     ((epSquare = 'b' === us ? move.to - 16 : move.to + 16),
764 - | | |     ((move.to - 1) & 136 ||
765 - | | |     'p' === this._board[move.to - 1].type ||

```



```

766 - | | | | | this._board[move.to - 1] ? .color !== them) &&
767 - | | | | | ((move.to + 1) & 136 ||
768 - | | | | | 'p' !== this._board[move.to + 1] ? .type ||
769 - | | | | | this._board[move.to + 1] ? .color !== them)
770 - | | | | | ? (this._epSquare = -1)
771 - | | | | | : ((this._epSquare = epSquare), (this._hash ^= this._epKey()));
772 - | | | | | } else this._epSquare = -1;
773 - | | | | | ('p' === move.piece || move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)
774 - | | | | | ? (this._halfMoves = 0)
775 - | | | | | : this._halfMoves++,
776 - | | | | | 'b' === us && this._moveNumber++,
777 - | | | | | (this._turn = them),
778 - | | | | | (this._hash ^= SIDE_KEY));
779 - | | | | | }
780 - | | | | | _undo() {
781 - | | | | |   const hash = this._hash,
782 - | | | | |   move = this._undoMove();
783 - | | | | |   if (move) {
784 - | | | | |     const prettyMove = new Move(this, move);
785 - | | | | |     return (this._decPositionCount(hash), prettyMove);
786 - | | | | |   }
787 - | | | | |   return null;
788 - | | | | | }
789 - | | | | | _undoMove() {
790 - | | | | |   const old = this._history.pop();
791 - | | | | |   if (void 0 === old) return null;
792 - | | | | |   ((this._hash ^= this._epKey()), (this._hash ^= this._castlingKey()));
793 - | | | | |   const move = old.move;
794 - | | | | |   ((this._kings = old.kings),
795 - | | | | |   (this._turn = old.turn),
796 - | | | | |   (this._castling = old.castling),
797 - | | | | |   (this._epSquare = old.epSquare),
798 - | | | | |   (this._halfMoves = old.halfMoves),
799 - | | | | |   (this._moveNumber = old.moveNumber),
800 - | | | | |   (this._hash ^= this._epKey()),
801 - | | | | |   (this._hash ^= this._castlingKey()),
802 - | | | | |   (this._hash ^= SIDE_KEY));
803 - | | | | |   const us = this._turn,
804 - | | | | |   them = swapColor(us);
805 - | | | | |   if (move.flags & BITS.NULL_MOVE) return move;
806 - | | | | |   if (
807 - | | | | |     (this._movePiece(move.to, move.from),
808 - | | | | |     move.piece && (this._clear(move.from), this._set(move.from, { type: move.piece, color: us })),
809 - | | | | |     move.captured)
810 - | | | | |   )
811 - | | | | |     if (move.flags & BITS.EP_CAPTURE) {
812 - | | | | |       let index;
813 - | | | | |       ((index = 'b' === us ? move.to - 16 : move.to + 16),
814 - | | | | |       this._set(index, { type: 'p', color: them }));
815 - | | | | |     } else this._set(move.to, { type: move.captured, color: them });
816 - | | | | |     if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
817 - | | | | |       let castlingTo, castlingFrom;
818 - | | | | |       (move.flags & BITS.KSIDE_CASTLE
819 - | | | | |       ? ((castlingTo = move.to + 1), (castlingFrom = move.to - 1))
820 - | | | | |       : ((castlingTo = move.to - 2), (castlingFrom = move.to + 1)),
821 - | | | | |       this._movePiece(castlingFrom, castlingTo));
822 - | | | | |     }
823 - | | | | |     return move;
824 - | | | | |   }
825 - | | | | |   _moveToSan(move, moves) {
826 - | | | | |     let output = '';
827 - | | | | |     if (move.flags & BITS.KSIDE_CASTLE) output = 'O-O';
828 - | | | | |     else if (move.flags & BITS.QSIDE_CASTLE) output = 'O-O-O';
829 - | | | | |     else {
830 - | | | | |       if ('p' !== move.piece) {
831 - | | | | |         const disambiguator = (function getDisambiguator(move, moves) {
832 - | | | | |           const from = move.from,
833 - | | | | |           to = move.to,
834 - | | | | |           piece = move.piece;
835 - | | | | |           let ambiguities = 0,
836 - | | | | |           sameRank = 0,

```

```

837 - | | | | | sameFile = 0;
838 - | | | | | for (let i = 0, len = moves.length; i < len; i++) {
839 - | | | | |   const ambigFrom = moves[i].from,
840 - | | | | |   ambigTo = moves[i].to;
841 - | | | | |   piece === moves[i].piece &&
842 - | | | | |   from !== ambigFrom &&
843 - | | | | |   to === ambigTo &&
844 - | | | | |   (ambiguities++,
845 - | | | | |   rank(from) === rank(ambigFrom) && sameRank++,
846 - | | | | |   file(from) === file(ambigFrom) && sameFile++);
847 - | | | | | }
848 - | | | | | return ambiguities > 0
849 - | | | | |   ? sameRank > 0 && sameFile > 0
850 - | | | | |   ? algebraic(from)
851 - | | | | |   : sameFile > 0
852 - | | | | |   ? algebraic(from).charAt(1)
853 - | | | | |   : algebraic(from).charAt(0)
854 - | | | | |   : '';
855 - | | | | | })(move, moves);
856 - | | | | | output += move.piece.toUpperCase() + disambiguator;
857 - | | | | | }
858 - | | | | | (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE) &&
859 - | | | | | ('p' === move.piece && (output += algebraic(move.from)[0]), (output += 'x')),
860 - | | | | | (output += algebraic(move.to)),
861 - | | | | | move.promotion && (output += '=' + move.promotion.toUpperCase()));
862 - | | | | | }
863 - | | | | | return (
864 - | | | | |   this._makeMove(move),
865 - | | | | |   this.isCheck() && (this.isCheckmate() ? (output += '#') : (output += '+')),
866 - | | | | |   this._undoMove(),
867 - | | | | |   output
868 - | | | | | );
869 - | | | | | }
870 - | | | | | turn() {
871 - | | | | |   return this._turn;
872 - | | | | | }
873 - | | | | | board() {
874 - | | | | |   const output = [];
875 - | | | | |   let row = [];
876 - | | | | |   for (let i = 0x88.a8; i ≤ 0x88.h1; i++)
877 - | | | | |     (null === this._board[i]
878 - | | | | |     ? row.push(null)
879 - | | | | |     : row.push({
880 - | | | | |       square: algebraic(i),
881 - | | | | |       type: this._board[i].type,
882 - | | | | |       color: this._board[i].color,
883 - | | | | |     })),
884 - | | | | |   (i + 1) & 136 && (output.push(row), (row = []), (i += 8));
885 - | | | | |   return output;
886 - | | | | | }
887 - | | | | | squareColor(square) {
888 - | | | | |   if (square in 0x88) {
889 - | | | | |     const sq = 0x88[square];
890 - | | | | |     return (rank(sq) + file(sq)) % 2 === 0 ? 'light' : 'dark';
891 - | | | | |   }
892 - | | | | |   return null;
893 - | | | | | }
894 - | | | | | history({ verbose: verbose = !1 } = {}) {
895 - | | | | |   const reversedHistory = [],
896 - | | | | |   moveHistory = [];
897 - | | | | |   for (; this._history.length > 0; ) reversedHistory.push(this._undoMove());
898 - | | | | |   for (;;) {
899 - | | | | |     const move = reversedHistory.pop();
900 - | | | | |     if (!move) break;
901 - | | | | |     (verbose
902 - | | | | |     ? moveHistory.push(new Move(this, move))
903 - | | | | |     : moveHistory.push(this._moveToSan(move, this._moves()))),
904 - | | | | |     this._makeMove(move);
905 - | | | | |   }
906 - | | | | |   return moveHistory;
907 - | | | | | }

```

```
908 - | | | _getPositionCount(hash) · {  
909 - | | |   return this._positionCount.get(hash) · ?? · 0;  
910 - | | | }  
911 - | | | _incPositionCount() · {  
912 - | | |   this._positionCount.set(this._hash, (this._positionCount.get(this._hash) · ?? · 0) + 1);  
913 - | | | }  
914 - | | | _decPositionCount(hash) · {  
915 - | | |   const currentCount = this._positionCount.get(hash) · ?? · 0;  
916 - | | |   1 === currentCount  
917 - | | |     ? this._positionCount.delete(hash)  
918 - | | |     : this._positionCount.set(hash, currentCount - 1);  
919 - | | | }  
920 - | | | moveNumber() · {  
921 - | | |   return this._moveNumber;  
922 - | | | }  
923 - | | | }  
924 - | | const AI_PRESETS = {  
    1 + | | | const sharedCore = globalThis.TSChessShared;  
    2 + | | | if (!sharedCore) throw new Error('Missing shared chess runtime: TSChessShared');  
    3 + | | | const Chess = sharedCore.Chess,  
    4 + | | | SQUARES = sharedCore.SQUARES,  
    5 + | | | HARD_MAX_DEPTH = Number.POSITIVE_INFINITY,  
    6 + | | | AI_PRESETS = {  
925 7 | | |   easy: { preset: 'easy', maxDepth: 2, timeLimitMs: 750, randomness: 30, quiescence: !0 },  
926 8 | | |   normal: { preset: 'normal', maxDepth: 5, timeLimitMs: 5e3, randomness: 30, quiescence: !0 },  
927 - | | |   hard: { preset: 'hard', maxDepth: 5, timeLimitMs: 5e3, randomness: 30, quiescence: !0 },  
    9 + | | |   hard: {  
    10 + | | |     preset: 'hard',  
    11 + | | |     maxDepth: HARD_MAX_DEPTH,  
    12 + | | |     timeLimitMs: 5e3,  
    13 + | | |     randomness: 30,  
    14 + | | |     quiescence: !0,  
    15 + | | |   },  
928 16 | | | },  
929 17 | | | DEFAULT_AI_SETTINGS = { ...AI_PRESETS.normal };  
930 18 | | function normalizeAiSettings(settings) · {  
931 - | | |   const timeLimitMs = clamp(Math.round(settings.timeLimitMs), 500, 1e4),  
932 - | | |   softTimeLimitMs =  
933 - | | |     void 0 === settings.softTimeLimitMs  
934 - | | |       ? void 0  
935 - | | |       : Math.min(timeLimitMs, clamp(Math.round(settings.softTimeLimitMs), 500, 1e4));  
    19 + | | |   const timeLimitMs = Math.max(100, Math.round(settings.timeLimitMs));  
936 20 | | |   return {  
937 21 | | |     preset: settings.preset,  
938 - | | |     maxDepth: ((depth = settings.maxDepth), clamp(Math.round(depth), 1, 5)),  
    22 + | | |     maxDepth:  
    23 + | | |       ((depth = settings.maxDepth),  
    24 + | | |         (preset = settings.preset),  
    25 + | | |         'hard' === preset  
    26 + | | |           ? HARD_MAX_DEPTH  
    27 + | | |           : (function clamp(value, min, max) · {  
    28 + | | |             return Math.min(max, Math.max(min, value));  
    29 + | | |           })(Math.round(depth), 1, 5)),  
939 30 | | |     timeLimitMs: timeLimitMs,  
940 - | | |     ... (void 0 === softTimeLimitMs ? {} : { softTimeLimitMs: softTimeLimitMs }),  
941 31 | | |     randomness: 30,  
942 32 | | |     quiescence: !0,  
943 33 | | |   };  
944 - | | | var depth;  
945 - | | | }  
946 - | | | function clamp(value, min, max) · {  
947 - | | |   return Math.min(max, Math.max(min, value));  
    34 + | | | var depth, preset;  
948 35 | | | }  
949 - | | const jsContent =  
950 - | | | "[main.js :: inline worker jsContent #1 extracted below: 35031 B / 1472 lines]",  
    36 + | | | const jsContent = (() => {  
    37 + | | |   const sharedElement = document.getElementById('ts-chess-shared-core-source');  
    38 + | | |   const workerElement = document.getElementById('ai-worker-source-1');  
    39 + | | |   if (!sharedElement) {  
    40 + | | |     throw new Error('Missing shared chess runtime source: ts-chess-shared-core-source');
```

```

41 + | | }
42 + | | if (!workerElement) {
43 + | | | throw new Error('Missing inline worker source: ai-worker-source-1');
44 + | | }
45 + | | return (sharedElement.textContent || '') + '\n' + (workerElement.textContent || '');
46 + | | }()),
951 47 | | blob =
952 48 | | 'undefined' !== typeof self &&
953 49 | | self.Blob &&
954 50 | | new Blob([(self.URL || self.webkitURL).revokeObjectURL(self.location.href);', jsContent], {
955 51 | | | type: 'text/javascript;charset=utf-8',
956 52 | | });
957 53 | function WorkerWrapper(options) {
958 54 | | let objURL;
959 55 | | try {
960 56 | | | if (((objURL = blob && (self.URL || self.webkitURL).createObjectURL(blob)), !objURL)) throw '';
961 57 | | | const worker = new Worker(objURL, { name: options?.name });
962 58 | | | return (
963 59 | | | | worker.addEventListener('error', () => {
964 60 | | | | | (self.URL || self.webkitURL).revokeObjectURL(objURL);
965 61 | | | | })),
966 62 | | | worker
967 63 | | | );
968 64 | | } catch (e) {
969 65 | | | return new Worker('data:text/javascript;charset=utf-8,' + encodeURIComponent(jsContent), {
970 66 | | | | name: options?.name,
971 67 | | | });
972 68 | | }
973 69 | }
974 70 | const BOARD_FILES = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'],
975 71 | | BLACK_FILES = [... BOARD_FILES].reverse(),
976 72 | | WHITE_RANKS = [8, 7, 6, 5, 4, 3, 2, 1],
977 73 | | BLACK_RANKS = [... WHITE_RANKS].reverse(),
978 74 | | PROMOTION_PIECES = ['q', 'r', 'b', 'n'],
979 - | | SQUARE_SET = new Set([
980 - | | | 'a8',
981 - | | | 'b8',
982 - | | | 'c8',
983 - | | | 'd8',
984 - | | | 'e8',
985 - | | | 'f8',
986 - | | | 'g8',
987 - | | | 'h8',
988 - | | | 'a7',
989 - | | | 'b7',
990 - | | | 'c7',
991 - | | | 'd7',
992 - | | | 'e7',
993 - | | | 'f7',
994 - | | | 'g7',
995 - | | | 'h7',
996 - | | | 'a6',
997 - | | | 'b6',
998 - | | | 'c6',
999 - | | | 'd6',
1000 - | | | 'e6',
1001 - | | | 'f6',
1002 - | | | 'g6',
1003 - | | | 'h6',
1004 - | | | 'a5',
1005 - | | | 'b5',
1006 - | | | 'c5',
1007 - | | | 'd5',
1008 - | | | 'e5',
1009 - | | | 'f5',
1010 - | | | 'g5',
1011 - | | | 'h5',
1012 - | | | 'a4',
1013 - | | | 'b4',
1014 - | | | 'c4',
1015 - | | | 'd4',

```

```

1016 - | | | 'e4',
1017 - | | | 'f4',
1018 - | | | 'g4',
1019 - | | | 'h4',
1020 - | | | 'a3',
1021 - | | | 'b3',
1022 - | | | 'c3',
1023 - | | | 'd3',
1024 - | | | 'e3',
1025 - | | | 'f3',
1026 - | | | 'g3',
1027 - | | | 'h3',
1028 - | | | 'a2',
1029 - | | | 'b2',
1030 - | | | 'c2',
1031 - | | | 'd2',
1032 - | | | 'e2',
1033 - | | | 'f2',
1034 - | | | 'g2',
1035 - | | | 'h2',
1036 - | | | 'a1',
1037 - | | | 'b1',
1038 - | | | 'c1',
1039 - | | | 'd1',
1040 - | | | 'e1',
1041 - | | | 'f1',
1042 - | | | 'g1',
1043 - | | | 'h1',
1044 - | | | ]),

75 + | | | SQUARE_SET = new Set(SQUARES),
76 + | | | AI_WHITE_OPENING_PRESET = [
77 + | | | { from: 'e2', to: 'e4', weight: 28 },
78 + | | | { from: 'd2', to: 'd4', weight: 25 },
79 + | | | { from: 'g1', to: 'f3', weight: 24 },
80 + | | | { from: 'c2', to: 'c4', weight: 20 },
81 + | | | { from: 'g2', to: 'g3', weight: 9 },
82 + | | | { from: 'b2', to: 'b3', weight: 6 },
83 + | | | ],

1045 84 | | | CLOCK_PRESETS = {
1046 85 | | | '1/1': { baseMs: 6e4, incrementMs: 1e3 },
1047 86 | | | '3/2': { baseMs: 18e4, incrementMs: 2e3 },
1048 87 | | | '5/3': { baseMs: 3e5, incrementMs: 3e3 },
1049 88 | | | '10/5': { baseMs: 6e5, incrementMs: 5e3 },
1050 89 | | | '15/10': { baseMs: 9e5, incrementMs: 1e4 },
1051 90 | | | },
1052 91 | | | PIECE_GLYPHS = {
1053 92 | | | w: { p: '♔', n: '♖', b: '♜', r: '♞', q: '♝', k: '♚' },
1054 93 | | | b: { p: '♚', n: '♛', b: '♞', r: '♟', q: '♜', k: '♔' },
1055 94 | | | };
1056 95 | | | function requireElement(selector, root) {
1057 96 | | |   const element = root.querySelector(selector);
1058 97 | | |   if (!element) throw new Error(`Missing element: ${selector}`);
1059 98 | | |   return element;
1060 99 | | | }
1061 100 | | | function createCoordinate(type, label) {
1062 101 | | |   const coordinate = document.createElement('span');
1063 102 | | |   return (
1064 103 | | |     (coordinate.className = `coordinate-coordinate-${type}`),
1065 104 | | |     (coordinate.textContent = label),
1066 105 | | |     coordinate
1067 106 | | |   );
1068 107 | | | }
1069 - | | | function toSquare(file2, rank2) {
1070 - | | |   return `${file2}${rank2}`;
108 + | | | function toSquare(file, rank) {
109 + | | |   return `${file}${rank}`;
1071 110 | | | }
1072 111 | | | function createClockState(preset) {
1073 112 | | |   const baseMs = CLOCK_PRESETS[preset].baseMs;
1074 113 | | |   return { w: baseMs, b: baseMs };
1075 114 | | | }

```

```

1076 115 | function colorName(color)·{
1077 116 | |·return 'w' === color·?·'백'·:·'흑';
1078 117 | }
1079 118 | function presetName(preset)·{
1080 119 | |·switch (preset)·{
1081 120 | |·|·case 'easy':
1082 121 | |·|·|·return '쉬움';
1083 122 | |·|·case 'normal':
1084 123 | |·|·|·return '보통';
1085 124 | |·|·case 'hard':
1086 125 | |·|·|·return '어려움';
1087 126 | |·}
1088 127 | }
1089 128 | function formatSeconds(milliseconds)·{
1090 129 | |·return `${(milliseconds·/·1e3).toFixed(milliseconds·%·1e3·===·0·?·0·:·2)}초`;
1091 130 | }
1092 131 | function formatDebugScore(score)·{
1093 132 | |·return Math.abs(score)·≥·9e5
1094 133 | |·|·?·score·>·0
1095 - | |·|·?·'+mate-ish'
1096 - | |·|·:·'-mate-ish'
134 + | |·|·?·'+mate'
135 + | |·|·:·'-mate'
1097 136 | |·:·score·>·0
1098 137 | |·|·?·`·+${Math.round(score)}·`
1099 138 | |·|·:·String(Math.round(score));
1100 139 | }
1101 140 | function oppositeColor(color)·{
1102 141 | |·return 'w' === color·?·'b'·:·'w';
1103 142 | }
1104 143 | function getSquareLabel(square,·piece)·{
1105 144 | |·return piece·?·`${square}·${colorName(piece.color)}·${pieceName(piece.type)}·`·:·`${square}·빈·칸`;
1106 145 | }
1107 146 | function pieceName(piece)·{
1108 147 | |·switch (piece)·{
1109 148 | |·|·case 'p':
1110 149 | |·|·|·return '폰';
1111 150 | |·|·case 'n':
1112 151 | |·|·|·return '나이트';
1113 152 | |·|·case 'b':
1114 153 | |·|·|·return '비숍';
1115 154 | |·|·case 'r':
1116 155 | |·|·|·return '룩';
1117 156 | |·|·case 'q':
1118 157 | |·|·|·return '퀸';
1119 158 | |·|·case 'k':
1120 159 | |·|·|·return '킹';
1121 160 | |·}
1122 161 | }
1123 162 | !(function createChessApp()·{
1124 163 | |·const root·=·requireElement('#chess-app',·document),
1125 164 | |·boardElement·=·requireElement('#chess-board',·root),
1126 165 | |·colorControlsElement·=·requireElement('#color-controls',·root),
1127 166 | |·aiControlsElement·=·requireElement('#ai-controls',·root),
1128 167 | |·statusElement·=·requireElement('#game-status',·root),
1129 168 | |·turnMetaElement·=·requireElement('#turn-meta',·root),
1130 169 | |·modeMetaElement·=·requireElement('#mode-meta',·root),
1131 170 | |·moveMetaElement·=·requireElement('#move-meta',·root),
1132 171 | |·aiCandidatePanelElement·=·requireElement('#ai-candidate-panel',·root),
1133 172 | |·aiCandidateMetaElement·=·requireElement('#ai-candidate-meta',·root),
1134 173 | |·aiInfoStatusElement·=·requireElement('#ai-info-status',·root),
1135 174 | |·aiInfoPresetElement·=·requireElement('#ai-info-preset',·root),
1136 175 | |·aiInfoHumanElement·=·requireElement('#ai-info-human',·root),
1137 176 | |·aiInfoDepthElement·=·requireElement('#ai-info-depth',·root),
1138 177 | |·aiInfoNodesElement·=·requireElement('#ai-info-nodes',·root),
1139 178 | |·aiInfoScoreElement·=·requireElement('#ai-info-score',·root),
1140 179 | |·aiCandidateListElement·=·requireElement('#ai-candidate-list',·root),
1141 180 | |·moveListElement·=·requireElement('#move-list',·root),
1142 181 | |·clockDisplayElement·=·requireElement('#clock-display',·root),
1143 182 | |·whiteClockElement·=·requireElement('#white-clock',·root),
1144 183 | |·blackClockElement·=·requireElement('#black-clock',·root),

```

```

1145 184 | | | | | whiteClockTimeElement := requireElement('#white-clock-time', root),
1146 185 | | | | | blackClockTimeElement := requireElement('#black-clock-time', root),
1147 186 | | | | | newGameButton := requireElement('#new-game-button', root),
1148 187 | | | | | undoButton := requireElement('#undo-button', root),
1149 188 | | | | | aiDepthInput := requireElement('#ai-depth-input', root),
1150 189 | | | | | aiDepthValue := requireElement('#ai-depth-value', root),
1151 190 | | | | | aiTimeInput := requireElement('#ai-time-input', root),
1152 191 | | | | | aiTimeValue := requireElement('#ai-time-value', root),
1153 192 | | | | | promotionOverlay := requireElement('#promotion-overlay', root),
1154 193 | | | | | promotionChoices := requireElement('#promotion-choices', root),
1155 194 | | | | | modeButtons := Array.from(root.querySelectorAll('[data-mode]')),
1156 195 | | | | | colorButtons := Array.from(root.querySelectorAll('[data-color]')),
1157 196 | | | | | presetButtons := Array.from(root.querySelectorAll('[data-preset]')),
1158 197 | | | | | clockPresetButtons := Array.from(root.querySelectorAll('[data-clock-preset]')),
1159 198 | | | | | game := new Chess();
1160 199 | | | | | let mode := 'local',
1161 200 | | | | | clockPreset := '10/5',
1162 201 | | | | | clockMs := createClockState(clockPreset),
1163 202 | | | | | activeClockColor := null,
1164 203 | | | | | lastClockTick := Date.now(),
1165 204 | | | | | clockTimerId := null,
1166 205 | | | | | timedOutColor := null,
1167 206 | | | | | humanColor := 'w',
1168 207 | | | | | selectedSquare := null,
1169 208 | | | | | legalMoves := [],
1170 209 | | | | | pendingPromotion := null,
1171 210 | | | | | aiThinking := !1,
1172 211 | | | | | aiTimerId := null,
1173 212 | | | | | aiWorker := null,
1174 213 | | | | | aiRequestId := 0,
1175 214 | | | | | activeAiRequestId := null,
1176 215 | | | | | activeAiFen := null,
1177 216 | | | | | aiSettings := { ... DEFAULT_AI_SETTINGS },
1178 217 | | | | | aiProgress := null;
1179 218 | | | | | function selectSquare(square) {
1180 219 | | | | | | | (selectedSquare := square),
1181 220 | | | | | | | (legalMoves := game.moves({ verbose: !0, square: square })),
1182 221 | | | | | | | render();
1183 222 | | | | | }
1184 223 | | | | | function playMove(moveCommand) {
1185 224 | | | | | | | if (!commitMove(moveCommand)) return (clearSelection(), void render());
1186 225 | | | | | | | ((pendingPromotion := null), clearSelection(), render(), scheduleAiIfNeeded());
1187 226 | | | | | }
1188 227 | | | | | function resetGame() {
1189 228 | | | | | | | (cancelAiMove(),
1190 229 | | | | | | | game.reset(),
1191 230 | | | | | | | (pendingPromotion := null),
1192 231 | | | | | | | clearSelection(),
1193 232 | | | | | | | resetClockState(),
1194 233 | | | | | | | restartClockForCurrentTurn(),
1195 234 | | | | | | | render(),
1196 235 | | | | | | | scheduleAiIfNeeded());
1197 236 | | | | | }
1198 237 | | | | | function scheduleAiIfNeeded() {
1199 238 | | | | | | | 'ai' !== mode ||
1200 239 | | | | | | | game.isGameOver() ||
1201 240 | | | | | | | null !== timedOutColor ||
1202 241 | | | | | | | game.turn() === humanColor ||
1203 242 | | | | | | | aiThinking ||
1204 243 | | | | | | | pendingPromotion ||
244 + | | | | | | | (function tryCommitAiWhiteOpeningPreset() {
245 + | | | | | | | | | if ('b' !== humanColor || 'w' !== game.turn() || game.history().length > 0) return !1;
246 + | | | | | | | | | const selectedMove = (function pickWeightedOpeningMove(moves) {
247 + | | | | | | | | | | | const totalWeight = moves.reduce((total, move) => total + move.weight, 0);
248 + | | | | | | | | | | | if (totalWeight ≤ 0) return null;
249 + | | | | | | | | | | | let draw = Math.random() * totalWeight;
250 + | | | | | | | | | | | for (const move of moves)
251 + | | | | | | | | | | | | | if (((draw -= move.weight), draw ≤ 0))
252 + | | | | | | | | | | | | | return { from: move.from, to: move.to, promotion: move.promotion };
253 + | | | | | | | | | | | const fallback = moves.at(-1);
254 + | | | | | | | | | | | return fallback

```

```

255 + | | | | | ? { from: fallback.from, to: fallback.to, promotion: fallback.promotion }
256 + | | | | | : null;
257 + | | | | | })(
258 + | | | | | AI_WHITE_OPENING_PRESET.filter((presetMove) =>
259 + | | | | |   game.moves({ verbose: !0, square: presetMove.from }).some((move) =>
260 + | | | | |     (function isSameMoveCommand(move, command) {
261 + | | | | |       return (
262 + | | | | |         move.from === command.from &&
263 + | | | | |         move.to === command.to &&
264 + | | | | |         (move.promotion ?? void 0) === (command.promotion ?? void 0)
265 + | | | | |       );
266 + | | | | |     })(move, presetMove),
267 + | | | | |   ),
268 + | | | | | ),
269 + | | | | | );
270 + | | | | | return !(
271 + | | | | |   !selectedMove ||
272 + | | | | |   !commitMove(selectedMove) ||
273 + | | | | |   ((aiProgress = null), clearSelection(), render(), 0)
274 + | | | | | );
275 + | | | | | })() ||

1205 276 | | | | | ((aiThinking = !0),
1206 277 | | | | | (aiProgress = null),
1207 278 | | | | | render(),
1208 279 | | | | | (aiTimerId = window.setTimeout(() => {
1209 280 | | | | |   ((aiTimerId = null),
1210 281 | | | | |   (function startAiSearch() {
1211 282 | | | | |     aiWorker?.terminate();
1212 283 | | | | |     const requestId = aiRequestId + 1,
1213 284 | | | | |     fen = game.fen();
1214 285 | | | | |     ((aiRequestId = requestId),
1215 286 | | | | |     (activeAiRequestId = requestId),
1216 287 | | | | |     (activeAiFen = fen),
1217 288 | | | | |     (aiWorker = new WorkerWrapper()),
1218 289 | | | | |     aiWorker.addEventListener('message', (event) => {
1219 290 | | | | |       !(function handleAiMessage(message) {
1220 291 | | | | |         if (
1221 292 | | | | |           message.id === activeAiRequestId &&
1222 293 | | | | |           null !== activeAiFen &&
1223 294 | | | | |           game.fen() === activeAiFen
1224 295 | | | | |         ) {
1225 296 | | | | |           if (((aiProgress = message), 'progress' === message.kind))
1226 297 | | | | |             return (renderStatus(), void renderAiCandidateDebug());
1227 298 | | | | |           ((aiThinking = !1),
1228 299 | | | | |           (activeAiRequestId = null),
1229 300 | | | | |           (activeAiFen = null),
1230 301 | | | | |           aiWorker?.terminate(),
1231 302 | | | | |           (aiWorker = null),
1232 303 | | | | |           message.move && commitMove(message.move),
1233 304 | | | | |           clearSelection(),
1234 305 | | | | |           render());
1235 306 | | | | |         }
1236 307 | | | | |       })(event.data);
1237 308 | | | | |     })),
1238 309 | | | | |     aiWorker.addEventListener('error', () => {
1239 310 | | | | |       activeAiRequestId === requestId &&
1240 311 | | | | |       ((aiThinking = !1),
1241 312 | | | | |       (activeAiRequestId = null),
1242 313 | | | | |       (activeAiFen = null),
1243 314 | | | | |       aiWorker?.terminate(),
1244 315 | | | | |       (aiWorker = null),
1245 316 | | | | |       render());
1246 317 | | | | |     })),
1247 318 | | | | |     aiWorker.postMessage({
1248 319 | | | | |       id: requestId,
1249 320 | | | | |       fen: fen,
1250 321 | | | | |       settings: resolveAiSettingsForSearch(),
1251 322 | | | | |     }));
1252 323 | | | | |   })());
1253 324 | | | | | }, 180));
1254 325 | | | }

```



```

1255 326 |   ·function·cancelAiMove()·{
1256 327 |   |   ·((aiRequestId·+=·1),
1257 328 |   |   ·(activeAiRequestId·=·null),
1258 329 |   |   ·(activeAiFen·=·null),
1259 330 |   |   ·(aiProgress·=·null),
1260 331 |   |   ·null·≠ aiTimerId·&&·(window.clearTimeout(aiTimerId),·(aiTimerId·=·null)),
1261 332 |   |   ·aiWorker·&&·(aiWorker.terminate(),·(aiWorker·=·null)),
1262 333 |   |   ·(aiThinking·=·!1));
1263 334 |   ·}
1264 335 |   ·function·commitMove(moveCommand)·{
1265 336 |   |   ·const·movingColor·=·game.turn();
1266 337 |   |   ·if·(!flushClockElapsed())·return·!1;
1267 338 |   |   ·try·{
1268 339 |   |   |   ·game.move(moveCommand);
1269 340 |   |   }·catch·{
1270 341 |   |   |   ·return·!1;
1271 342 |   |   }
1272 343 |   |   ·return·(
1273 344 |   |   |   ·(clockMs[movingColor]·+=·CLOCK_PRESETS[clockPreset].incrementMs),
1274 345 |   |   |   ·restartClockForCurrentTurn(),
1275 346 |   |   |   ·!0
1276 347 |   |   );
1277 348 |   ·}
1278 349 |   ·function·updateAiSettings(settings)·{
1279 350 |   |   ·((aiSettings·=·normalizeAiSettings(settings)),
1280 351 |   |   ·aiThinking·&&·cancelAiMove(),
1281 352 |   |   ·render(),
1282 353 |   |   ·scheduleAiIfNeeded());
1283 354 |   ·}
1284 355 |   ·function·resolveAiSettingsForSearch()·{
1285 356 |   |   ·const·settings·=·normalizeAiSettings(aiSettings);
1286 357 |   |   ·if·('hard'·≠ settings.preset)·return·settings;
1287 358 |   |   ·const·aiColor·=·game.turn(),
1288 359 |   |   ·remainingMs·=·clockMs[aiColor],
1289 360 |   |   ·targetMs·=·remainingMs·/·30·+·0.8·*·CLOCK_PRESETS[clockPreset].incrementMs,
1290 - |   |   ·remainingSearchBudgetMs·=·Math.max(500,·remainingMs·-·250),
1291 - |   |   ·softTimeLimitMs·=·Math.min(Math.max(targetMs,·800),·6e3,·remainingSearchBudgetMs),
1292 - |   |   ·timeLimitMs·=·Math.min(Math.max(1.8·*·softTimeLimitMs,·1200),·1e4,·remainingSearchBudgetMs);
1293 - |   |   ·return·normalizeAiSettings({
1294 - |   |   |   ·...·settings,
1295 - |   |   |   ·softTimeLimitMs:·softTimeLimitMs,
1296 - |   |   |   ·timeLimitMs:·timeLimitMs,
1297 - |   |   });
361 + |   |   ·timeLimitMs·=·remainingMs·≤·3e3·?·Math.min(targetMs,·0.35·*·remainingMs)·:·targetMs;
362 + |   |   ·return·normalizeAiSettings({·...·settings,·timeLimitMs:·timeLimitMs·});
1298 363 |   ·}
1299 364 |   ·function·resetClockState()·{
1300 365 |   |   ·((clockMs·=·createClockState(clockPreset)),
1301 366 |   |   ·(timedOutColor·=·null),
1302 367 |   |   ·(activeClockColor·=·null),
1303 368 |   |   ·(lastClockTick·=·Date.now()));
1304 369 |   ·}
1305 370 |   ·function·restartClockForCurrentTurn()·{
1306 371 |   |   ·if·((clearClockTimer(),·game.isGameOver()·||·null·≠ timedOutColor))
1307 372 |   |   |   ·return·((activeClockColor·=·null),·void·renderClocks());
1308 373 |   |   ·(activeClockColor·=·game.turn()),
1309 374 |   |   ·(lastClockTick·=·Date.now()),
1310 375 |   |   ·(clockTimerId·=·window.setInterval(tickClock,·100)),
1311 376 |   |   ·renderClocks());
1312 377 |   ·}
1313 378 |   ·function·clearClockTimer()·{
1314 379 |   |   ·null·≠ clockTimerId·&&·(window.clearInterval(clockTimerId),·(clockTimerId·=·null));
1315 380 |   ·}
1316 381 |   ·function·tickClock()·{
1317 382 |   |   ·flushClockElapsed()·&&·renderClocks();
1318 383 |   ·}
1319 384 |   ·function·flushClockElapsed()·{
1320 385 |   |   ·if·(null·≡ activeClockColor·||·null·≠ timedOutColor·||·game.isGameOver())·return·!0;
1321 386 |   |   ·const·now·=·Date.now(),
1322 387 |   |   ·elapsedMs·=·Math.max(0,·now·-·lastClockTick);
1323 388 |   |   ·return·(

```

```

1324 389 | | | | | ·(lastClockTick = now),
1325 390 | | | | | ·0 === elapsedMs ·||
1326 391 | | | | | ·((clockMs[activeClockColor] = Math.max(0, clockMs[activeClockColor] - elapsedMs)),
1327 392 | | | | | ·0 !== clockMs[activeClockColor] ·||
1328 393 | | | | | ·((function handleClockTimeout(color) {
1329 394 | | | | | ·((timedOutColor = color),
1330 395 | | | | | ·(activeClockColor = null),
1331 396 | | | | | ·(clockMs[color] = 0),
1332 397 | | | | | ·clearClockTimer(),
1333 398 | | | | | ·cancelAiMove(),
1334 399 | | | | | ·clearSelection(),
1335 400 | | | | | ·render());
1336 401 | | | | | })(activeClockColor),
1337 402 | | | | | !1))
1338 403 | | | | | );
1339 404 | | | | | }
1340 405 | | | | | function clearSelection() {
1341 406 | | | | | ((selectedSquare = null), (legalMoves = []));
1342 407 | | | | | }
1343 408 | | | | | function render() {
1344 409 | | | | | (!function renderControls() {
1345 410 | | | | | (modeButtons.forEach((button) => {
1346 411 | | | | | const isActive = button.dataset.mode === mode;
1347 412 | | | | | (button.classList.toggle('is-active', isActive),
1348 413 | | | | | button.setAttribute('aria-pressed', String(isActive)));
1349 414 | | | | | })),
1350 415 | | | | | colorButtons.forEach((button) => {
1351 416 | | | | | const isActive = button.dataset.color === humanColor;
1352 417 | | | | | (button.classList.toggle('is-active', isActive),
1353 418 | | | | | button.setAttribute('aria-pressed', String(isActive)));
1354 419 | | | | | })),
1355 420 | | | | | (colorControlsElement.hidden = 'ai' !== mode),
1356 421 | | | | | (aiControlsElement.hidden = 'ai' !== mode),
1357 422 | | | | | (function renderAiSettingsControls() {
1358 423 | | | | | (presetButtons.forEach((button) => {
1359 424 | | | | | const isActive = button.dataset.preset === aiSettings.preset;
1360 425 | | | | | (button.classList.toggle('is-active', isActive),
1361 426 | | | | | button.setAttribute('aria-pressed', String(isActive)));
1362 427 | | | | | })),
1363 - | | | | | (aiDepthInput.value = String(aiSettings.maxDepth)),
1364 - | | | | | (aiDepthValue.textContent = `${aiSettings.maxDepth}`),
1365 + | | | | | (aiDepthInput.value =
1366 + | | | | | ·'hard' === aiSettings.preset ? aiDepthInput.max : String(aiSettings.maxDepth)),
1367 + | | | | | (aiDepthInput.disabled = 'hard' === aiSettings.preset),
1368 + | | | | | (aiDepthValue.textContent =
1369 + | | | | | ·'hard' === aiSettings.preset ? '무제한' : `${aiSettings.maxDepth}`),
1370 + | | | | | (aiTimeInput.value = String(aiSettings.timeLimitMs)),
1371 + | | | | | (aiTimeInput.disabled = 'hard' === aiSettings.preset),
1372 + | | | | | (aiTimeValue.textContent =
1373 + | | | | | ·'hard' === aiSettings.preset ? '자동' : `${formatSeconds(aiSettings.timeLimitMs)}`));
1374 + | | | | | }()),
1375 + | | | | | (function renderClockSettingsControls() {
1376 + | | | | | clockPresetButtons.forEach((button) => {
1377 + | | | | | const isActive = button.dataset.clockPreset === clockPreset;
1378 + | | | | | (button.classList.toggle('is-active', isActive),
1379 + | | | | | button.setAttribute('aria-pressed', String(isActive)));
1380 + | | | | | }));
1381 + | | | | | }()),
1382 + | | | | | (undoButton.disabled = 0 === game.history().length));
1383 + | | | | | }()),
1384 + | | | | | renderClocks(),
1385 + | | | | | (function renderBoard() {
1386 + | | | | | const fragment = document.createDocumentFragment(),
1387 + | | | | | orientation = getBoardOrientation(),
1388 + | | | | | files = 'w' === orientation ? BOARD_FILES : BLACK_FILES,
1389 + | | | | | ranks = 'w' === orientation ? WHITE_RANKS : BLACK_RANKS,
1390 + | | | | | lastMove = (function getLastMove() {
1391 + | | | | | return game.history({ verbose: !0 }).at(-1) ?? null;
1392 + | | | | | })();
1393 + | | | | | boardElement.classList.toggle('is-locked', !canUseBoard());
1394 + | | | | | for (const rank2 of ranks)

```

```

1390 - | | | | | for (const file2 of files) {
1391 - | | | | |   const square = toSquare(file2, rank2),
    457 + | | | | |   for (const rank of ranks)
    458 + | | | | |     for (const file of files) {
    459 + | | | | |       const square = toSquare(file, rank),
1392 460 | | | | |       piece = game.get(square),
1393 461 | | | | |       button = document.createElement('button');
1394 462 | | | | |       if (
1395 463 | | | | |         ((button.type = 'button'),
1396 464 | | | | |         (button.className = getSquareClasses(square, lastMove)),
1397 465 | | | | |         (button.dataset.square = square),
1398 466 | | | | |         (button.disabled = !canUseBoard()),
1399 467 | | | | |         button.setAttribute('role', 'gridcell'),
1400 468 | | | | |         button.setAttribute('aria-label', getSquareLabel(square, piece)),
1401 - | | | | |         file2 === files[0] && button.appendChild(createCoordinate('rank', String(rank2))),
1402 - | | | | |         rank2 === ranks[ranks.length - 1] &&
1403 - | | | | |         button.appendChild(createCoordinate('file', file2)),
    469 + | | | | |         file === files[0] && button.appendChild(createCoordinate('rank', String(rank))),
    470 + | | | | |         rank === ranks[ranks.length - 1] &&
    471 + | | | | |         button.appendChild(createCoordinate('file', file)),
1404 472 | | | | |       piece)
1405 473 | | | | |     ) {
1406 474 | | | | |       const pieceElement = document.createElement('span');
1407 475 | | | | |       ((pieceElement.className = 'piece',
1408 476 | | | | |       (pieceElement.textContent = PIECE_GLYPHS[piece.color][piece.type]),
1409 477 | | | | |       button.appendChild(pieceElement));
1410 478 | | | | |     }
1411 479 | | | | |     fragment.appendChild(button);
1412 480 | | | | |   }
1413 481 | | | | |   boardElement.replaceChildren(fragment);
1414 482 | | | | | }()),
1415 483 | | | | | renderStatus(),
1416 484 | | | | | renderAiCandidateDebug(),
1417 485 | | | | | (function renderMoveList() {
1418 486 | | | | |   const history = game.history({ verbose: !0 });
1419 487 | | | | |   fragment = document.createDocumentFragment();
1420 488 | | | | |   if (0 === history.length) {
1421 489 | | | | |     const empty = document.createElement('p');
1422 490 | | | | |     return (
1423 491 | | | | |       (empty.className = 'move-empty'),
1424 492 | | | | |       (empty.textContent = '아직 둔 수가 없습니다.'),
1425 493 | | | | |       fragment.appendChild(empty),
1426 494 | | | | |       void moveListElement.replaceChildren(fragment)
1427 495 | | | | |     );
1428 496 | | | | |   }
1429 497 | | | | |   for (let index = 0; index < history.length; index += 2) {
1430 498 | | | | |     const row = document.createElement('div'),
1431 499 | | | | |     moveNumber = document.createElement('span'),
1432 500 | | | | |     whiteMove = document.createElement('span'),
1433 501 | | | | |     blackMove = document.createElement('span');
1434 502 | | | | |     ((row.className = 'move-row',
1435 503 | | | | |     (moveNumber.className = 'move-number'),
1436 504 | | | | |     (whiteMove.className = 'move-san'),
1437 505 | | | | |     (blackMove.className = 'move-san'),
1438 506 | | | | |     (moveNumber.textContent = `${Math.floor(index / 2) + 1}.`),
1439 507 | | | | |     (whiteMove.textContent = history[index] ? .san ?? ''),
1440 508 | | | | |     (blackMove.textContent = history[index + 1] ? .san ?? ''),
1441 509 | | | | |     row.append(moveNumber, whiteMove, blackMove),
1442 510 | | | | |     fragment.appendChild(row));
1443 511 | | | | |   }
1444 512 | | | | |   (moveListElement.replaceChildren(fragment),
1445 513 | | | | |   (moveListElement.scrollTop = moveListElement.scrollHeight));
1446 514 | | | | | }()),
1447 515 | | | | | (function renderPromotionDialog() {
1448 516 | | | | |   if (
1449 517 | | | | |     ((promotionOverlay.hidden = null === pendingPromotion),
1450 518 | | | | |     promotionChoices.replaceChildren(),
1451 519 | | | | |     pendingPromotion)
1452 520 | | | | |   )
1453 521 | | | | |     for (const piece of PROMOTION_PIECES) {
1454 522 | | | | |       const button = document.createElement('button');

```

```

1455 523 | | | | | ((button.className = 'promotion-button'),
1456 524 | | | | | (button.type = 'button'),
1457 525 | | | | | (button.dataset.promotion = piece),
1458 526 | | | | | button.setAttribute('aria-label', `${pieceName(piece)} 승격`),
1459 527 | | | | | (button.textContent = PIECE_GLYPHS[game.turn()][piece]),
1460 528 | | | | | promotionChoices.appendChild(button));
1461 529 | | | | | }
1462 530 | | | | | })());
1463 531 | | | }
1464 532 | function renderClocks() {
1465 533 | | const bottomColor = getBoardOrientation(),
1466 534 | | topClockElement = 'w' === oppositeColor(bottomColor) ? whiteClockElement : blackClockElement,
1467 535 | | bottomClockElement = 'w' === bottomColor ? whiteClockElement : blackClockElement;
1468 536 | | ((clockDisplayElement.children[0] === topClockElement &&
1469 537 | | clockDisplayElement.children[1] === bottomClockElement) ||
1470 538 | | clockDisplayElement.replaceChildren(topClockElement, bottomClockElement),
1471 539 | | renderClockPosition('w', bottomColor),
1472 540 | | renderClockPosition('b', bottomColor),
1473 541 | | renderClock('w', whiteClockElement, whiteClockTimeElement),
1474 542 | | renderClock('b', blackClockElement, blackClockTimeElement));
1475 543 | }
1476 544 | function renderClockPosition(color, bottomColor) {
1477 545 | | const clockElement = 'w' === color ? whiteClockElement : blackClockElement,
1478 546 | | isBottom = color === bottomColor;
1479 547 | | (clockElement.classList.toggle('is-bottom', isBottom),
1480 548 | | clockElement.classList.toggle('is-top', !isBottom));
1481 549 | }
1482 550 | function renderClock(color, clockElement, timeElement) {
1483 551 | | const remainingMs = clockMs[color],
1484 552 | | isActive = activeClockColor === color && null !== timedOutColor,
1485 553 | | isTimedOut = timedOutColor === color;
1486 554 | | (clockElement.classList.toggle('is-active', isActive),
1487 555 | | clockElement.classList.toggle('is-warning', remainingMs ≤ 3e4),
1488 556 | | clockElement.classList.toggle('is-danger', remainingMs ≤ 1e4),
1489 557 | | clockElement.classList.toggle('is-timeout', isTimedOut),
1490 558 | | (timeElement.textContent = (function formatClockTime(milliseconds) {
1491 559 | | | const safeMilliseconds = Math.max(0, milliseconds);
1492 560 | | | if (safeMilliseconds < 1e4)
1493 561 | | | return `0:${(safeMilliseconds / 1e3).toFixed(1).padStart(4, '0')}`;
1494 562 | | | const totalSeconds = Math.ceil(safeMilliseconds / 1e3);
1495 563 | | | return `${Math.floor(totalSeconds / 60)}:${String(totalSeconds % 60).padStart(2, '0')}`;
1496 564 | | | })(remainingMs)));
1497 565 | }
1498 566 | function renderStatus() {
1499 567 | | const historyLength = game.history().length;
1500 568 | | ((statusElement.textContent = (function getStatusText() {
1501 569 | | | return game.isCheckmate()
1502 570 | | | ? `${colorName(oppositeColor(game.turn()))} 체크메이트 승리`
1503 571 | | | : game.isStalemate()
1504 572 | | | ? '스테일메이트 무승부'
1505 573 | | | : game.isInsufficientMaterial()
1506 574 | | | ? '기물 부족 무승부'
1507 575 | | | : game.isThreefoldRepetition()
1508 576 | | | ? '3회 반복 무승부'
1509 577 | | | : game.isDrawByFiftyMoves()
1510 578 | | | ? '50수 규칙 무승부'
1511 579 | | | : game.isDraw()
1512 580 | | | ? '무승부'
1513 581 | | | : timedOutColor
1514 582 | | | ? [
1515 583 | | | | `${colorName(timedOutColor)} 시간 초과`,
1516 584 | | | | `${colorName(oppositeColor(timedOutColor))} 승리`,
1517 585 | | | ].join('...')
1518 586 | | | : aiThinking
1519 587 | | | ? 'AI 계산 중'
1520 588 | | | : game.isCheck()
1521 589 | | | ? `${colorName(game.turn())} 체크`
1522 590 | | | : `${colorName(game.turn())} 차례`;
1523 591 | | | })()),
1524 592 | | (turnMetaElement.textContent = colorName(game.turn())),
1525 593 | | (modeMetaElement.textContent =

```

```

594 | | | | | 'local' === mode
595 | | | | | ? '로컬 2인'
596 | | | | | : [ 'AI-상대', `인간-${colorName(humanColor)}`, presetName(aiSettings.preset)].join(
597 | | | | | '...',
598 | | | | | ),
599 | | | | | (moveMetaElement.textContent := [`${game.moveNumber()}수`, `${historyLength}·half-move`].join(
600 | | | | | '...',
601 | | | | | ));
602 | | | | | }
603 | | | | | function renderAiCandidateDebug(){
604 | | | | | if ('ai' !== mode)
605 | | | | | return (
606 | | | | | (aiCandidatePanelElement.hidden := !0),
607 | | | | | (aiCandidateMetaElement.textContent := ''),
608 | | | | | (aiInfoStatusElement.textContent := ''),
609 | | | | | (aiInfoPresetElement.textContent := ''),
610 | | | | | (aiInfoHumanElement.textContent := ''),
611 | | | | | (aiInfoDepthElement.textContent := ''),
612 | | | | | (aiInfoNodesElement.textContent := ''),
613 | | | | | (aiInfoScoreElement.textContent := ''),
614 | | | | | void aiCandidateListElement.replaceChildren()
615 | | | | | );
616 | | | | | const debug := aiProgress?.debug;
617 | | | | | if (
618 | | | | | ((aiCandidatePanelElement.hidden := !1),
619 | | | | | (aiCandidateMetaElement.textContent := aiProgress
620 | | | | | ? `경과-${formatSeconds(aiProgress.elapsedMs)}`
621 | | | | | : ''),
622 | | | | | (aiInfoStatusElement.textContent := (function getAiInfoStatusText(){
623 | | | | | return game.isGameOver() || null !== timedOutColor
624 | | | | | ? '대국 종료'
625 | | | | | : aiThinking
626 | | | | | ? aiProgress
627 | | | | | ? '계산 중'
628 | | | | | : '계산 준비 중'
629 | | | | | : game.turn() === humanColor
630 | | | | | ? '대기 중'
631 | | | | | : '응수 대기';
632 | | | | | })()),
633 | | | | | (aiInfoPresetElement.textContent := presetName(aiSettings.preset)),
634 | | | | | (aiInfoHumanElement.textContent := colorName(humanColor)),
635 | | | | | (aiInfoDepthElement.textContent := aiProgress ? String(aiProgress.depthReached) : '-'),
636 | | | | | (aiInfoNodesElement.textContent := aiProgress
637 | | | | | ? `${(function formatNodes(nodes){
1570 - | | | | | return nodes ≥ 1e6
1571 - | | | | | ? `${(nodes / 1e6).toFixed(1)}M`
1572 - | | | | | : nodes ≥ 1e3
1573 - | | | | | ? `${Math.round(nodes / 1e3)}k`
1574 - | | | | | : String(nodes);
638 + | | | | | return nodes ≥ 1e6 ? `${Math.round(nodes / 1e3)}k` : String(nodes);
1575 639 | | | | | })(aiProgress.nodes)}·nodes`
1576 640 | | | | | : '-'),
1577 641 | | | | | (aiInfoScoreElement.textContent := aiProgress ? formatDebugScore(aiProgress.score) : '-'),
1578 642 | | | | | !debug || 0 === debug.candidates.length)
1579 643 | | | | | )
1580 644 | | | | | return void aiCandidateListElement.replaceChildren();
1581 645 | | | | | aiCandidateMetaElement.textContent := [
1582 646 | | | | | aiCandidateMetaElement.textContent,
1583 647 | | | | | `best-${formatDebugScore(debug.bestScore)}`,
1584 648 | | | | | `창-${Math.round(debug.windowCp)}`,
1585 649 | | | | | `p-${debug.ticketPower.toFixed(2)}`,
1586 650 | | | | | ].join(' ');
1587 651 | | | | | const fragment := document.createDocumentFragment();
1588 652 | | | | | for (const candidate of debug.candidates) fragment.appendChild(createAiCandidateRow(candidate));
1589 653 | | | | | aiCandidateListElement.replaceChildren(fragment);
1590 654 | | | | | }
1591 655 | | | | | function createAiCandidateRow(candidate){
1592 656 | | | | | const row := document.createElement('div');
1593 657 | | | | | ((row.className := 'ai-candidate-row'),
1594 658 | | | | | row.classList.toggle('is-selected', candidate.selected),
1595 659 | | | | | row.classList.toggle('is-principal', candidate.principal),

```

```

1596 660 | | | | | row.setAttribute('role', 'row'));
1597 661 | | | | | const sanCell = createAiCandidateCell('ai-candidate-san', candidate.san);
1598 662 | | | | | return (
1599 - | | | | | candidate.selected && sanCell.appendChild(createAiCandidateBadge('SEL')),
1600 - | | | | | candidate.principal && sanCell.appendChild(createAiCandidateBadge('PV')),
    663 + | | | | | candidate.selected && sanCell.appendChild(createAiCandidateBadge('선택')),
    664 + | | | | | candidate.principal && sanCell.appendChild(createAiCandidateBadge('최선')),
1601 665 | | | | | row.append(
1602 666 | | | | | | | sanCell,
1603 667 | | | | | | | createAiCandidateCell('ai-candidate-score', formatDebugScore(candidate.score)),
1604 668 | | | | | | | createAiCandidateCell(
1605 669 | | | | | | | | | 'ai-candidate-weight',
1606 670 | | | | | | | | | (function formatDebugWeight(weight) {
1607 671 | | | | | | | | | | | return 0 === weight
1608 672 | | | | | | | | | | | ? '0'
1609 673 | | | | | | | | | | | : weight < 0.001
1610 674 | | | | | | | | | | | ? '<0.001'
1611 675 | | | | | | | | | | | : weight.toFixed(3).replace(/0+$/, '').replace(/\.$/, '');
1612 676 | | | | | | | | | | | })(candidate.tickets),
1613 677 | | | | | | | | | | | ),
1614 678 | | | | | | | | | | | createAiCandidateCell(
1615 679 | | | | | | | | | | | | | 'ai-candidate-probability',
1616 680 | | | | | | | | | | | | | (function formatDebugProbability(probability) {
1617 681 | | | | | | | | | | | | | | | const percent = 100 * probability;
1618 682 | | | | | | | | | | | | | | | return 0 === percent ? '0%' : percent < 0.1 ? '<0.1%' : `${percent.toFixed(1)}%`;
1619 683 | | | | | | | | | | | | | | | })(candidate.probability),
1620 684 | | | | | | | | | | | | | | | ),
1621 685 | | | | | | | | | | | | | | | ),
1622 686 | | | | | | | | | | | | | | | row
1623 687 | | | | | | | | | | | | | | | );
1624 688 | | | | | | | | | | | | | | | }
1625 689 | | | | | function createAiCandidateCell(className, text) {
1626 690 | | | | | | | const cell = document.createElement('span');
1627 691 | | | | | | | return (
1628 692 | | | | | | | | | (cell.className = className),
1629 693 | | | | | | | | | cell.setAttribute('role', 'cell'),
1630 694 | | | | | | | | | (cell.textContent = text),
1631 695 | | | | | | | | | cell
1632 696 | | | | | | | | | );
1633 697 | | | | | | | | | }
1634 698 | | | | | function createAiCandidateBadge(text) {
1635 699 | | | | | | | const badge = document.createElement('span');
1636 700 | | | | | | | return ((badge.className = 'ai-candidate-badge'), (badge.textContent = text), badge);
1637 701 | | | | | | | | | }
1638 702 | | | | | function getSquareClasses(square, lastMove) {
1639 703 | | | | | | | const classes = ['board-square', 'light' === game.squareColor(square) ? 'is-light' : 'is-dark'],
1640 704 | | | | | | | destinationMove = legalMoves.find((move) => move.to === square);
1641 705 | | | | | | | return (
1642 706 | | | | | | | | | selectedSquare === square && classes.push('is-selected'),
1643 707 | | | | | | | | | destinationMove && classes.push(destinationMove.isCapture() ? 'is-capture' : 'is-legal'),
1644 708 | | | | | | | | | !lastMove ||
1645 709 | | | | | | | | | (lastMove.from === square && lastMove.to === square) ||
1646 710 | | | | | | | | | classes.push('is-last-move'),
1647 711 | | | | | | | | | (function isCheckedKing(square) {
1648 712 | | | | | | | | | | | const piece = game.get(square);
1649 713 | | | | | | | | | | | return Boolean(
1650 714 | | | | | | | | | | | | | piece && 'k' === piece.type && piece.color === game.turn() && game.isCheck(),
1651 715 | | | | | | | | | | | | | );
1652 716 | | | | | | | | | | | })(square) && classes.push('is-check'),
1653 717 | | | | | | | | | | | classes.join(' ')
1654 718 | | | | | | | | | );
1655 719 | | | | | | | | | }
1656 720 | | | | | function getBoardOrientation() {
1657 721 | | | | | | | return 'ai' === mode ? humanColor : game.turn();
1658 722 | | | | | | | }
1659 723 | | | | | function canUseBoard() {
1660 724 | | | | | | | return !(
1661 725 | | | | | | | | | game.isGameOver() ||
1662 726 | | | | | | | | | null === timedOutColor ||
1663 727 | | | | | | | | | aiThinking ||
1664 728 | | | | | | | | | pendingPromotion ||

```

```

1665 729 | | | | ('local' !== mode && game.turn() !== humanColor)
1666 730 | | | | );
1667 731 | | | | }
1668 732 | | | | (boardElement.addEventListener('click', (event) => {
1669 733 | | | |   const button = (function getSquareButton(target, boardElement) {
1670 734 | | | |     if (!target instanceof Element) return null;
1671 735 | | | |     const button = target.closest('.board-square');
1672 736 | | | |     return button && boardElement.contains(button) ? button : null;
1673 737 | | | |   })(event.target, boardElement);
1674 738 | | | |   button &&
1675 739 | | | |     (function isSquare(value) {
1676 740 | | | |       return 'string' === typeof value && SQUARE_SET.has(value);
1677 741 | | | |     })(button.dataset.square) &&
1678 742 | | | |     (function handleSquareClick(square) {
1679 743 | | | |       if (!canUseBoard()) return;
1680 744 | | | |       const piece = game.get(square);
1681 745 | | | |       if (selectedSquare) {
1682 746 | | | |         const movesToDestination = legalMoves.filter((move) => move.to === square);
1683 747 | | | |         return movesToDestination.length > 0
1684 748 | | | |           ? movesToDestination.some((move) => move.isPromotion())
1685 749 | | | |             ? ((pendingPromotion = {
1686 750 | | | |               from: selectedSquare,
1687 751 | | | |               to: square,
1688 752 | | | |               moves: movesToDestination,
1689 753 | | | |             }),
1690 754 | | | |             void render())
1691 755 | | | |           : void playMove({ from: selectedSquare, to: square })
1692 756 | | | |           : piece?.color === game.turn()
1693 757 | | | |             ? void selectSquare(square)
1694 758 | | | |             : (clearSelection(), void render());
1695 759 | | | |       }
1696 760 | | | |       piece?.color === game.turn() && selectSquare(square);
1697 761 | | | |     })(button.dataset.square);
1698 762 | | | |   }),
1699 763 | | | |   modeButtons.forEach((button) => {
1700 764 | | | |     button.addEventListener('click', () => {
1701 765 | | | |       const nextMode = button.dataset.mode;
1702 766 | | | |       (function isGameMode(value) {
1703 767 | | | |         return 'local' === value || 'ai' === value;
1704 768 | | | |       })(nextMode) &&
1705 769 | | | |       nextMode !== mode &&
1706 770 | | | |       ((mode = nextMode), resetGame());
1707 771 | | | |     });
1708 772 | | | |   }),
1709 773 | | | |   colorButtons.forEach((button) => {
1710 774 | | | |     button.addEventListener('click', () => {
1711 775 | | | |       const nextColor = button.dataset.color;
1712 776 | | | |       (function isColor(value) {
1713 777 | | | |         return 'w' === value || 'b' === value;
1714 778 | | | |       })(nextColor) &&
1715 779 | | | |       nextColor !== humanColor &&
1716 780 | | | |       ((humanColor = nextColor), resetGame());
1717 781 | | | |     });
1718 782 | | | |   }),
1719 783 | | | |   presetButtons.forEach((button) => {
1720 784 | | | |     button.addEventListener('click', () => {
1721 785 | | | |       const preset = button.dataset.preset;
1722 786 | | | |       (function isAiPreset(value) {
1723 787 | | | |         return 'easy' === value || 'normal' === value || 'hard' === value;
1724 788 | | | |       })(preset) && updateAiSettings({... AI_PRESETS[preset]});
1725 789 | | | |     });
1726 790 | | | |   }),
1727 791 | | | |   clockPresetButtons.forEach((button) => {
1728 792 | | | |     button.addEventListener('click', () => {
1729 793 | | | |       const preset = button.dataset.clockPreset;
1730 794 | | | |       (function isClockPreset(value) {
1731 795 | | | |         return (
1732 796 | | | |           '1/1' === value ||
1733 797 | | | |           '3/2' === value ||
1734 798 | | | |           '5/3' === value ||
1735 799 | | | |           '10/5' === value ||

```



```

1736 800 |         .value;
1737 801 |     });
1738 802 |   })(preset) &&
1739 803 |   preset !== clockPreset &&
1740 804 |   ((clockPreset = preset), resetGame());
1741 805 |   });
1742 806 | },
1743 807 | aiDepthInput.addEventListener('input', () => {
1744 808 |   var value;
1745 809 |   updateAiSettings({
1746 810 |     ...aiSettings,
1747 811 |     maxDepth:
1748 812 |       ((value = aiDepthInput.valueAsNumber), Math.min(5, Math.max(1, Math.round(value)))),
1749 813 |   });
1750 814 | },
1751 815 | aiTimeInput.addEventListener('input', () => {
1752 816 |   updateAiSettings({ ...aiSettings, timeLimitMs: aiTimeInput.valueAsNumber });
1753 817 | }),
1754 818 | newGameButton.addEventListener('click', resetGame),
1755 819 | undoButton.addEventListener('click', function undoLastTurn() {
1756 820 |   cancelAiMove(),
1757 821 |   (pendingPromotion = null),
1758 822 |   0 !== game.history().length
1759 823 |   ? ('ai' === mode && game.turn() === humanColor ? (game.undo(), game.undo()) : game.undo()),
1760 824 |   clearSelection(),
1761 825 |   resetClockState(),
1762 826 |   restartClockForCurrentTurn(),
1763 827 |   render()
1764 828 |   : render();
1765 829 | })),
1766 830 | promotionChoices.addEventListener('click', (event) => {
1767 831 |   const target = event.target;
1768 832 |   if (!(target instanceof Element)) return;
1769 833 |   const button = target.closest('[data-promotion]'),
1770 834 |   promotion = button?.dataset.promotion;
1771 835 |   (function isPromotionEvent(piece) {
1772 836 |     return 'q' === piece || 'r' === piece || 'b' === piece || 'n' === piece;
1773 837 |   })(promotion) &&
1774 838 |   (function finishPromotion(promotion) {
1775 839 |     if (!pendingPromotion) return;
1776 840 |     if (!pendingPromotion.moves.find((move) => move.promotion === promotion)) return;
1777 841 |     const { from, to } = pendingPromotion;
1778 842 |     ((pendingPromotion = null), playMove({ from, to, promotion }));
1779 843 |   })(promotion);
1780 844 | },
1781 845 | resetClockState(),
1782 846 | restartClockForCurrentTurn(),
1783 847 | render());
1784 848 | })();
```

```
==== D main.js :: inline worker jsContent #1 (52396 B / 865fe4169163 → -) ====
```

```

1 - |!(function ()-{
2 - |    'use strict';
3 - |    const MASK64 = 0xffffffffffffffff;
4 - |    function rotl(x, k){
5 - |        return 0xffffffffffffffff & ((x << k) | (x >> (64n - k)));
6 - |    }
7 - |    function wrappingMul(x, y){
8 - |        return (x * y) & MASK64;
9 - |    }
10 - |    const rand = (function xoroshiro128(state){
11 - |        return function ()-{
12 - |            let s0 = BigInt(state & MASK64),
13 - |            s1 = BigInt((state >> 64n) & MASK64);
14 - |            const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
15 - |            return (
16 - |                (s1 ^ s0),
17 - |                (s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64),
18 - |                (s1 = rotl(s1, 37n)),
19 - |                (state = (s1 << 64n) | s0),
20 - |                result

```



```

21 - | | | | |);
22 - | | | | |};
23 - | | | | |})(0xa187eb39cdcaed8f31c4b365b102e01en),
24 - | | | | |PIECE_KEYS = Array.from({length: 2}, () =>
25 - | | | | |Array.from({length: 6}, () => Array.from({length: 128}, () => rand()))),
26 - | | | | |),
27 - | | | | |EP_KEYS = Array.from({length: 8}, () => rand()),
28 - | | | | |CASTLING_KEYS = Array.from({length: 16}, () => rand()),
29 - | | | | |SIDE_KEY = rand(),
30 - | | | | |DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR-w·KQkq--0·1';
31 - | | | | |class Move {
32 - | | | | |color;
33 - | | | | |from;
34 - | | | | |to;
35 - | | | | |piece;
36 - | | | | |captured;
37 - | | | | |promotion;
38 - | | | | |flags;
39 - | | | | |san;
40 - | | | | |lan;
41 - | | | | |before;
42 - | | | | |after;
43 - | | | | |constructor(chess, internal) {
44 - | | | | |const {
45 - | | | | |color: color,
46 - | | | | |piece: piece,
47 - | | | | |from: from,
48 - | | | | |to: to,
49 - | | | | |flags: flags,
50 - | | | | |captured: captured,
51 - | | | | |promotion: promotion,
52 - | | | | |} = internal,
53 - | | | | |fromAlgebraic = algebraic(from),
54 - | | | | |toAlgebraic = algebraic(to);
55 - | | | | |(this.color = color),
56 - | | | | |(this.piece = piece),
57 - | | | | |(this.from = fromAlgebraic),
58 - | | | | |(this.to = toAlgebraic),
59 - | | | | |(this.san = chess._moveToSan(internal, chess._moves({legal: !0}))),
60 - | | | | |(this.lan = fromAlgebraic + toAlgebraic),
61 - | | | | |(this.before = chess.fen()),
62 - | | | | |chess._makeMove(internal),
63 - | | | | |(this.after = chess.fen()),
64 - | | | | |chess._undoMove(),
65 - | | | | |(this.flags = '');
66 - | | | | |for (const flag in BITS) BITS[flag] & flags & (this.flags += FLAGS[flag]);
67 - | | | | |(captured & (this.captured = captured),
68 - | | | | |promotion & ((this.promotion = promotion), (this.lan += promotion)));
69 - | | | | |}
70 - | | | | |isCapture() {
71 - | | | | |return this.flags.indexOf(FLAGS.CAPTURE) > -1;
72 - | | | | |}
73 - | | | | |isPromotion() {
74 - | | | | |return this.flags.indexOf(FLAGS.PROMOTION) > -1;
75 - | | | | |}
76 - | | | | |isEnPassant() {
77 - | | | | |return this.flags.indexOf(FLAGS.EP_CAPTURE) > -1;
78 - | | | | |}
79 - | | | | |isKingsideCastle() {
80 - | | | | |return this.flags.indexOf(FLAGS.KSIDE_CASTLE) > -1;
81 - | | | | |}
82 - | | | | |isQueensideCastle() {
83 - | | | | |return this.flags.indexOf(FLAGS.QSIDE_CASTLE) > -1;
84 - | | | | |}
85 - | | | | |isBigPawn() {
86 - | | | | |return this.flags.indexOf(FLAGS.BIG_PAWN) > -1;
87 - | | | | |}
88 - | | | | |}
89 - | | | | |const FLAGS = {
90 - | | | | |NORMAL: 'n',
91 - | | | | |CAPTURE: 'c',

```

```

92 - | | | BIG_PAWN: 'b',
93 - | | | EP_CAPTURE: 'e',
94 - | | | PROMOTION: 'p',
95 - | | | KSIDE_CASTLE: 'k',
96 - | | | QSIDE_CASTLE: 'q',
97 - | | | NULL_MOVE: '-',
98 - | | | },
99 - | | | BITS = {
100 - | | | NORMAL: 1,
101 - | | | CAPTURE: 2,
102 - | | | BIG_PAWN: 4,
103 - | | | EP_CAPTURE: 8,
104 - | | | PROMOTION: 16,
105 - | | | KSIDE_CASTLE: 32,
106 - | | | QSIDE_CASTLE: 64,
107 - | | | NULL_MOVE: 128,
108 - | | | },
109 - | | | 0x88 = {
110 - | | | a8: 0,
111 - | | | b8: 1,
112 - | | | c8: 2,
113 - | | | d8: 3,
114 - | | | e8: 4,
115 - | | | f8: 5,
116 - | | | g8: 6,
117 - | | | h8: 7,
118 - | | | a7: 16,
119 - | | | b7: 17,
120 - | | | c7: 18,
121 - | | | d7: 19,
122 - | | | e7: 20,
123 - | | | f7: 21,
124 - | | | g7: 22,
125 - | | | h7: 23,
126 - | | | a6: 32,
127 - | | | b6: 33,
128 - | | | c6: 34,
129 - | | | d6: 35,
130 - | | | e6: 36,
131 - | | | f6: 37,
132 - | | | g6: 38,
133 - | | | h6: 39,
134 - | | | a5: 48,
135 - | | | b5: 49,
136 - | | | c5: 50,
137 - | | | d5: 51,
138 - | | | e5: 52,
139 - | | | f5: 53,
140 - | | | g5: 54,
141 - | | | h5: 55,
142 - | | | a4: 64,
143 - | | | b4: 65,
144 - | | | c4: 66,
145 - | | | d4: 67,
146 - | | | e4: 68,
147 - | | | f4: 69,
148 - | | | g4: 70,
149 - | | | h4: 71,
150 - | | | a3: 80,
151 - | | | b3: 81,
152 - | | | c3: 82,
153 - | | | d3: 83,
154 - | | | e3: 84,
155 - | | | f3: 85,
156 - | | | g3: 86,
157 - | | | h3: 87,
158 - | | | a2: 96,
159 - | | | b2: 97,
160 - | | | c2: 98,
161 - | | | d2: 99,
162 - | | | e2: 100,

```

```

163 - | | | f2: 101,
164 - | | | g2: 102,
165 - | | | h2: 103,
166 - | | | a1: 112,
167 - | | | b1: 113,
168 - | | | c1: 114,
169 - | | | d1: 115,
170 - | | | e1: 116,
171 - | | | f1: 117,
172 - | | | g1: 118,
173 - | | | h1: 119,
174 - | | | },
175 - | | | PAWN_OFFSETS = { b: [16, 32, 17, 15], w: [-16, -32, -17, -15] },
176 - | | | PIECE_OFFSETS = {
177 - | | | | n: [-18, -33, -31, -14, 18, 33, 31, 14],
178 - | | | | b: [-17, -15, 17, 15],
179 - | | | | r: [-16, 1, 16, -1],
180 - | | | | q: [-17, -16, -15, -1, 17, 16, 15, -1],
181 - | | | | k: [-17, -16, -15, -1, 17, 16, 15, -1],
182 - | | | },
183 - | | | ATTACKS = [
184 - | | | | 20, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0,
185 - | | | | 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0,
186 - | | | | 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 20, 2,
187 - | | | | 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 24, 24, 24,
188 - | | | | 24, 24, 24, 56, 0, 56, 24, 24, 24, 24, 24, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0,
189 - | | | | 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20,
190 - | | | | 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0,
191 - | | | | 0, 0, 0, 20, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 0,
192 - | | | | 24, 0, 0, 0, 0, 0, 0, 20,
193 - | | | ],
194 - | | | RAYS = [
195 - | | | | 17, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0,
196 - | | | | 15, 0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 15, 0, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0,
197 - | | | | 0, 15, 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 0,
198 - | | | | 16, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
199 - | | | | 1, 1, 0, -1, -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0, 0, 0,
200 - | | | | 0, 0, 0, 0, 0, -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0, 0, -17,
201 - | | | | 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, -15, 0, 0, 0, 0,
202 - | | | | -16, 0, 0, 0, 0, -17, 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, 0, -17, 0, 0, -15, 0,
203 - | | | | 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, 0, -17,
204 - | | | ],
205 - | | | PIECE_MASKS = { p: 1, n: 2, b: 4, r: 8, q: 16, k: 32 },
206 - | | | PROMOTIONS = ['n', 'b', 'r', 'q'],
207 - | | | ROOKS = {
208 - | | | | w: [
209 - | | | | | { square: 0x88.a1, flag: BITS.QSIDE_CASTLE },
210 - | | | | | { square: 0x88.h1, flag: BITS.KSIDE_CASTLE },
211 - | | | | ],
212 - | | | | b: [
213 - | | | | | { square: 0x88.a8, flag: BITS.QSIDE_CASTLE },
214 - | | | | | { square: 0x88.h8, flag: BITS.KSIDE_CASTLE },
215 - | | | | ],
216 - | | | | },
217 - | | | SECOND_RANK = { b: 1, w: 6 };
218 - | | | function rank(square) {
219 - | | | | return square >> 4;
220 - | | | }
221 - | | | function file(square) {
222 - | | | | return 15 & square;
223 - | | | }
224 - | | | function isDigit(c) {
225 - | | | | return -1 !== '0123456789'.indexOf(c);
226 - | | | }
227 - | | | function algebraic(square) {
228 - | | | | const f = file(square),
229 - | | | | r = rank(square);
230 - | | | | return 'abcdefgh'.substring(f, f + 1) + '87654321'.substring(r, r + 1);
231 - | | | }
232 - | | | function swapColor(color) {
233 - | | | | return 'w' === color ? 'b' : 'w';

```

```

234 - | }
235 - | function addMove(moves, color, from, to, piece, captured = void 0, flags = BITS.NORMAL) {
236 - |   const r = rank(to);
237 - |   if ('p' !== piece || (7 !== r & 0 !== r))
238 - |     moves.push({
239 - |       color: color,
240 - |       from: from,
241 - |       to: to,
242 - |       piece: piece,
243 - |       captured: captured,
244 - |       flags: flags,
245 - |     });
246 - |   else
247 - |     for (let i = 0; i < PROMOTIONS.length; i++) {
248 - |       const promotion = PROMOTIONS[i];
249 - |       moves.push({
250 - |         color: color,
251 - |         from: from,
252 - |         to: to,
253 - |         piece: piece,
254 - |         captured: captured,
255 - |         promotion: promotion,
256 - |         flags: flags | BITS.PROMOTION,
257 - |       });
258 - |     }
259 - | }
260 - | class Chess {
261 - |   _board = new Array(128);
262 - |   _turn = 'w';
263 - |   _kings = { w: -1, b: -1 };
264 - |   _epSquare = -1;
265 - |   _halfMoves = 0;
266 - |   _moveNumber = 0;
267 - |   _history = [];
268 - |   _castling = { w: 0, b: 0 };
269 - |   _hash = 0n;
270 - |   _positionCount = new Map();
271 - |   constructor(fen = DEFAULT_POSITION, { skipValidation: skipValidation = !1 } = {}) {
272 - |     this.load(fen, { skipValidation: skipValidation });
273 - |   }
274 - |   clear() {
275 - |     ((this._board = new Array(128)),
276 - |     (this._kings = { w: -1, b: -1 }),
277 - |     (this._turn = 'w'),
278 - |     (this._castling = { w: 0, b: 0 }),
279 - |     (this._epSquare = -1),
280 - |     (this._halfMoves = 0),
281 - |     (this._moveNumber = 1),
282 - |     (this._history = []),
283 - |     (this._hash = this._computeHash()),
284 - |     (this._positionCount = new Map()));
285 - |   }
286 - |   load(fen, { skipValidation: skipValidation = !1 } = {}) {
287 - |     let tokens = fen.split(/\s+/);
288 - |     if (tokens.length ≥ 2 && tokens.length < 6) {
289 - |       const adjustments = ['- ', '- ', '0', '1'];
290 - |       fen = tokens.concat(adjustments.slice(-(6 - tokens.length))).join(' ');
291 - |     }
292 - |     if (((tokens = fen.split(/\s+/)), !skipValidation)) {
293 - |       const { ok: ok, error: error } = (function validateFen(fen) {
294 - |         const tokens = fen.split(/\s+/);
295 - |         if (6 !== tokens.length)
296 - |           return { ok: !1, error: 'Invalid FEN: must contain six space-delimited fields' };
297 - |         const moveNumber = parseInt(tokens[5], 10);
298 - |         if (isNaN(moveNumber) || moveNumber ≤ 0)
299 - |           return { ok: !1, error: 'Invalid FEN: move number must be a positive integer' };
300 - |         const halfMoves = parseInt(tokens[4], 10);
301 - |         if (isNaN(halfMoves) || halfMoves < 0)
302 - |           return {
303 - |             ok: !1,
304 - |             error: 'Invalid FEN: half move counter number must be a non-negative integer',

```

```

305 - |         };
306 - |         if (!/^(-[abcdefgh][36])$/.test(tokens[3]))
307 - |             return { ok: !1, error: 'Invalid FEN: en-passant square is invalid' };
308 - |         if (/^[kKqQ-]/.test(tokens[2]))
309 - |             return { ok: !1, error: 'Invalid FEN: castling availability is invalid' };
310 - |         if (!/(w|b)$/.test(tokens[1]))
311 - |             return { ok: !1, error: 'Invalid FEN: side-to-move is invalid' };
312 - |         const rows = tokens[0].split('/');
313 - |         if (8 !== rows.length)
314 - |             return {
315 - |                 ok: !1,
316 - |                 error: "Invalid FEN: piece data does not contain 8 '/'-delimited rows",
317 - |             };
318 - |         for (let i = 0; i < rows.length; i++) {
319 - |             let sumFields = 0,
320 - |                 previousWasNumber = !1;
321 - |             for (let k = 0; k < rows[i].length; k++)
322 - |                 if (isDigit(rows[i][k])) {
323 - |                     if (previousWasNumber)
324 - |                         return {
325 - |                             ok: !1,
326 - |                             error: 'Invalid FEN: piece data is invalid (consecutive number)',
327 - |                         };
328 - |                     ((sumFields += parseInt(rows[i][k], 10)), (previousWasNumber = !0));
329 - |                 } else {
330 - |                     if (!/[prnbqkPRNBQK]$/.test(rows[i][k]))
331 - |                         return { ok: !1, error: 'Invalid FEN: piece data is invalid (invalid piece)' };
332 - |                     ((sumFields += 1), (previousWasNumber = !1));
333 - |                 }
334 - |             if (8 !== sumFields)
335 - |                 return {
336 - |                     ok: !1,
337 - |                     error: 'Invalid FEN: piece data is invalid (too many squares in rank)',
338 - |                 };
339 - |         }
340 - |         if (
341 - |             ('3' === tokens[3][1] && 'w' === tokens[1]) ||
342 - |             ('6' === tokens[3][1] && 'b' === tokens[1])
343 - |         )
344 - |             return { ok: !1, error: 'Invalid FEN: illegal en-passant square' };
345 - |         const kings = [
346 - |             { color: 'white', regex: /K/g },
347 - |             { color: 'black', regex: /k/g },
348 - |         ];
349 - |         for (const { color, regex: regex } of kings) {
350 - |             if (!regex.test(tokens[0]))
351 - |                 return { ok: !1, error: `Invalid FEN: missing ${color} king` };
352 - |             if ((tokens[0].match(regex) || []).length > 1)
353 - |                 return { ok: !1, error: `Invalid FEN: too many ${color} kings` };
354 - |         }
355 - |         return Array.from(rows[0] + rows[7]).some((char) => 'P' === char.toUpperCase())
356 - |             ? { ok: !1, error: 'Invalid FEN: some pawns are on the edge rows' }
357 - |             : { ok: !0 };
358 - |     })(fen);
359 - |     if (!ok) throw new Error(error);
360 - | }
361 - |
362 - | const position = tokens[0];
363 - | let square = 0;
364 - | this.clear();
365 - | for (let i = 0; i < position.length; i++) {
366 - |     const piece = position.charAt(i);
367 - |     if ('/' === piece) square += 8;
368 - |     else if (isDigit(piece)) square += parseInt(piece, 10);
369 - |     else {
370 - |         const color = piece < 'a' ? 'w' : 'b';
371 - |         (this._put({ type: piece.toLowerCase(), color: color }, algebraic(square)), square++);
372 - |     }
373 - | }
374 - | ((this._turn = tokens[1]),
375 - |   tokens[2].indexOf('K') > -1 && (this._castling.w |= BITS.KSIDE_CASTLE),
376 - |   tokens[2].indexOf('Q') > -1 && (this._castling.w |= BITS.QSIDE_CASTLE),

```

```

376 - | .tokens[2].indexOf('k') > -1 && (this._castling.b |= BITS.KSIDE_CASTLE),
377 - | .tokens[2].indexOf('q') > -1 && (this._castling.b |= BITS.QSIDE_CASTLE),
378 - | (this._epSquare = '-' === tokens[3] ? -1 : 0x88[tokens[3]]),
379 - | (this._halfMoves = parseInt(tokens[4], 10)),
380 - | (this._moveNumber = parseInt(tokens[5], 10)),
381 - | (this._hash = this._computeHash()),
382 - | this._incPositionCount();
383 - | }
384 - | fen({ forceEnpassantSquare: forceEnpassantSquare = !1 } = {}) {
385 - |   let empty = 0,
386 - |   fen = '';
387 - |   for (let i = 0x88.a8; i <= 0x88.h1; i++) {
388 - |     if (this._board[i]) {
389 - |       empty > 0 && ((fen += empty), (empty = 0));
390 - |       const { color, type: piece } = this._board[i];
391 - |       fen += 'w' === color ? piece.toUpperCase() : piece.toLowerCase();
392 - |     } else empty++;
393 - |     (i + 1) & 136 &&
394 - |     (empty > 0 && (fen += empty), i !== 0x88.h1 && (fen += '/'), (empty = 0), (i += 8));
395 - |   }
396 - |   let castling = '';
397 - |   (this._castling.w & BITS.KSIDE_CASTLE && (castling += 'K'),
398 - |   this._castling.w & BITS.QSIDE_CASTLE && (castling += 'Q'),
399 - |   this._castling.b & BITS.KSIDE_CASTLE && (castling += 'k'),
400 - |   this._castling.b & BITS.QSIDE_CASTLE && (castling += 'q'),
401 - |   (castling = castling || '-'));
402 - |   let epSquare = '-';
403 - |   if (-1 !== this._epSquare)
404 - |     if (forceEnpassantSquare) epSquare = algebraic(this._epSquare);
405 - |     else {
406 - |       const bigPawnSquare = this._epSquare + ('w' === this._turn ? 16 : -16),
407 - |       squares = [bigPawnSquare + 1, bigPawnSquare - 1];
408 - |       for (const square of squares) {
409 - |         if (136 & square) continue;
410 - |         const color = this._turn;
411 - |         if (this._board[square]?.color === color && 'p' === this._board[square]?.type) {
412 - |           this._makeMove({
413 - |             color: color,
414 - |             from: square,
415 - |             to: this._epSquare,
416 - |             piece: 'p',
417 - |             captured: 'p',
418 - |             flags: BITS.EP_CAPTURE,
419 - |           });
420 - |           const isLegal = !this._isKingAttacked(color);
421 - |           if ((this._undoMove(), isLegal)) {
422 - |             epSquare = algebraic(this._epSquare);
423 - |             break;
424 - |           }
425 - |         }
426 - |       }
427 - |     }
428 - |   return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
429 - | }
430 - | _pieceKey(i) {
431 - |   if (!this._board[i]) return 0n;
432 - |   const { color, type: type } = this._board[i],
433 - |   typeIndex = { p: 0, n: 1, b: 2, r: 3, q: 4, k: 5 }[type];
434 - |   return PIECE_KEYS[{'w': 0, 'b': 1}[color]][typeIndex][i];
435 - | }
436 - | _epKey() {
437 - |   return -1 === this._epSquare ? 0n : EP_KEYS[7 & this._epSquare];
438 - | }
439 - | _castlingKey() {
440 - |   const index = (this._castling.w >> 5) | (this._castling.b >> 3);
441 - |   return CASTLING_KEYS[index];
442 - | }
443 - | _computeHash() {
444 - |   let hash = 0n;
445 - |   for (let i = 0x88.a8; i <= 0x88.h1; i++)
446 - |     136 & i ? (i += 7) : this._board[i] && (hash ^= this._pieceKey(i));

```

```

447 - | .return (
448 - | .    (hash ^≡ this._epKey()),
449 - | .    (hash ^≡ this._castlingKey()),
450 - | .    'b' ≡≡ this._turn && (hash ^≡ SIDE_KEY),
451 - | .    hash
452 - | .);
453 - | .}
454 - | .reset() {
455 - | .    this.load(DEFAULT_POSITION);
456 - | .}
457 - | .get(square) {
458 - | .    return this._board[0x88[square]];
459 - | .}
460 - | ._set(sq, piece) {
461 - | .    ((this._hash ^≡ this._pieceKey(sq)),
462 - | .    (this._board[sq] = piece),
463 - | .    (this._hash ^≡ this._pieceKey(sq)));
464 - | .}
465 - | ._put({ type: type, color: color }, square) {
466 - | .    if (-1 ≡≡ 'pnbrqkPNBRQK'.indexOf(type.toLowerCase())) return !1;
467 - | .    if (!(square in 0x88)) return !1;
468 - | .    const sq = 0x88[square];
469 - | .    if ('k' ≡ type && -1 ≠ this._kings[color] && this._kings[color] ≠ sq) return !1;
470 - | .    const currentPieceOnSquare = this._board[sq];
471 - | .    return (
472 - | .        currentPieceOnSquare &&
473 - | .        'k' ≡≡ currentPieceOnSquare.type &&
474 - | .        (this._kings[currentPieceOnSquare.color] = -1),
475 - | .        this._set(sq, { type: type, color: color }),
476 - | .        'k' ≡≡ type && (this._kings[color] = sq),
477 - | .        !0
478 - | .    );
479 - | .}
480 - | ._clear(sq) {
481 - | .    ((this._hash ^≡ this._pieceKey(sq)), delete this._board[sq]);
482 - | .}
483 - | ._attacked(color, square, verbose) {
484 - | .    const attackers = [];
485 - | .    for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
486 - | .        if (136 & i) {
487 - | .            i += 7;
488 - | .            continue;
489 - | .        }
490 - | .        if (void 0 ≡≡ this._board[i] || this._board[i].color ≠ color) continue;
491 - | .        const piece = this._board[i],
492 - | .            difference = i - square;
493 - | .        if (0 ≡≡ difference) continue;
494 - | .        const index = difference + 119;
495 - | .        if (ATTACKS[index] & PIECE_MASKS[piece.type]) {
496 - | .            if ('p' ≡≡ piece.type) {
497 - | .                if (
498 - | .                    (difference > 0 && 'w' ≡≡ piece.color) ||
499 - | .                    (difference ≤ 0 && 'b' ≡≡ piece.color)
500 - | .                ) {
501 - | .                    if (!verbose) return !0;
502 - | .                    attackers.push(algebraic(i));
503 - | .                }
504 - | .                continue;
505 - | .            }
506 - | .            if ('n' ≡≡ piece.type || 'k' ≡≡ piece.type) {
507 - | .                if (verbose) {
508 - | .                    attackers.push(algebraic(i));
509 - | .                    continue;
510 - | .                }
511 - | .                return !0;
512 - | .            }
513 - | .            const offset = RAYS[index];
514 - | .            let j = i + offset,
515 - | .                blocked = !1;
516 - | .            for (; j ≠ square; ) {
517 - | .                if (null ≠ this._board[j]) {

```

```

518 - | . . . . . blocked = !0;
519 - | . . . . . break;
520 - | . . . . . }
521 - | . . . . . j += offset;
522 - | . . . . . }
523 - | . . . . . if (!blocked) {
524 - | . . . . . if (verbose) {
525 - | . . . . . attackers.push(algebraic(i));
526 - | . . . . . continue;
527 - | . . . . . }
528 - | . . . . . return !0;
529 - | . . . . . }
530 - | . . . . . }
531 - | . . . . . }
532 - | . . . . . return !!verbose && attackers;
533 - | . . . . . }
534 - | . . . . . attackers(square, attackedBy) {
535 - | . . . . . return attackedBy
536 - | . . . . . ? this._attacked(attackedBy, 0x88[square], !0)
537 - | . . . . . : this._attacked(this._turn, 0x88[square], !0);
538 - | . . . . . }
539 - | . . . . . _isKingAttacked(color) {
540 - | . . . . . const square = this._kings[color];
541 - | . . . . . return -1 !== square && this._attacked(swapColor(color), square);
542 - | . . . . . }
543 - | . . . . . hash() {
544 - | . . . . . return this._hash.toString(16);
545 - | . . . . . }
546 - | . . . . . isAttacked(square, attackedBy) {
547 - | . . . . . return this._attacked(attackedBy, 0x88[square]);
548 - | . . . . . }
549 - | . . . . . isCheck() {
550 - | . . . . . return this._isKingAttacked(this._turn);
551 - | . . . . . }
552 - | . . . . . inCheck() {
553 - | . . . . . return this.isCheck();
554 - | . . . . . }
555 - | . . . . . isCheckmate() {
556 - | . . . . . return this.isCheck() && 0 === this._moves().length;
557 - | . . . . . }
558 - | . . . . . isStalemate() {
559 - | . . . . . return !this.isCheck() && 0 === this._moves().length;
560 - | . . . . . }
561 - | . . . . . isInsufficientMaterial() {
562 - | . . . . . const pieces = { b: 0, n: 0, r: 0, q: 0, k: 0, p: 0 },
563 - | . . . . . bishops = [];
564 - | . . . . . let numPieces = 0,
565 - | . . . . . squareColor = 0;
566 - | . . . . . for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
567 - | . . . . . if (((squareColor = (squareColor + 1) % 2), 136 & i)) {
568 - | . . . . . i += 7;
569 - | . . . . . continue;
570 - | . . . . . }
571 - | . . . . . const piece = this._board[i];
572 - | . . . . . piece &&
573 - | . . . . . ((pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1),
574 - | . . . . . 'b' === piece.type && bishops.push(squareColor),
575 - | . . . . . numPieces++);
576 - | . . . . . }
577 - | . . . . . if (2 === numPieces) return !0;
578 - | . . . . . if (3 === numPieces && (1 === pieces.b || 1 === pieces.n)) return !0;
579 - | . . . . . if (numPieces === pieces.b + 2) {
580 - | . . . . . let sum = 0;
581 - | . . . . . const len = bishops.length;
582 - | . . . . . for (let i = 0; i < len; i++) sum += bishops[i];
583 - | . . . . . if (0 === sum || sum === len) return !0;
584 - | . . . . . }
585 - | . . . . . return !1;
586 - | . . . . . }
587 - | . . . . . isThreefoldRepetition() {
588 - | . . . . . return this._getPositionCount(this._hash) ≥ 3;

```



```

589 - | .}
590 - | .isDrawByFiftyMoves()·{
591 - | .    return this._halfMoves ≥ 100;
592 - | .}
593 - | .isDraw()·{
594 - | .    return (
595 - | .        this.isDrawByFiftyMoves()·||
596 - | .        this.isStalemate()·||
597 - | .        this.isInsufficientMaterial()·||
598 - | .        this.isThreefoldRepetition()
599 - | .    );
600 - | .}
601 - | .isGameOver()·{
602 - | .    return this.isCheckmate()·|| this.isDraw();
603 - | .}
604 - | .moves({ verbose: verbose = !1, square: square, piece: piece }·=·{} )·{
605 - | .    const moves = this._moves({ square: square, piece: piece });
606 - | .    return verbose
607 - | .        ? moves.map((move) ⇒ new Move(this, move))
608 - | .        : moves.map((move) ⇒ this._moveToSan(move, moves));
609 - | .}
610 - | ._moves({ legal: legal = !0, piece: piece, square: square }·=·{} )·{
611 - | .    const forSquare = square ? square.toLowerCase()·: void 0,
612 - | .        forPiece = piece ? piece.toLowerCase(),
613 - | .        moves = [],
614 - | .        us = this._turn,
615 - | .        them = swapColor(us);
616 - | .    let firstSquare = 0x88.a8,
617 - | .        lastSquare = 0x88.h1,
618 - | .        singleSquare = !1;
619 - | .    if (forSquare) {
620 - | .        if (!(forSquare in 0x88)) return [];
621 - | .        ((firstSquare = lastSquare = 0x88[forSquare]), (singleSquare = !0));
622 - | .    }
623 - | .    for (let from = firstSquare; from ≤ lastSquare; from++)·{
624 - | .        if (136 & from)·{
625 - | .            from += 7;
626 - | .            continue;
627 - | .        }
628 - | .        if (!this._board[from]·|| this._board[from].color === them) continue;
629 - | .        const { type: type } = this._board[from];
630 - | .        let to;
631 - | .        if ('p' === type)·{
632 - | .            if (forPiece && forPiece !== type) continue;
633 - | .            ((to = from + PAWN_OFFSETS[us][0]),
634 - | .            this._board[to]·||
635 - | .            (addMove(moves, us, from, to, 'p'),
636 - | .            (to = from + PAWN_OFFSETS[us][1]),
637 - | .            SECOND_RANK[us] !== rank(from)·||
638 - | .            this._board[to]·||
639 - | .            addMove(moves, us, from, to, 'p', void 0, BITS.BIG_PAWN)));
640 - | .            for (let j = 2; j < 4; j++)
641 - | .                ((to = from + PAWN_OFFSETS[us][j]),
642 - | .                136 & to·||
643 - | .                (this._board[to] ? color === them
644 - | .                ? addMove(moves, us, from, to, 'p', this._board[to].type, BITS.CAPTURE)
645 - | .                : to === this._epSquare &&
646 - | .                addMove(moves, us, from, to, 'p', 'p', BITS.EP_CAPTURE))));
647 - | .        } else {
648 - | .            if (forPiece && forPiece !== type) continue;
649 - | .            for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++)·{
650 - | .                const offset = PIECE_OFFSETS[type][j];
651 - | .                for (to = from; (to += offset), !(136 & to);)·{
652 - | .                    if (this._board[to])·{
653 - | .                        if (this._board[to].color === us) break;
654 - | .                        addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
655 - | .                        break;
656 - | .                    }
657 - | .                    if ((addMove(moves, us, from, to, type), 'n' === type·|| 'k' === type)) break;
658 - | .                }
659 - | .            }

```

```

660 - | .}
661 - | .}
662 - | . if (
663 - | . |(
664 - | . |(void 0 !== forPiece & 'k' !== forPiece) ||
665 - | . |(singleSquare & lastSquare !== this._kings[us])
666 - | . |)
667 - | . |){
668 - | . |if (this._castling[us] & BITS.KSIDE_CASTLE){
669 - | . |const castlingFrom = this._kings[us],
670 - | . |castlingTo = castlingFrom + 2;
671 - | . |this._board[castlingFrom + 1] ||
672 - | . |this._board[castlingTo] ||
673 - | . |this._attacked(them, this._kings[us]) ||
674 - | . |this._attacked(them, castlingFrom + 1) ||
675 - | . |this._attacked(them, castlingTo) ||
676 - | . |addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.KSIDE_CASTLE);
677 - | . |}
678 - | . |if (this._castling[us] & BITS.QSIDE_CASTLE){
679 - | . |const castlingFrom = this._kings[us],
680 - | . |castlingTo = castlingFrom - 2;
681 - | . |this._board[castlingFrom - 1] ||
682 - | . |this._board[castlingFrom - 2] ||
683 - | . |this._board[castlingFrom - 3] ||
684 - | . |this._attacked(them, this._kings[us]) ||
685 - | . |this._attacked(them, castlingFrom - 1) ||
686 - | . |this._attacked(them, castlingTo) ||
687 - | . |addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.QSIDE_CASTLE);
688 - | . |}
689 - | . |}
690 - | . |if (!legal || -1 === this._kings[us]) return moves;
691 - | . |const legalMoves = [];
692 - | . |for (let i = 0, len = moves.length; i < len; i++)
693 - | . | (this._makeMove(moves[i]),
694 - | . | this._isKingAttacked(us) || legalMoves.push(moves[i]),
695 - | . | this._undoMove());
696 - | . |return legalMoves;
697 - | . |}
698 - | . |move(move){
699 - | . |let moveObj = null;
700 - | . |const moves = this._moves();
701 - | . |for (let i = 0, len = moves.length; i < len; i++)
702 - | . |if (
703 - | . |move.from === algebraic(moves[i].from) &
704 - | . |move.to === algebraic(moves[i].to) &
705 - | . |(!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
706 - | . |){
707 - | . |moveObj = moves[i];
708 - | . |break;
709 - | . |}
710 - | . |if (!moveObj) throw new Error(`Invalid move: ${JSON.stringify(move)}`);
711 - | . |const prettyMove = new Move(this, moveObj);
712 - | . |return (this._makeMove(moveObj), this._incPositionCount(), prettyMove);
713 - | . |}
714 - | . |_push(move){
715 - | . |this._history.push({
716 - | . |move: move,
717 - | . |kings: {b: this._kings.b, w: this._kings.w},
718 - | . |turn: this._turn,
719 - | . |castling: {b: this._castling.b, w: this._castling.w},
720 - | . |epSquare: this._epSquare,
721 - | . |halfMoves: this._halfMoves,
722 - | . |moveNumber: this._moveNumber,
723 - | . |});
724 - | . |}
725 - | . |_movePiece(from, to){
726 - | . |((this._hash ^= this._pieceKey(from)),
727 - | . |(this._board[to] = this._board[from]),
728 - | . |delete this._board[from],
729 - | . |(this._hash ^= this._pieceKey(to)));
730 - | . |}

```

```

731 - | | | _makeMove(move) {
732 - | | |     const us = this._turn,
733 - | | |     them = swapColor(us);
734 - | | |     if ((this._push(move), move.flags & BITS.NULL_MOVE))
735 - | | |         return (
736 - | | |             'b' === us && this._moveNumber++,
737 - | | |             this._halfMoves++,
738 - | | |             (this._turn = them),
739 - | | |             void (this._epSquare = -1)
740 - | | |         );
741 - | | |     if (
742 - | | |         ((this._hash ^ this._epKey()),
743 - | | |         (this._hash ^ this._castlingKey()),
744 - | | |         move.captured && (this._hash ^ this._pieceKey(move.to)),
745 - | | |         this._movePiece(move.from, move.to),
746 - | | |         move.flags & BITS.EP_CAPTURE &&
747 - | | |         ('b' === this._turn ? this._clear(move.to - 16) : this._clear(move.to + 16)),
748 - | | |         move.promotion &&
749 - | | |         (this._clear(move.to), this._set(move.to, { type: move.promotion, color: us })),
750 - | | |         'k' === this._board[move.to].type)
751 - | | |     ) {
752 - | | |         if (((this._kings[us] = move.to), move.flags & BITS.KSIDE_CASTLE)) {
753 - | | |             const castlingTo = move.to - 1,
754 - | | |                 castlingFrom = move.to + 1;
755 - | | |             this._movePiece(castlingFrom, castlingTo);
756 - | | |         } else if (move.flags & BITS.QSIDE_CASTLE) {
757 - | | |             const castlingTo = move.to + 1,
758 - | | |                 castlingFrom = move.to - 2;
759 - | | |             this._movePiece(castlingFrom, castlingTo);
760 - | | |         }
761 - | | |         this._castling[us] = 0;
762 - | | |     }
763 - | | |     if (this._castling[us])
764 - | | |         for (let i = 0, len = ROOKS[us].length; i < len; i++)
765 - | | |             if (move.from === ROOKS[us][i].square && this._castling[us] & ROOKS[us][i].flag) {
766 - | | |                 this._castling[us] ^= ROOKS[us][i].flag;
767 - | | |                 break;
768 - | | |             }
769 - | | |     if (this._castling[them])
770 - | | |         for (let i = 0, len = ROOKS[them].length; i < len; i++)
771 - | | |             if (move.to === ROOKS[them][i].square && this._castling[them] & ROOKS[them][i].flag) {
772 - | | |                 this._castling[them] ^= ROOKS[them][i].flag;
773 - | | |                 break;
774 - | | |             }
775 - | | |     if (((this._hash ^ this._castlingKey()), move.flags & BITS.BIG_PAWN)) {
776 - | | |         let epSquare;
777 - | | |         ((epSquare = 'b' === us ? move.to - 16 : move.to + 16),
778 - | | |         ((move.to - 1) & 136 ||
779 - | | |         'p' !== this._board[move.to - 1].type ||
780 - | | |         this._board[move.to - 1].color !== them) &&
781 - | | |         ((move.to + 1) & 136 ||
782 - | | |         'p' !== this._board[move.to + 1].type ||
783 - | | |         this._board[move.to + 1].color !== them)
784 - | | |         ? (this._epSquare = -1)
785 - | | |         : ((this._epSquare = epSquare), (this._hash ^ this._epKey())));
786 - | | |     } else this._epSquare = -1;
787 - | | |     ('p' === move.piece || move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)
788 - | | |     ? (this._halfMoves = 0)
789 - | | |     : this._halfMoves++,
790 - | | |     'b' === us && this._moveNumber++,
791 - | | |     (this._turn = them),
792 - | | |     (this._hash ^ SIDE_KEY));
793 - | | | }
794 - | | | _undo() {
795 - | | |     const hash = this._hash,
796 - | | |     move = this._undoMove();
797 - | | |     if (move) {
798 - | | |         const prettyMove = new Move(this, move);
799 - | | |         return (this._decPositionCount(hash), prettyMove);
800 - | | |     }
801 - | | |     return null;

```

```

802 - | .}
803 - | .undoMove()-{
804 - | .const old = this._history.pop();
805 - | .if (void 0 === old) return null;
806 - | .((this._hash ^ this._epKey()), (this._hash ^ this._castlingKey()));
807 - | .const move = old.move;
808 - | .((this._kings = old.kings),
809 - | .(this._turn = old.turn),
810 - | .(this._castling = old.castling),
811 - | .(this._epSquare = old.epSquare),
812 - | .(this._halfMoves = old.halfMoves),
813 - | .(this._moveNumber = old.moveNumber),
814 - | .(this._hash ^ this._epKey()),
815 - | .(this._hash ^ this._castlingKey()),
816 - | .(this._hash ^ SIDE_KEY));
817 - | .const us = this._turn,
818 - | .them = swapColor(us);
819 - | .if (move.flags & BITS.NULL_MOVE) return move;
820 - | .if (
821 - | .(this._movePiece(move.to, move.from),
822 - | .move.piece &
823 - | .(this._clear(move.from), this._set(move.from, { type: move.piece, color: us })),
824 - | .move.captured)
825 - | .)
826 - | .if (move.flags & BITS.EP_CAPTURE) {
827 - | .let index;
828 - | .((index = 'b' === us ? move.to - 16 : move.to + 16),
829 - | .this._set(index, { type: 'p', color: them }));
830 - | .} else this._set(move.to, { type: move.captured, color: them });
831 - | .if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
832 - | .let castlingTo, castlingFrom;
833 - | .(move.flags & BITS.KSIDE_CASTLE
834 - | .? ((castlingTo = move.to + 1), (castlingFrom = move.to - 1))
835 - | .: ((castlingTo = move.to - 2), (castlingFrom = move.to + 1)),
836 - | .this._movePiece(castlingFrom, castlingTo));
837 - | .}
838 - | .return move;
839 - | .}
840 - | .moveToSan(move, moves)-{
841 - | .let output = '';
842 - | .if (move.flags & BITS.KSIDE_CASTLE) output = '0-0';
843 - | .else if (move.flags & BITS.QSIDE_CASTLE) output = '0-0-0';
844 - | .else {
845 - | .if ('p' !== move.piece) {
846 - | .const disambiguator = (function getDisambiguator(move, moves) {
847 - | .const from = move.from,
848 - | .to = move.to,
849 - | .piece = move.piece;
850 - | .let ambiguities = 0,
851 - | .sameRank = 0,
852 - | .sameFile = 0;
853 - | .for (let i = 0, len = moves.length; i < len; i++) {
854 - | .const ambigFrom = moves[i].from,
855 - | .ambigTo = moves[i].to;
856 - | .piece === moves[i].piece &
857 - | .from === ambigFrom &
858 - | .to === ambigTo &
859 - | .(ambiguities++,
860 - | .rank(from) === rank(ambigFrom) & sameRank++,
861 - | .file(from) === file(ambigFrom) & sameFile++);
862 - | .}
863 - | .return ambiguities > 0
864 - | .? sameRank > 0 & sameFile > 0
865 - | .? algebraic(from)
866 - | .: sameFile > 0
867 - | .? algebraic(from).charAt(1)
868 - | .: algebraic(from).charAt(0)
869 - | .: '';
870 - | .})(move, moves);
871 - | .output += move.piece.toUpperCase() + disambiguator;
872 - | .}

```

```

873 - | | | | | (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE) &&
874 - | | | | | ('p' === move.piece && (output += algebraic(move.from)[0]), (output += 'x')),
875 - | | | | | (output += algebraic(move.to)),
876 - | | | | | move.promotion && (output += '=' + move.promotion.toUpperCase()));
877 - | | | | | }
878 - | | | | | return (
879 - | | | | | this._makeMove(move),
880 - | | | | | this.isCheck() && (this.isCheckmate() ? (output += '#') : (output += '+')),
881 - | | | | | this._undoMove(),
882 - | | | | | output
883 - | | | | | );
884 - | | | | | }
885 - | | | | | turn() {
886 - | | | | | return this._turn;
887 - | | | | | }
888 - | | | | | board() {
889 - | | | | | const output = [];
890 - | | | | | let row = [];
891 - | | | | | for (let i = 0x88.a8; i <= 0x88.h1; i++)
892 - | | | | | (null === this._board[i]
893 - | | | | | ? row.push(null)
894 - | | | | | : row.push({
895 - | | | | | square: algebraic(i),
896 - | | | | | type: this._board[i].type,
897 - | | | | | color: this._board[i].color,
898 - | | | | | })),
899 - | | | | | (i + 1) & 136 && (output.push(row), (row = []), (i += 8)));
900 - | | | | | return output;
901 - | | | | | }
902 - | | | | | squareColor(square) {
903 - | | | | | if (square in 0x88) {
904 - | | | | | const sq = 0x88[square];
905 - | | | | | return (rank(sq) + file(sq)) % 2 === 0 ? 'light' : 'dark';
906 - | | | | | }
907 - | | | | | return null;
908 - | | | | | }
909 - | | | | | history({ verbose: verbose = !1 } = {}) {
910 - | | | | | const reversedHistory = [],
911 - | | | | | moveHistory = [];
912 - | | | | | for (; this._history.length > 0; ) reversedHistory.push(this._undoMove());
913 - | | | | | for (;;) {
914 - | | | | | const move = reversedHistory.pop();
915 - | | | | | if (!move) break;
916 - | | | | | (verbose
917 - | | | | | ? moveHistory.push(new Move(this, move))
918 - | | | | | : moveHistory.push(this._moveToSan(move, this._moves()))),
919 - | | | | | this._makeMove(move));
920 - | | | | | }
921 - | | | | | return moveHistory;
922 - | | | | | }
923 - | | | | | _getPositionCount(hash) {
924 - | | | | | return this._positionCount.get(hash) ?? 0;
925 - | | | | | }
926 - | | | | | _incPositionCount() {
927 - | | | | | this._positionCount.set(this._hash, (this._positionCount.get(this._hash) ?? 0) + 1);
928 - | | | | | }
929 - | | | | | _decPositionCount(hash) {
930 - | | | | | const currentCount = this._positionCount.get(hash) ?? 0;
931 - | | | | | 1 === currentCount
932 - | | | | | ? this._positionCount.delete(hash)
933 - | | | | | : this._positionCount.set(hash, currentCount - 1);
934 - | | | | | }
935 - | | | | | moveNumber() {
936 - | | | | | return this._moveNumber;
937 - | | | | | }
938 - | | | | | }
939 - | | | | | const PIECE_VALUES = { p: 100, n: 320, b: 330, r: 500, q: 900, k: 0 };
940 - | | | | | function searchRoot(chess, depth, ctx, previousBestMove) {
941 - | | | | | const candidates = [],
942 - | | | | | legalMoves = orderMoves(chess.moves({ verbose: !0 }), previousBestMove);
943 - | | | | | for (const move of legalMoves) {

```

```

944 - | | | | (checkTime(ctx), chess.move(toMoveCommand(move)));
945 - | | | | const score = negamax(chess, depth - 1, -1e7, 1e7, 1, ctx);
946 - | | | | (chess.undo(), candidates.push({ move: toMoveCommand(move), san: move.san, score: score }));
947 - | | | | }
948 - | | | | if (0 === candidates.length) throw new SearchTimeout();
949 - | | | | const principal = (function selectBestRootMove(candidates) {
950 - | | | |   return candidates.reduce((currentBest, candidate) =>
951 - | | | |     candidate.score > currentBest.score ? candidate : currentBest,
952 - | | | |     {});
953 - | | | | })(candidates),
954 - | | | | selection = (function selectRootMove(candidates, settings, best) {
955 - | | | |   const level = settings.randomness / 30,
956 - | | | |   windowCp = 20 + 150 * level,
957 - | | | |   floorScore = best.score - windowCp,
958 - | | | |   ticketPower = 4 - 1.5 * level;
959 - | | | |   if (
960 - | | | |     (function shouldForceBestMove(settings, best) {
961 - | | | |       return (
962 - | | | |         0 === settings.randomness ||
963 - | | | |         (function isMateProtectedScore(score) {
964 - | | | |           return Math.abs(score) >= 99e4;
965 - | | | |         })(best.score) ||
966 - | | | |         ('hard' === settings.preset && best.score < 0)
967 - | | | |       );
968 - | | | |     })(settings, best)
969 - | | | |   )
970 - | | | |   return {
971 - | | | |     selected: best,
972 - | | | |     debug: createDeterministicDebug(
973 - | | | |       candidates,
974 - | | | |       best,
975 - | | | |       best,
976 - | | | |       windowCp,
977 - | | | |       floorScore,
978 - | | | |       ticketPower,
979 - | | | |     ),
980 - | | | |   };
981 - | | | |   const weightedCandidates = candidates
982 - | | | |     .map((candidate) => ({
983 - | | | |       candidate: candidate,
984 - | | | |       tickets: (Math.max(0, candidate.score - floorScore) / windowCp) ** ticketPower,
985 - | | | |     })))
986 - | | | |     .filter(({ tickets: tickets }) => tickets > 0),
987 - | | | |   totalTickets = weightedCandidates.reduce(
988 - | | | |     (total, { tickets: tickets }) => total + tickets,
989 - | | | |     0,
990 - | | | |   );
991 - | | | |   if (totalTickets <= 0)
992 - | | | |     return {
993 - | | | |       selected: best,
994 - | | | |       debug: createDeterministicDebug(
995 - | | | |         candidates,
996 - | | | |         best,
997 - | | | |         best,
998 - | | | |         windowCp,
999 - | | | |         floorScore,
1000 - | | | |         ticketPower,
1001 - | | | |       ),
1002 - | | | |     };
1003 - | | | |   let draw = Math.random() * totalTickets,
1004 - | | | |   selected = best;
1005 - | | | |   for (const { candidate: candidate, tickets: tickets } of weightedCandidates)
1006 - | | | |     if (((draw -= tickets), draw <= 0)) {
1007 - | | | |       selected = candidate;
1008 - | | | |       break;
1009 - | | | |     }
1010 - | | | |   return {
1011 - | | | |     selected: selected,
1012 - | | | |     debug: createWeightedDebug(
1013 - | | | |       candidates,
1014 - | | | |       best,

```

```

1015 - |         selected,
1016 - |         weightedCandidates,
1017 - |         windowCp,
1018 - |         floorScore,
1019 - |         ticketPower,
1020 - |         totalTickets,
1021 - |     ),
1022 - |     });
1023 - |     })(candidates, ctx.settings, principal);
1024 - |     return { selected: selection.selected, principal: principal, debug: selection.debug };
1025 - | }
1026 - |
1027 - | function shouldStopAfterSoftLimit(
1028 - |     chess,
1029 - |     rootMoves,
1030 - |     result,
1031 - |     previousPrincipal,
1032 - |     depth,
1033 - |     elapsedMs,
1034 - |     settings,
1035 - | ) {
1036 - |     if (
1037 - |         'hard' !== settings.preset ||
1038 - |         void 0 === settings.softTimeLimitMs ||
1039 - |         elapsedMs < settings.softTimeLimitMs ||
1040 - |         depth < 2
1041 - |     )
1042 - |         return !1;
1043 - |     if (chess.isCheck() || result.principal.score < 0) return !1;
1044 - |     if (
1045 - |         null === previousPrincipal ||
1046 - |         !isSameMoveCommand(previousPrincipal.move, result.principal.move) ||
1047 - |         Math.abs(previousPrincipal.score - result.principal.score) > 90
1048 - |     )
1049 - |         return !1;
1050 - |     const scoreGap = (function rootScoreGap(debug) {
1051 - |         const [bestCandidate, secondCandidate] = debug.candidates;
1052 - |         return bestCandidate && secondCandidate ? bestCandidate.score - secondCandidate.score : 1e7;
1053 - |     })(result.debug);
1054 - |     return !(
1055 - |         scoreGap < 120 ||
1056 - |         ((function tacticalMoveRatio(moves) {
1057 - |             return 0 === moves.length ? 0 : moves.filter(isTacticalMove).length / moves.length;
1058 - |         })(rootMoves) >= 0.35 &&
1059 - |         scoreGap < 220)
1060 - |     );
1061 - | }
1062 - |
1063 - | function negamax(chess, depth, alpha, beta, ply, ctx) {
1064 - |     if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
1065 - |     if (depth <= 0)
1066 - |         return ctx.settings.quiescence
1067 - |             ? quiescence(chess, alpha, beta, ply, ctx)
1068 - |             : evaluatePosition(chess);
1069 - |     let bestScore = -1e7,
1070 - |         currentAlpha = alpha;
1071 - |     const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null);
1072 - |     for (const move of legalMoves) {
1073 - |         chess.move(toMoveCommand(move));
1074 - |         const score = -negamax(chess, depth - 1, -beta, -currentAlpha, ply + 1, ctx);
1075 - |         if (
1076 - |             (chess.undo(),
1077 - |             (bestScore = Math.max(bestScore, score)),
1078 - |             (currentAlpha = Math.max(currentAlpha, score)),
1079 - |             currentAlpha >= beta)
1080 - |         )
1081 - |             break;
1082 - |     }
1083 - |     return bestScore;
1084 - | }
1085 - |
1086 - | function quiescence(chess, alpha, beta, ply, ctx) {
1087 - |     if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
1088 - |     let currentAlpha = alpha;

```



```

1086 - | | | | | const standPat = evaluatePosition(chess);
1087 - | | | | | if (standPat ≥ beta) return beta;
1088 - | | | | | if (((currentAlpha = Math.max(currentAlpha, standPat)), ply ≥ 6)) return currentAlpha;
1089 - | | | | | const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null),
1090 - | | | | | tacticalMoves = chess.isCheck() ? legalMoves : legalMoves.filter(isTacticalMove);
1091 - | | | | | for (const move of tacticalMoves) {
1092 - | | | | | | | chess.move(toMoveCommand(move));
1093 - | | | | | | | const score = -quiescence(chess, -beta, -currentAlpha, ply + 1, ctx);
1094 - | | | | | | | if ((chess.undo(), score ≥ beta)) return beta;
1095 - | | | | | | | currentAlpha = Math.max(currentAlpha, score);
1096 - | | | | | }
1097 - | | | | | return currentAlpha;
1098 - | | | | | }
1099 - | | | | | function evaluatePosition(chess) {
1100 - | | | | | | | const board = chess.board(),
1101 - | | | | | | | pawns = [],
1102 - | | | | | | | bishops = { w: 0, b: 0 },
1103 - | | | | | | | kings = {};
1104 - | | | | | | | let whiteScore = 0,
1105 - | | | | | | | blackScore = 0,
1106 - | | | | | | | nonPawnMaterial = 0;
1107 - | | | | | | | for (const row of board)
1108 - | | | | | | | | | for (const pieceOnSquare of row) {
1109 - | | | | | | | | | | | if (null === pieceOnSquare) continue;
1110 - | | | | | | | | | | | const { color: color, type: type, square: square } = pieceOnSquare,
1111 - | | | | | | | | | | | pieceScore = PIECE_VALUES[type] + pieceSquareScore(type, color, square);
1112 - | | | | | | | | | | | ('w' === color ? (whiteScore += pieceScore) : (blackScore += pieceScore),
1113 - | | | | | | | | | | | | | 'p' === type
1114 - | | | | | | | | | | | | | ? pawns.push({ color: color, file: fileIndex(square), rank: rankNumber(square) })
1115 - | | | | | | | | | | | | | : 'k' !== type && (nonPawnMaterial += PIECE_VALUES[type]),
1116 - | | | | | | | | | | | | | 'b' === type && (bishops[color] += 1),
1117 - | | | | | | | | | | | | | 'k' === type && (kings[color] = square));
1118 - | | | | | | | | | | | }
1119 - | | | | | | | | ((whiteScore += bishops.w ≥ 2 ? 35 : 0),
1120 - | | | | | | | | (blackScore += bishops.b ≥ 2 ? 35 : 0),
1121 - | | | | | | | | (whiteScore += pawnStructureScore(pawns, 'w')),
1122 - | | | | | | | | (blackScore += pawnStructureScore(pawns, 'b')),
1123 - | | | | | | | | (whiteScore += developmentScore(board, 'w')),
1124 - | | | | | | | | (blackScore += developmentScore(board, 'b')),
1125 - | | | | | | | | (whiteScore += kingSafetyScore(kings.w, pawns, 'w', nonPawnMaterial)),
1126 - | | | | | | | | (blackScore += kingSafetyScore(kings.b, pawns, 'b', nonPawnMaterial)));
1127 - | | | | | | | | let score = (whiteScore - blackScore) * ('w' === chess.turn() ? 1 : -1);
1128 - | | | | | | | | return ((score += 2 * chess.moves().length), chess.isCheck() && (score -= 45), score);
1129 - | | | | | | }
1130 - | | | | | function pieceSquareScore(type, color, square) {
1131 - | | | | | | | const file2 = fileIndex(square),
1132 - | | | | | | | relativeRank = 'w' === color ? rankNumber(square) : 9 - rankNumber(square),
1133 - | | | | | | | centerDistance = Math.abs(file2 - 3.5) + Math.abs(rankNumber(square) - 4.5);
1134 - | | | | | | | switch (type) {
1135 - | | | | | | | | case 'p':
1136 - | | | | | | | | | return 7 * relativeRank + (file2 ≥ 2 && file2 ≤ 5 ? 8 : 0);
1137 - | | | | | | | | case 'n':
1138 - | | | | | | | | | return 42 - 12 * centerDistance - (1 === relativeRank ? 12 : 0);
1139 - | | | | | | | | case 'b':
1140 - | | | | | | | | | return 30 - 7 * centerDistance;
1141 - | | | | | | | | case 'r':
1142 - | | | | | | | | | return (relativeRank ≥ 7 ? 18 : 0) + (0 === file2 || 7 === file2 ? 4 : 0);
1143 - | | | | | | | | case 'q':
1144 - | | | | | | | | | return 14 - 3 * centerDistance;
1145 - | | | | | | | | case 'k':
1146 - | | | | | | | | | return relativeRank ≥ 7 ? 18 : 0;
1147 - | | | | | | | }
1148 - | | | | | | }
1149 - | | | | | function pawnStructureScore(pawns, color) {
1150 - | | | | | | | const ownPawns = pawns.filter((pawn) => pawn.color === color),
1151 - | | | | | | | fileCounts = Array.from({ length: 8 }, () => 0);
1152 - | | | | | | | let score = 0;
1153 - | | | | | | | for (const pawn of ownPawns) fileCounts[pawn.file] += 1;
1154 - | | | | | | | for (const pawn of ownPawns) {
1155 - | | | | | | | | const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
1156 - | | | | | | | | (fileCounts[pawn.file] > 1 && (score -= 14),

```



```

1157 - | | | | 0 === (fileCounts[pawn.file - 1] ?? 0) &&
1158 - | | | | 0 === (fileCounts[pawn.file + 1] ?? 0) &&
1159 - | | | | (score -= 10),
1160 - | | | | isPassedPawn(pawn, pawns) && (score += (relativeRank - 1) * (relativeRank - 1) * 9));
1161 - | | | }
1162 - | | return score;
1163 - | }
1164 - | function isPassedPawn(pawn, pawns) {
1165 - | | const enemyColor = (function oppositeColor(color) {
1166 - | | | return 'w' === color ? 'b' : 'w';
1167 - | | })(pawn.color);
1168 - | | return !pawns.some(
1169 - | | | (candidate) =>
1170 - | | | | ! (candidate.color !== enemyColor || Math.abs(candidate.file - pawn.file) > 1) &&
1171 - | | | | ('w' === pawn.color ? candidate.rank > pawn.rank : candidate.rank < pawn.rank),
1172 - | | | );
1173 - | | }
1174 - | function developmentScore(board, color) {
1175 - | | const homeRank = 'w' === color ? '1' : '8',
1176 - | | knightHomeSquares = 'w' === color ? new Set(['b1', 'g1']) : new Set(['b8', 'g8']),
1177 - | | bishopHomeSquares = 'w' === color ? new Set(['c1', 'f1']) : new Set(['c8', 'f8']);
1178 - | | let score = 0;
1179 - | | for (const row of board)
1180 - | | | for (const piece of row)
1181 - | | | | null !== piece &&
1182 - | | | | piece.color === color &&
1183 - | | | | ('n' !== piece.type || knightHomeSquares.has(piece.square) || (score += 18),
1184 - | | | | 'b' !== piece.type || bishopHomeSquares.has(piece.square) || (score += 16),
1185 - | | | | 'q' === piece.type &&
1186 - | | | | piece.square.endsWith(homeRank) &&
1187 - | | | | hasMovedMinorPieces(board, color) &&
1188 - | | | | (score -= 12));
1189 - | | return score;
1190 - | }
1191 - | function hasMovedMinorPieces(board, color) {
1192 - | | const homeSquares =
1193 - | | | 'w' === color ? new Set(['b1', 'c1', 'f1', 'g1']) : new Set(['b8', 'c8', 'f8', 'g8']);
1194 - | | for (const row of board)
1195 - | | | for (const piece of row)
1196 - | | | | if (
1197 - | | | | | null !== piece &&
1198 - | | | | | piece.color === color &&
1199 - | | | | | ('n' === piece.type || 'b' === piece.type) &&
1200 - | | | | | homeSquares.has(piece.square)
1201 - | | | | )
1202 - | | | | return !1;
1203 - | | return !0;
1204 - | }
1205 - | function kingSafetyScore(kingSquare, pawns, color, nonPawnMaterial) {
1206 - | | if (!kingSquare || nonPawnMaterial < 1800) return 0;
1207 - | | const kingFile = fileIndex(kingSquare),
1208 - | | kingRank = rankNumber(kingSquare),
1209 - | | shieldRank = 'w' === color ? kingRank + 1 : kingRank - 1;
1210 - | | let score = 2 === kingFile || 6 === kingFile ? 28 : -18;
1211 - | | for (let file2 = kingFile - 1; file2 <= kingFile + 1; file2 += 1)
1212 - | | | file2 < 0 ||
1213 - | | | file2 >= 8 ||
1214 - | | | (score += pawns.some(
1215 - | | | | (pawn) => pawn.color === color && pawn.file === file2 && pawn.rank === shieldRank,
1216 - | | | ))
1217 - | | | ? 14
1218 - | | | : -10);
1219 - | | return score;
1220 - | }
1221 - | function terminalScore(chess, ply) {
1222 - | | return chess.isCheckmate() ? -1e6 + ply : chess.isDraw() ? 0 : evaluatePosition(chess);
1223 - | }
1224 - | function orderMoves(moves, preferredMove) {
1225 - | | return [...moves].sort(
1226 - | | | (left, right) =>
1227 - | | | | moveOrderingScore(right, preferredMove) - moveOrderingScore(left, preferredMove),

```

```

1228 - | | |);
1229 - | | }
1230 - | | function moveOrderingScore(move, preferredMove) {
1231 - | | | let score = 0;
1232 - | | | return (
1233 - | | | | preferredMove &&
1234 - | | | | (function isSameMove(move, command) {
1235 - | | | | | return (
1236 - | | | | | | move.from === command.from &&
1237 - | | | | | | move.to === command.to &&
1238 - | | | | | | (move.promotion ?? void 0) === (command.promotion ?? void 0)
1239 - | | | | | );
1240 - | | | | })(move, preferredMove) &&
1241 - | | | | (score += 2e6),
1242 - | | | | move.san.includes('#') ? (score += 15e5) : move.san.includes('+') && (score += 16e3),
1243 - | | | | move.isCapture() &&
1244 - | | | | (score +=
1245 - | | | | | 12 * (move.captured ? PIECE_VALUES[move.captured] : PIECE_VALUES.p) -
1246 - | | | | | PIECE_VALUES[move.piece]),
1247 - | | | | | move.isPromotion() && move.promotion && (score += PIECE_VALUES[move.promotion] + 8e3),
1248 - | | | | | (move.isKingsideCastle() || move.isQueensideCastle()) && (score += 1500),
1249 - | | | | | (score += 2 * pieceSquareScore(move.piece, move.color, move.to)),
1250 - | | | | | score
1251 - | | | | );
1252 - | | | }
1253 - | | function checkTime(ctx) {
1254 - | | | if (((ctx.nodes += 1), ctx.nodes % 2048 !== 0)) return;
1255 - | | | const now = Date.now();
1256 - | | | if (
1257 - | | | | (now - ctx.lastProgressAt ≥ 100 &&
1258 - | | | | | publishSearchProgress(ctx, ctx.progressState, now - ctx.startTime, now),
1259 - | | | | | now ≥ ctx.deadline)
1260 - | | | | )
1261 - | | | | throw new SearchTimeout();
1262 - | | | }
1263 - | | function publishSearchProgress(
1264 - | | | ctx,
1265 - | | | progressState,
1266 - | | | elapsedMs = Date.now() - ctx.startTime,
1267 - | | | progressAt = Date.now(),
1268 - | | | ) {
1269 - | | | | ((ctx.progressState = progressState),
1270 - | | | | (ctx.lastProgressAt = progressAt),
1271 - | | | | | ctx.onProgress?.({
1272 - | | | | | | kind: 'progress',
1273 - | | | | | | id: ctx.requestId,
1274 - | | | | | | move: progressState.move,
1275 - | | | | | | depthReached: progressState.depthReached,
1276 - | | | | | | nodes: ctx.nodes,
1277 - | | | | | | elapsedMs: elapsedMs,
1278 - | | | | | | score: progressState.score,
1279 - | | | | | | debug: progressState.debug,
1280 - | | | | | })));
1281 - | | | }
1282 - | | function isTacticalMove(move) {
1283 - | | | return (
1284 - | | | | move.isCapture() || move.isPromotion() || move.san.includes('+') || move.san.includes('#')
1285 - | | | | );
1286 - | | | }
1287 - | | function createDeterministicDebug(
1288 - | | | candidates,
1289 - | | | principal,
1290 - | | | selected,
1291 - | | | windowCp,
1292 - | | | floorScore,
1293 - | | | ticketPower,
1294 - | | | ) {
1295 - | | | return {
1296 - | | | | candidates: sortCandidatesByScore(candidates).map((candidate) => ({
1297 - | | | | | move: candidate.move,
1298 - | | | | | san: candidate.san,

```

```

1299 - | | | | | score: candidate.score,
1300 - | | | | | tickets: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
1301 - | | | | | probability: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
1302 - | | | | | selected: isSameMoveCommand(candidate.move, selected.move),
1303 - | | | | | principal: isSameMoveCommand(candidate.move, principal.move),
1304 - | | | | | })),
1305 - | | | | | bestScore: principal.score,
1306 - | | | | | windowCp: windowCp,
1307 - | | | | | floorScore: floorScore,
1308 - | | | | | ticketPower: ticketPower,
1309 - | | | | | totalTickets: 1,
1310 - | | | | | };
1311 - | | | | | }
1312 - | | | | | function createWeightedDebug(
1313 - | | | | | candidates,
1314 - | | | | | principal,
1315 - | | | | | selected,
1316 - | | | | | weightedCandidates,
1317 - | | | | | windowCp,
1318 - | | | | | floorScore,
1319 - | | | | | ticketPower,
1320 - | | | | | totalTickets,
1321 - | | | | | ) {
1322 - | | | | | return {
1323 - | | | | | candidates: sortCandidatesByScore(candidates).map((candidate) => {
1324 - | | | | |   const weightedCandidate = weightedCandidates.find(({ candidate: weighted }) =>
1325 - | | | | |     isSameMoveCommand(weighted.move, candidate.move),
1326 - | | | | |   ),
1327 - | | | | |   tickets = weightedCandidate?.tickets ?? 0;
1328 - | | | | |   return {
1329 - | | | | |     move: candidate.move,
1330 - | | | | |     san: candidate.san,
1331 - | | | | |     score: candidate.score,
1332 - | | | | |     tickets: tickets,
1333 - | | | | |     probability: totalTickets > 0 ? tickets / totalTickets : 0,
1334 - | | | | |     selected: isSameMoveCommand(candidate.move, selected.move),
1335 - | | | | |     principal: isSameMoveCommand(candidate.move, principal.move),
1336 - | | | | |   };
1337 - | | | | | })),
1338 - | | | | | bestScore: principal.score,
1339 - | | | | | windowCp: windowCp,
1340 - | | | | | floorScore: floorScore,
1341 - | | | | | ticketPower: ticketPower,
1342 - | | | | | totalTickets: totalTickets,
1343 - | | | | | };
1344 - | | | | | }
1345 - | | | | | function sortCandidatesByScore(candidates) {
1346 - | | | | | return [...candidates].sort((left, right) => right.score - left.score);
1347 - | | | | | }
1348 - | | | | | function toMoveCommand(move) {
1349 - | | | | | return move.promotion
1350 - | | | | |   ? { from: move.from, to: move.to, promotion: move.promotion }
1351 - | | | | |   : { from: move.from, to: move.to };
1352 - | | | | | }
1353 - | | | | | function isSameMoveCommand(left, right) {
1354 - | | | | | return (
1355 - | | | | |   left.from === right.from &&
1356 - | | | | |   left.to === right.to &&
1357 - | | | | |   (left.promotion ?? void 0) === (right.promotion ?? void 0)
1358 - | | | | | );
1359 - | | | | | }
1360 - | | | | | function fileIndex(square) {
1361 - | | | | | return square.charCodeAt(0) - 97;
1362 - | | | | | }
1363 - | | | | | function rankNumber(square) {
1364 - | | | | | return Number(square[1]);
1365 - | | | | | }
1366 - | | | | | function clamp(value, min, max) {
1367 - | | | | | return Math.min(max, Math.max(min, value));
1368 - | | | | | }
1369 - | | | | | class SearchTimeout extends Error {}

```

```

1370 - | const workerScope = globalThis;
1371 - | workerScope.addEventListener('message', (event) => {
1372 - |   const response = (function searchBestMove(request, onProgress) {
1373 - |     const chess = new Chess(request.fen),
1374 - |     settings = (function normalizeAiSettings(settings) {
1375 - |       const timeLimitMs = clamp(Math.round(settings.timeLimitMs), 500, 1e4),
1376 - |       softTimeLimitMs =
1377 - |         void 0 === settings.softTimeLimitMs
1378 - |         ? void 0
1379 - |         : Math.min(timeLimitMs, clamp(Math.round(settings.softTimeLimitMs), 500, 1e4));
1380 - |     return {
1381 - |       preset: settings.preset,
1382 - |       maxDepth: ((depth = settings.maxDepth), clamp(Math.round(depth), 1, 5)),
1383 - |       timeLimitMs: timeLimitMs,
1384 - |       ... (void 0 === softTimeLimitMs ? {} : { softTimeLimitMs: softTimeLimitMs }),
1385 - |       randomness: 30,
1386 - |       quiescence: !0,
1387 - |     };
1388 - |     var depth;
1389 - |   })(request.settings),
1390 - |   legalMoves = chess.moves({ verbose: !0 }),
1391 - |   startTime = Date.now(),
1392 - |   ctx = {
1393 - |     requestId: request.id,
1394 - |     settings: settings,
1395 - |     deadline: startTime + settings.timeLimitMs,
1396 - |     startTime: startTime,
1397 - |     nodes: 0,
1398 - |     onProgress: onProgress,
1399 - |     lastProgressAt: startTime,
1400 - |     progressState: { move: null, depthReached: 0, score: 0 },
1401 - |   };
1402 - |   if (0 === legalMoves.length)
1403 - |     return {
1404 - |       id: request.id,
1405 - |       move: null,
1406 - |       depthReached: 0,
1407 - |       nodes: 0,
1408 - |       elapsedMs: 0,
1409 - |       score: terminalScore(chess, 0),
1410 - |     };
1411 - |   let debug,
1412 - |   best = null,
1413 - |   previousBestMove = null,
1414 - |   previousPrincipal = null,
1415 - |   depthReached = 0;
1416 - |   for (let depth = 1; depth ≤ settings.maxDepth; depth += 1) {
1417 - |     try {
1418 - |       const result = searchRoot(chess, depth, ctx, previousBestMove),
1419 - |       elapsedMs = Date.now() - startTime;
1420 - |       if (
1421 - |         ((best = result.selected),
1422 - |         (debug = result.debug),
1423 - |         (previousBestMove = result.principal.move),
1424 - |         (depthReached = depth),
1425 - |         publishSearchProgress(
1426 - |           ctx,
1427 - |           { move: best.move, depthReached: depthReached, score: best.score, debug: debug },
1428 - |           elapsedMs,
1429 - |         ),
1430 - |         shouldStopAfterSoftLimit(
1431 - |           chess,
1432 - |           legalMoves,
1433 - |           result,
1434 - |           previousPrincipal,
1435 - |           depth,
1436 - |           elapsedMs,
1437 - |           settings,
1438 - |         ))
1439 - |       )
1440 - |         break;

```

```

1441 - | | | | | previousPrincipal = result.principal;
1442 - | | | | | } catch (error) {
1443 - | | | | | if (error instanceof SearchTimeout) break;
1444 - | | | | | throw error;
1445 - | | | | | }
1446 - | | | | | if (Date.now() >= ctx.deadline) break;
1447 - | | | | | }
1448 - | | | | | return (
1449 - | | | | | (best ??= (function chooseFallbackMove(chess) {
1450 - | | | | | const [move] = orderMoves(chess.moves({ verbose: !0 }), null);
1451 - | | | | | if (!move) return { move: { from: 'a1', to: 'a1' }, score: terminalScore(chess, 0) };
1452 - | | | | | chess.move(toMoveCommand(move));
1453 - | | | | | const score = -evaluatePosition(chess);
1454 - | | | | | return (chess.undo(), { move: toMoveCommand(move), score: score });
1455 - | | | | | })(chess)),
1456 - | | | | | {
1457 - | | | | | id: request.id,
1458 - | | | | | move: best.move,
1459 - | | | | | depthReached: depthReached,
1460 - | | | | | nodes: ctx.nodes,
1461 - | | | | | elapsedMs: Date.now() - startTime,
1462 - | | | | | score: best.score,
1463 - | | | | | debug: debug,
1464 - | | | | | }
1465 - | | | | | );
1466 - | | | | | })(event.data, (progress) => {
1467 - | | | | | workerScope.postMessage(progress);
1468 - | | | | | });
1469 - | | | | | workerScope.postMessage({ ... response, kind: 'result' });
1470 - | | | | | });
1471 - | | | | | })();

```

==== M style.css (11844 B / 017d5c46ea47 → 14083 B / 3bb92984b4f4) ====

```

1 1 | *,
2 2 | *:before,
3 3 | *:after {
4 4 |   box-sizing: border-box;
5 5 | }
6 6 | html {
7 7 |   min-width: 1280px;
8 + |   min-height: 100%;
9 + |   background: var(--page);
10 10 | color-scheme: light;
11 11 | }
12 12 | body {
13 13 |   min-width: 1280px;
12 - |   min-height: 720px;
14 + |   min-height: 100vh;
15 + |   min-height: 100dvh;
13 16 |   margin: 0;
17 + |   background: var(--page);
14 18 |   font-family:
15 19 |     Inter,
16 20 |     ui-sans-serif,
17 21 |     system-ui,
18 22 |     -apple-system,
19 23 |     BlinkMacSystemFont,
20 24 |     Segoe UI,
21 25 |     sans-serif;
22 26 |   text-rendering: geometricPrecision;
23 27 | }
24 28 | button {
25 29 |   font: inherit;
26 30 | }
27 31 | button:not(:disabled) {
28 32 |   cursor: pointer;
29 33 | }
30 34 | button:disabled {
31 35 |   cursor: not-allowed;
32 36 | }
33 37 | :root {

```

```

34 38 | --app-padding: 28px;
35 - | --page: #f3eee3;
36 - | --ink: #20252b;
37 - | --muted: #66716f;
38 - | --line: #c9c0b1;
39 - | --panel: #fffaf0;
40 - | --button: #263a3c;
41 - | --button-text: #ffffff;
42 - | --accent: #b94b36;
43 - | --light-square: #eadbb7;
44 - | --dark-square: #5f8a65;
45 - | --last-move: #e0b84b;
46 - | --legal: #264f8f;
47 - | --capture: #b83232;
48 - | --shadow: 0 18px 42px rgb(36 31 24 / 18%);
39 + | --viewport-height: 100vh;
40 + | --page: oklch(95% 0.02 85deg / 1);
41 + | --ink: oklch(26% 0.01 250deg / 1);
42 + | --muted: oklch(54% 0.01 185deg / 1);
43 + | --line: oklch(81% 0.02 80deg / 1);
44 + | --panel: oklch(99% 0.01 85deg / 1);
45 + | --button: oklch(33% 0.03 205deg / 1);
46 + | --button-text: oklch(100% 0 0deg / 1);
47 + | --accent: oklch(55% 0.15 35deg / 1);
48 + | --light-square: oklch(89% 0.05 90deg / 1);
49 + | --dark-square: oklch(59% 0.08 150deg / 1);
50 + | --last-move: oklch(80% 0.12 90deg / 1);
51 + | --legal: oklch(43% 0.12 260deg / 1);
52 + | --capture: oklch(52% 0.15 25deg / 1);
53 + | --shadow: 0 18px 42px oklch(24% 0.01 75deg / 0.18);
54 + | --clear-color: oklch(0% 0 0deg / 0);
55 + | --panel-soft: oklch(99% 0.01 85deg / 0.68);
56 + | --panel-raised: oklch(99% 0.01 85deg / 0.74);
57 + | --panel-strong: oklch(99% 0.01 85deg / 0.88);
58 + | --board-edge: oklch(33% 0.03 155deg / 1);
59 + | --last-move-strong: oklch(80% 0.12 90deg / 0.76);
60 + | --last-move-soft: oklch(80% 0.12 90deg / 0.68);
61 + | --legal-marker: oklch(43% 0.12 260deg / 0.42);
62 + | --capture-marker: oklch(52% 0.15 25deg / 0.74);
63 + | --piece-shadow: oklch(0% 0 0deg / 0.2);
64 + | --active-ring: oklch(100% 0 0deg / 0.3);
65 + | --active-halo: oklch(33% 0.03 205deg / 0.18);
66 + | --warning: oklch(69% 0.15 75deg / 1);
67 + | --line-soft: oklch(81% 0.02 80deg / 0.58);
68 + | --line-firm: oklch(81% 0.02 80deg / 0.62);
69 + | --accent-soft: oklch(55% 0.15 35deg / 0.12);
70 + | --button-soft: oklch(33% 0.03 205deg / 0.08);
71 + | --overlay: oklch(21% 0.01 240deg / 0.58);
72 + | }
73 + | @supports (height: 100dvh) {
74 + |   :root {
75 + |     --viewport-height: 100dvh;
76 + |   }
49 77 | }
50 78 | [hidden] {
51 79 |   display: none !important;
52 80 | }
53 81 | .chess-app {
54 - | --board-max-size: 560px;
82 + | --board-min-size: 560px;
83 + | --board-vertical-breathing: 104px;
84 + | --side-columns-width: 628px;
85 + | --grid-gaps-width: 30px;
86 + | --board-size: max(
87 + |   var(--board-min-size),
88 + |   min(
89 + |     calc(
90 + |       var(--viewport-height) - var(--app-padding) - var(--app-padding) -
91 + |       var(--board-vertical-breathing)
92 + |     ),
93 + |     calc(

```

```

94 + | | | | 100vw -- var(--app-padding) -- var(--app-padding) -- var(--side-columns-width) --
95 + | | | | var(--grid-gaps-width)
96 + | | | | )
97 + | | | | )
98 + | | | | );
99 + | | | | --panel-max-height: calc(var(--viewport-height) -- var(--app-padding) -- var(--app-padding));
55 100 | | | | width: 100%;
56 101 | | | | min-width: 1280px;
57 - | | | | height: 720px;
58 - | | | | min-height: 720px;
102 + | | | | min-height: var(--viewport-height);
59 103 | | | | display: grid;
60 - | | | | grid-template-columns: 230px var(--board-max-size) 118px 280px;
104 + | | | | grid-template-columns: 230px var(--board-size) 118px 280px;
61 105 | | | | gap: 10px;
62 106 | | | | align-items: stretch;
63 107 | | | | justify-content: center;
64 108 | | | | overflow: hidden;
65 109 | | | | padding: var(--app-padding);
66 110 | | | | background: var(--page);
67 111 | | | | color: var(--ink);
68 112 | | | | }
69 113 | | | | .board-stage {
70 - | | | | width: var(--board-max-size);
114 + | | | | width: var(--board-size);
71 115 | | | | align-self: center;
72 116 | | | | justify-self: center;
73 117 | | | | }
74 118 | | | | .clock-panel {
75 119 | | | | width: 100%;
76 - | | | | height: var(--board-max-size);
120 + | | | | height: var(--board-size);
77 121 | | | | min-width: 0;
78 122 | | | | min-height: 0;
79 123 | | | | align-self: center;
80 124 | | | | justify-self: stretch;
81 125 | | | | display: flex;
82 126 | | | | }
83 127 | | | | .chess-board {
84 128 | | | | width: 100%;
85 129 | | | | aspect-ratio: 1;
86 130 | | | | display: grid;
87 131 | | | | grid-template-columns: repeat(8, minmax(0, 1fr));
88 132 | | | | overflow: hidden;
89 - | | | | border: 3px solid #2e3a32;
133 + | | | | border: 3px solid var(--board-edge);
90 134 | | | | border-radius: 8px;
91 135 | | | | box-shadow: var(--shadow);
92 - | | | | background: #2e3a32;
136 + | | | | background: var(--board-edge);
93 137 | | | | }
94 138 | | | | .board-square {
95 139 | | | | position: relative;
96 140 | | | | width: 100%;
97 141 | | | | aspect-ratio: 1;
98 142 | | | | display: grid;
99 143 | | | | place-items: center;
100 144 | | | | min-width: 0;
101 145 | | | | padding: 0;
102 146 | | | | border: 0;
103 147 | | | | color: var(--ink);
104 148 | | | | transition:
105 149 | | | | | box-shadow 0.14s ease,
106 150 | | | | | filter 0.14s ease,
107 151 | | | | | transform 0.14s ease;
108 152 | | | | }
109 153 | | | | .board-square:not(:disabled):hover {
110 154 | | | | | filter: brightness(1.05);
111 155 | | | | }
112 156 | | | | .board-square.is-light {
113 157 | | | | | background: var(--light-square);

```

```

114 158 | }
115 159 | .board-square.is-dark-{
116 160 |   background: var(--dark-square);
117 161 | }
118 162 | .board-square.is-selected-{
119 163 |   box-shadow: inset 0 0 0 4px var(--legal);
120 164 | }
121 165 | .board-square.is-last-move-{
122 - |   box-shadow: inset 0 0 0 4px #e0b84bc2;
166 + |   box-shadow: inset 0 0 0 4px var(--last-move-strong);
123 167 | }
124 168 | .board-square.is-selected.is-last-move-{
125 169 |   box-shadow:
126 170 |     inset 0 0 0 4px var(--legal),
127 - |     inset 0 0 0 8px #e0b84bad;
171 + |     inset 0 0 0 8px var(--last-move-soft);
128 172 | }
129 173 | .board-square.is-legal:after,
130 174 | .board-square.is-capture:after-{
131 175 |   position: absolute;
132 176 |   z-index: 1;
133 177 |   content: '';
134 178 |   border-radius: 999px;
135 179 |   pointer-events: none;
136 180 | }
137 181 | .board-square.is-legal:after-{
138 182 |   width: 28%;
139 183 |   height: 28%;
140 - |   background: #264f8f6b;
184 + |   background: var(--legal-marker);
141 185 | }
142 186 | .board-square.is-capture:after-{
143 187 |   inset: 12%;
144 - |   border: 3px solid rgba(184, 50, 50, 0.74);
188 + |   border: 3px solid var(--capture-marker);
145 189 | }
146 190 | .board-square.is-check-{
147 191 |   box-shadow: inset 0 0 0 5px var(--capture);
148 192 | }
149 193 | .chess-board.is-locked .board-square-{
150 194 |   filter: saturate(0.86);
151 195 | }
152 196 | .piece-{
153 197 |   z-index: 2;
154 198 |   display: block;
155 199 |   font-size: 4rem;
200 + |   font-size: max(4rem, var(--board-size) / 8 * 0.92);
156 201 |   line-height: 1;
157 - |   text-shadow: 0 2px 6px rgba(0, 0, 0, 0.2);
202 + |   text-shadow: 0 2px 6px var(--piece-shadow);
158 203 |   -webkit-user-select: none;
159 204 |   user-select: none;
160 205 | }
161 206 | .coordinate-{
162 207 |   position: absolute;
163 208 |   z-index: 3;
164 209 |   font-size: 0.7rem;
210 + |   font-size: max(0.7rem, var(--board-size) / 80);
165 211 |   font-weight: 800;
166 212 |   line-height: 1;
167 213 |   opacity: 0.72;
168 214 |   pointer-events: none;
169 215 | }
170 216 | .coordinate-rank-{
171 217 |   top: 6px;
172 218 |   left: 7px;
173 219 | }
174 220 | .coordinate-file-{
175 221 |   right: 7px;
176 222 |   bottom: 6px;
177 223 | }

```



```

178 224 | .control-panel,
179 225 | .info-panel {
180 226 |   min-width: 0;
181 227 |   min-height: 0;
182 - |   max-height: 664px;
228 + |   max-height: var(--panel-max-height);
183 229 |   display: flex;
184 230 |   flex-direction: column;
185 231 |   gap: 14px;
186 232 |   overflow: auto;
187 233 |   overscroll-behavior: contain;
188 234 |   scrollbar-gutter: stable;
189 235 | }
190 236 | .control-panel {
191 237 |   justify-self: end;
192 238 | }
193 239 | .info-panel {
194 240 |   justify-self: start;
195 241 | }
196 242 | .control-label {
197 243 |   margin: 0;
198 244 |   color: var(--muted);
199 245 |   font-size: 0.78rem;
200 246 |   font-weight: 800;
201 247 |   letter-spacing: 0;
202 248 |   text-transform: uppercase;
203 249 | }
204 250 | .control-group,
205 251 | .status-panel,
206 252 | .ai-candidate-panel,
207 253 | .history-panel {
208 254 |   display: grid;
209 255 |   gap: 10px;
210 256 |   padding-top: 16px;
211 257 |   border-top: 1px solid var(--line);
212 258 | }
213 259 | .history-panel {
214 260 |   min-height: 0;
215 261 |   flex: 1;
216 262 |   grid-template-rows: auto minmax(0, 1fr);
217 263 | }
218 264 | .segmented-control {
219 265 |   display: grid;
220 266 |   grid-template-columns: repeat(2, minmax(0, 1fr));
221 267 |   gap: 4px;
222 268 |   padding: 4px;
223 269 |   border: 1px solid var(--line);
224 270 |   border-radius: 8px;
225 - |   background: #fffaf0ad;
271 + |   background: var(--panel-soft);
226 272 | }
227 273 | .segmented-control-three {
228 274 |   grid-template-columns: repeat(3, minmax(0, 1fr));
229 275 | }
230 276 | .clock-preset-control {
231 277 |   display: grid;
232 278 |   grid-template-columns: repeat(5, minmax(0, 1fr));
233 279 |   gap: 4px;
234 280 |   padding: 4px;
235 281 |   border: 1px solid var(--line);
236 282 |   border-radius: 8px;
237 - |   background: #fffaf0ad;
283 + |   background: var(--panel-soft);
238 284 | }
239 285 | .segmented-button,
240 286 | .primary-button,
241 287 | .ghost-button,
242 288 | .promotion-button {
243 289 |   min-height: 42px;
244 290 |   border-radius: 6px;
245 291 |   font-weight: 800;

```

```

246 292 | letter-spacing: 0;
247 293 | }
248 294 | .segmented-button {
249 295 | border: 0;
250 296 - | background: transparent;
251 297 | background: var(--clear-color);
252 298 | color: var(--muted);
253 299 | }
254 300 | .segmented-button.is-active {
255 301 | background: var(--button);
256 302 | color: var(--button-text);
257 303 | }
258 304 | .clock-settings .segmented-button {
259 305 | min-height: 36px;
260 306 | padding-inline: 4px;
261 307 | font-size: 0.82rem;
262 308 | }
263 309 | .ai-settings {
264 310 | gap: 12px;
265 311 | }
266 312 | .range-control,
267 313 | .toggle-control {
268 314 | display: grid;
269 315 | gap: 8px;
270 316 | min-width: 0;
271 317 | color: var(--ink);
272 318 | font-size: 0.92rem;
273 319 | font-weight: 800;
274 320 | }
275 321 | .range-control span,
276 322 | .toggle-control {
277 323 | align-items: center;
278 324 | }
279 325 | .range-control span {
280 326 | display: flex;
281 327 | justify-content: space-between;
282 328 | gap: 10px;
283 329 | }
284 330 | .range-control strong {
285 331 | color: var(--accent);
286 332 | white-space: nowrap;
287 333 | }
288 334 | .range-control input[type='range'] {
289 335 | width: 100%;
290 336 | accent-color: var(--button);
291 337 | }
292 338 | .toggle-control {
293 339 | grid-template-columns: auto minmax(0, 1fr);
294 340 | padding: 10px;
295 341 | border: 1px solid var(--line);
296 342 - | border-radius: 8px;
297 343 | background: #fffaf0bd;
298 344 | background: var(--panel-raised);
299 345 | }
300 346 | .toggle-control input {
301 347 | width: 18px;
302 348 | height: 18px;
303 349 | accent-color: var(--button);
304 350 | }
305 351 | .action-row {
306 352 | display: grid;
307 353 | grid-template-columns: 1fr 1fr;
308 354 | gap: 10px;
309 355 | }
310 356 | .primary-button,
311 357 | .ghost-button {
312 358 | border: 1px solid var(--button);
313 359 | }
314 360 | .primary-button {
315 361 | background: var(--button);
316 362 | color: var(--button-text);

```

```

315 361 | }
316 362 | .ghost-button {
317 - |   background: transparent;
363 + |   background: var(--clear-color);
318 364 |   color: var(--button);
319 365 | }
320 366 | .primary-button:disabled,
321 367 | .ghost-button:disabled,
322 368 | .segmented-button:disabled {
323 369 |   opacity: 0.46;
324 370 | }
325 371 | .game-status {
326 372 |   min-height: 1.4em;
327 373 |   color: var(--accent);
328 374 |   font-size: 1.35rem;
329 375 |   line-height: 1.2;
330 376 | }
331 377 | .clock-display {
332 378 |   width: 100%;
333 379 |   height: 100%;
334 380 |   min-height: 0;
335 381 |   min-width: 0;
336 382 |   display: flex;
337 383 |   flex-direction: column;
338 384 |   gap: 10px;
339 385 |   justify-content: space-between;
340 386 | }
341 387 | .clock-card {
342 - |   --clock-bg: rgb(255 250 240 / 74%);
388 + |   --clock-bg: var(--panel-raised);
343 389 |   --clock-border: var(--line);
344 390 |   --clock-side: var(--muted);
345 391 |   --clock-time: var(--ink);
346 392 |   width: 100%;
347 393 |   min-block-size: 82px;
348 394 |   max-block-size: 110px;
349 395 |   min-width: 0;
350 396 |   display: grid;
351 397 |   place-content: center;
352 398 |   gap: 4px;
353 399 |   padding: 12px 8px;
354 400 |   border: 1px solid var(--clock-border);
355 401 |   border-radius: 8px;
356 402 |   background: var(--clock-bg);
357 403 |   text-align: center;
358 404 | }
359 405 | #white-clock {
360 - |   --clock-bg: #fffaf0;
361 - |   --clock-border: #c9c0b1;
362 - |   --clock-side: #66716f;
363 - |   --clock-time: #20252b;
406 + |   --clock-bg: oklch(99% 0.01 85deg / 1);
407 + |   --clock-border: oklch(81% 0.02 80deg / 1);
408 + |   --clock-side: oklch(54% 0.01 185deg / 1);
409 + |   --clock-time: oklch(26% 0.01 250deg / 1);
364 410 | }
365 411 | #black-clock {
366 - |   --clock-bg: #263a3c;
367 - |   --clock-border: #172426;
368 - |   --clock-side: rgb(255 250 240 / 74%);
369 - |   --clock-time: #fffaf0;
412 + |   --clock-bg: oklch(33% 0.03 205deg / 1);
413 + |   --clock-border: oklch(25% 0.02 210deg / 1);
414 + |   --clock-side: oklch(99% 0.01 85deg / 0.74);
415 + |   --clock-time: oklch(99% 0.01 85deg / 1);
370 416 | }
371 417 | .clock-card.is-active {
372 418 |   border-color: var(--button);
373 419 |   box-shadow:
374 - |     inset 0 0 0 2px #ffffff4d,
375 - |     0 0 0 2px #263a3c2e;

```

```

420 + | | | inset 0 0 0 2px var(--active-ring),
421 + | | | 0 0 0 2px var(--active-halo);
376 422 | }
377 423 | .clock-card.is-warning .clock-time {
378 - | | | color: #d18b00;
424 + | | | color: var(--warning);
379 425 | }
380 426 | .clock-card.is-danger .clock-time,
381 427 | .clock-card.is-timeout .clock-time {
382 428 | | | color: var(--capture);
383 429 | }
384 430 | .clock-side {
385 431 | | | color: var(--clock-side);
386 432 | | | font-size: 0.72rem;
387 433 | | | font-weight: 800;
388 434 | | | line-height: 1;
389 435 | }
390 436 | .clock-time {
391 437 | | | font-variant-numeric: tabular-nums;
392 438 | | | color: var(--clock-time);
393 439 | | | font-size: 1.35rem;
394 440 | | | line-height: 1;
395 441 | }
396 442 | .status-grid {
397 443 | | | display: grid;
398 444 | | | grid-template-columns: 1fr;
399 445 | | | gap: 8px;
400 446 | | | margin: 0;
401 447 | }
402 448 | .status-grid div {
403 449 | | | min-width: 0;
404 450 | | | padding: 10px;
405 451 | | | border: 1px solid var(--line);
406 452 | | | border-radius: 8px;
407 - | | | background: #fffaf0bd;
453 + | | | background: var(--panel-raised);
408 454 | }
409 455 | .status-grid dt,
410 456 | .status-grid dd {
411 457 | | | margin: 0;
412 458 | }
413 459 | .status-grid dt {
414 460 | | | color: var(--muted);
415 461 | | | font-size: 0.72rem;
416 462 | | | font-weight: 800;
417 463 | }
418 464 | .status-grid dd {
419 465 | | | margin-top: 4px;
420 466 | | | overflow-wrap: anywhere;
421 467 | | | font-size: 0.92rem;
422 468 | | | font-weight: 800;
423 469 | }
424 470 | .history-header {
425 471 | | | display: flex;
426 472 | | | align-items: center;
427 473 | | | justify-content: space-between;
428 474 | }
429 475 | .ai-candidate-panel {
430 476 | | | min-height: 0;
431 477 | }
432 478 | .ai-info-grid {
433 479 | | | display: grid;
434 480 | | | grid-template-columns: repeat(2, minmax(0, 1fr));
435 481 | | | gap: 6px;
436 482 | | | margin: 0;
437 483 | }
438 484 | .ai-info-grid div {
439 485 | | | min-width: 0;
440 486 | | | padding: 8px;
441 487 | | | border: 1px solid var(--line);
442 488 | | | border-radius: 8px;

```

```

443 - | background: #fffaf0bd;
489 + | background: var(--panel-raised);
444 490 | }
445 491 | .ai-info-grid-dt,
446 492 | .ai-info-grid-dd-{
447 493 |   margin: 0;
448 494 | }
449 495 | .ai-info-grid-dt-{
450 496 |   color: var(--muted);
451 497 |   font-size: 0.64rem;
452 498 |   font-weight: 800;
453 499 |   text-transform: uppercase;
454 500 | }
455 501 | .ai-info-grid-dd-{
456 502 |   margin-top: 3px;
457 503 |   overflow: hidden;
458 504 |   font-size: 0.8rem;
459 505 |   font-weight: 800;
460 506 |   line-height: 1.1;
461 507 |   text-overflow: ellipsis;
462 508 |   white-space: nowrap;
463 509 | }
464 510 | .ai-info-grid-dd[id='ai-info-nodes'],
465 511 | .ai-info-grid-dd[id='ai-info-score'],
466 512 | .ai-info-grid-dd[id='ai-info-depth']-{
467 513 |   font-variant-numeric: tabular-nums;
468 514 | }
469 515 | .ai-candidate-header-{
470 516 |   display: flex;
471 517 |   align-items: baseline;
472 518 |   justify-content: space-between;
473 519 |   gap: 8px;
474 520 | }
475 521 | .ai-candidate-meta-{
476 522 |   min-width: 0;
477 523 |   overflow: hidden;
478 524 |   color: var(--muted);
479 525 |   font-size: 0.68rem;
480 526 |   font-weight: 800;
481 527 |   text-align: right;
482 528 |   text-overflow: ellipsis;
483 529 |   white-space: nowrap;
484 530 | }
485 531 | .ai-candidate-table-{
486 532 |   min-height: 0;
487 533 |   overflow: hidden;
488 534 |   border: 1px solid var(--line);
489 535 |   border-radius: 8px;
490 - | background: #fffaf0bd;
536 + | background: var(--panel-raised);
491 537 | }
492 538 | .ai-candidate-list-{
493 539 |   max-height: 172px;
494 540 |   overflow: auto;
495 541 | }
496 542 | .ai-candidate-row-{
497 543 |   display: grid;
498 544 |   grid-template-columns: minmax(3rem, 1.3fr) 2.6rem 3.2rem 3.2rem;
499 545 |   gap: 6px;
500 546 |   align-items: center;
501 547 |   min-width: 0;
502 548 |   padding: 6px 8px;
503 - | border-bottom: 1px solid rgba(201, 192, 177, 0.58);
549 + | border-bottom: 1px solid var(--line-soft);
504 550 |   font-size: 0.72rem;
505 551 |   font-weight: 800;
506 552 |   line-height: 1.15;
507 553 | }
508 554 | .ai-candidate-row:last-child-{
509 555 |   border-bottom: 0;
510 556 | }

```

```

511 557 | .ai-candidate-head {
512 558 |   color: var(--muted);
513 - |   background: #fffaf0e0;
514 559 + |   background: var(--panel-strong);
515 560 |   font-size: 0.66rem;
516 561 |   text-transform: uppercase;
517 562 | }
518 563 | .ai-candidate-row.is-selected {
519 - |   background: #b94b361f;
520 564 + |   background: var(--accent-soft);
521 565 | }
522 566 | .ai-candidate-row.is-principal:not(.is-selected) {
523 - |   background: #263a3c14;
524 567 + |   background: var(--button-soft);
525 568 | }
526 569 | .ai-candidate-san,
527 570 | .ai-candidate-score,
528 571 | .ai-candidate-weight,
529 572 | .ai-candidate-probability {
530 573 |   min-width: 0;
531 574 |   overflow: hidden;
532 575 |   text-overflow: ellipsis;
533 576 |   white-space: nowrap;
534 577 | }
535 578 | .ai-candidate-score,
536 579 | .ai-candidate-weight,
537 580 | .ai-candidate-probability {
538 581 |   text-align: right;
539 582 |   font-variant-numeric: tabular-nums;
540 583 | }
541 584 | .ai-candidate-badge {
542 585 |   display: inline-block;
543 586 |   margin-left: 4px;
544 587 |   padding: 1px 3px;
545 588 |   border-radius: 4px;
546 589 |   background: var(--button);
547 590 |   color: var(--button-text);
548 591 |   font-size: 0.56rem;
549 592 |   line-height: 1;
550 593 |   vertical-align: 1px;
551 594 | }
552 595 | .move-list {
553 596 |   min-height: 0;
554 597 |   max-height: none;
555 598 |   overflow: auto;
556 599 |   border: 1px solid var(--line);
557 600 |   border-radius: 8px;
558 - |   background: #fffaf0bd;
559 601 + |   background: var(--panel-raised);
560 602 | }
561 603 | .move-row {
562 604 |   display: grid;
563 605 |   grid-template-columns: 3rem minmax(0, 1fr) minmax(0, 1fr);
564 606 |   gap: 8px;
565 607 |   padding: 8px 10px;
566 - |   border-bottom: 1px solid rgba(201, 192, 177, 0.62);
567 608 + |   border-bottom: 1px solid var(--line-firm);
568 609 |   font-weight: 800;
569 610 | }
570 611 | .move-row:last-child {
571 612 |   border-bottom: 0;
572 613 | }
573 614 | .move-number {
574 615 |   color: var(--muted);
575 616 | }
576 617 | .move-san {
577 618 |   min-width: 0;
578 619 |   overflow-wrap: anywhere;
579 620 | }
580 621 | .move-empty {
581 622 |   margin: 0;

```

```

577 623 | padding: 14px 12px;
578 624 | color: var(--muted);
579 625 | font-weight: 700;
580 626 | }
581 627 | .promotion-overlay {
582 628 | position: fixed;
583 629 | inset: 0;
584 630 | z-index: 10;
585 631 | display: grid;
586 632 | place-items: center;
587 633 | padding: 20px;
588 - | background: #15191c94;
    634 + | background: var(--overlay);
589 635 | }
590 636 | .promotion-overlay[hidden] {
591 637 | display: none;
592 638 | }
593 639 | .promotion-dialog {
594 640 | width: 360px;
595 641 | display: grid;
596 642 | gap: 16px;
597 643 | padding: 20px;
598 644 | border: 1px solid var(--line);
599 645 | border-radius: 8px;
600 646 | background: var(--panel);
601 647 | box-shadow: var(--shadow);
602 648 | }
603 649 | .promotion-dialog h2 {
604 650 | margin: 0;
605 651 | font-size: 1.1rem;
606 652 | }
607 653 | .promotion-choices {
608 654 | display: grid;
609 655 | grid-template-columns: repeat(4, minmax(0, 1fr));
610 656 | gap: 8px;
611 657 | }
612 658 | .promotion-button {
613 659 | min-height: 70px;
614 660 | border: 1px solid var(--button);
615 - | background: #fff;
    661 + | background: var(--button-text);
616 662 | color: var(--ink);
617 663 | font-size: 2.4rem;
618 664 | line-height: 1;
619 665 | }

```